

LIFE OS Unified Implementation Contract

Orchestrator handoff for backend, UI, assets, migration and verification

STATUS Additive merged contract. It combines frozen backend scope, frozen and partially open UI scope, full visual references, unresolved decision gates, and repository-preservation rules. It is not a claim that any listed work is already implemented or verified.

Part One Unified orchestration plan

This document is the single execution entry point for an orchestrator. It does not erase or paraphrase away either source package. Every unique source paragraph, table, task, route, acceptance criterion, open question and figure remains in Parts Two through Four. The orchestration layer below adds sequencing and ownership; it does not broaden approval.

The orchestrator must inspect the live repository before assigning work, preserve newer working behavior, and stop on a material conflict. Document text describes target behavior and task constraints; only the live code, migrations, tests and runtime evidence establish what currently exists.

1 Authority and source preservation

Authority	How it controls execution	Treatment in this document
Latest explicit user FREEZE	Highest authority for approved product and visual decisions. A later explicit freeze may supersede an earlier one.	Preserved and reflected in the orchestration map.
Live Life OS repository and AGENTS rules	Implementation truth for routes, services, schemas, permissions, tests and current Phengos milestone continuity.	Must be audited before edits; no silent replacement of newer working behavior.
Part Two Frozen UI implementation	Detailed visual, interaction, route-migration and figure contract.	Copied in full with every embedded figure and table.
Part Three Backend and agent runtime contract	Frozen platform, integrations, domain backends, agents, workflows, tests and acceptance order.	Copied in full and assigned through disjoint backend workstreams.
Part Four UI roadmap and open decision register	Tracks status, unfinished design areas, update protocol and backend discussion gates.	Copied in full; OPEN and PROPOSED items remain blocked from implementation.
Embedded figures and Photos assets	Visual evidence or approved direction according to each figure label.	All 25 figures remain embedded. Written corrections override image artifacts and sample data.

2 Repository continuity rules

- A request for usable navigation means Phengos milestone 2. It does not authorize a new three-dimensional scene or advancement into later milestones without the required feedback gate.
- Preserve every routed workflow in docs/PHENGOS_FEATURE_INVENTORY.md until replacement behavior is implemented, discoverable and regression-tested.
- Calendar and Kitchen are migrations of working products, not rewrites. Fitness, Finance, Movies, Chat, Settings and Developer work must extend current services and routes rather than create parallel frameworks.

- Canonical domain services own writes. UI components and agents use typed commands, version checks, confirmation and tenant boundaries. No frontend-only mutation engine and no direct provider access from agents.
- The existing outbox, world revision and Server-Sent Events transport remain the cross-domain invalidation path unless current code proves an equivalent canonical mechanism.

3 Unified orchestrator workstreams

ID	Scope and source linkage	Dependencies and dispatch gate	Disjoint ownership
W0	Repository audit, contract reconciliation, baseline screenshots and migration inventory.	Runs first. Report conflicts; do not write product code.	Orchestrator only.
W1	Shared shell, direct navigation, Self to Chat migration, contextual PHENGOS overlay, shared neon tabs. UI NAV-001, CHAT-001, PHENGOS-002, MOTION-001.	After W0. Preserve milestone 2 and every routed feature.	App shell, router, navigation and shared assistant presentation only.
W2	Provider-neutral Settings and Vault credentials for API-Football, Plaid, food, image and TMDB; Letterboxd import Settings entry.	After W0. Unblocks provider-backed domains.	Integration schemas, backend credential store/routes and Settings integration UI.
W3	Calendar migration plus Besiktas next-match projection and Calendar binding.	W1 shell; W2 API-Football configuration; existing Planner remains canonical.	Calendar frontend and standing-calendar sports adapter/service/tests.
W4	Kitchen migration, food catalog, nutrition provenance and semantic food/training media cache.	W1 shell; W2 provider keys. Coordinate with Fitness and Finance through typed contracts.	Kitchen UI, food provider adapters, catalog/media models and services.
W5	Fitness frozen UI, existing training/measurement flows and approved backend integrations.	W1 shell; W4 nutrition/media. FIT-BE-01/02/03 remain blocked pending user approval.	Fitness frontend and existing Fitness service extensions only.
W6	Finance frozen UI and provider-neutral connected banking with qualification gates.	W1 shell; W2 credentials. FIN-BE-01/02 remain blocked pending user approval.	Finance frontend, BankDataProvider/Plaid adapter, normalization and sync.
W7	Movies Overview/Watchlist/Logs, TMDB catalog/posters, Letterboxd baseline import and typed movie tools.	W1 shell; W2 TMDB credential. MOV-BE-01/02 remain blocked pending user approval.	Existing Movies backend/frontend only; no duplicate catalog or importer.
W8	Agent jobs, triggers, completion ledger, leases, waits, JEV typed decisions, provider-call attribution and costs.	After W0; supplies runtime contracts to W9 and domain agents.	Canonical agent runtime, cost ledger and invalidation events.
W9	Developer control area, Agent Brain projection and Workflow Studio.	Depends on W8. Backend capabilities may proceed; final Developer visual design remains OPEN where the UI roadmap says so.	Developer projections/UI, memory graph and workflow definitions; no second scheduler or memory database.
W10	Cross-workstream integration, migrations, security, visual	Runs at each milestone and owns final verification.	Orchestrator-managed integration only; domain

ID	Scope and source linkage	Dependencies and dispatch gate	Disjoint ownership
	regression, accessibility and end-to-end acceptance.		owners repair their own defects.

4 Domain dependency crosswalk

Surface	Frozen UI and figures	Backend and cross-domain contract
Chat and PHENGOS	CHAT-001, PHENGOS-002, CH01-CH06; white global Chat orb and domain-colored contextual overlay.	Reuse canonical assistant threads, role registry, proposals, permissions and bounded typed context. Agent runtime work is W8; no second chatbot.
Calendar and sports	CAL-001, C01-C02; Overview, Daily List, Planning; preserve grid and insight rail.	Planner remains canonical. API-Football supplies one next Besiktas fixture through standing-calendar binding; no AI call.
Kitchen	KIT-001, K01-K08; five workspaces and corrected content-aware overview.	Food catalog and media are deterministic. Fitness publishes nutrition target; Finance exposes budget through typed read; Kitchen never reads their tables directly.
Fitness	FIT-001, FIT01-FIT03; deterministic exercise-specific anatomy focus and correct chart axes.	Reuse FitnessGoal, BodyMeasurement, programs, sessions and sets. New anatomy taxonomy, trajectory formula and corrections wait at explicit decision gates.
Finance	FIN-001, FIN01-FIN02; bank-first overview and one navigation strip.	Connected banking normalizes accounts, balance snapshots and transactions. CSV remains fallback until qualified production connection and initial sync. Liability/forecast expansions remain gated.
Movies	MOV-001, MOV01-MOV02; Overview, Watchlist, Logs and poster-first catalog.	Existing Movies service remains canonical. TMDB and Letterboxd are deterministic server integrations. Custom lists/favorites and rating policy remain gated.
Settings	Preferences, Integrations and Agent Settings; Developer remains separate.	Write-only Vault credentials, status/test/remove routes, Letterboxd import, TMDB attribution and provider health.
Developer	Independent control plane; detailed final visual design remains open.	Agent state, costs, schedules, governed memory graph and constrained workflow composition read/write canonical backend contracts.

5 Open backend decisions that must not be dispatched

The UI freeze does not authorize the following schemas, formulas or endpoints. The orchestrator records them as BLOCKED_DECISION until the user explicitly approves their contracts. Existing models and read projections remain the first option.

Ticket	Unapproved extension	Required decision
FIT-BE-01	Versioned anatomy taxonomy and primary/secondary exercise-to-muscle mapping.	Granularity, source, weighting, correction and proposed/approved state.

Ticket	Unapproved extension	Required decision
FIT-BE-02	Goal trajectory normalization, forecast and estimated crossing.	Metric definitions, minimum evidence, confidence, horizon and display semantics.
FIT-BE-03	Additional measurements and historical correction/edit behavior.	Allowed fields, source provenance, audit and retention policy.
FIN-BE-01	Klarna/card liabilities, installments, statements and payment reconciliation.	Provider/manual source, accounting ownership and double-count prevention.
FIN-BE-02	Historical balance series, backfill and cash-flow forecast.	Snapshot authority, time buckets, assumptions and projected labeling.
MOV-BE-01	Pinned favorites and ordered named movie lists.	Membership/order, limits, CRUD and privacy behavior.
MOV-BE-02	Rating summary/distribution and monthly activity projections.	Per-viewing versus per-film policy, rewatch treatment, filters and buckets.

6 Asset authority and handling

- Every image from the UI package remains embedded in Part Two and retains its stable figure ID, caption, role and mandatory correction. Full-resolution originals remain available in the supplied Photos directory.
- CURRENT and EXISTING images document the present implementation. TARGET and FROZEN images control composition and interaction only where the written contract says so. CONCEPT images are directional. HISTORICAL images remain traceability evidence and never override the latest freeze.
- Amounts, dates, meals, measurements, dialogue, posters and status in visual references are illustrative unless the live repository and canonical data prove otherwise.
- Agents must open the full-resolution asset assigned to their workstream and record any deliberate responsive deviation in the acceptance report. Generated or provider media follows the licensing, provenance and cache contracts in Part Three.

7 Execution order and integration gates

1. Run W0 and produce a current implementation map, conflicts list, baseline screenshots and targeted test commands.
2. Complete W2 integration credentials and the minimum W8 runtime/cost foundations needed by provider-backed and agent work.
3. Implement W1 shared shell and navigation without removing routed workflows; stop at the milestone feedback gate.
4. Run W3, W4, W5, W6 and W7 only with disjoint ownership. Each domain must preserve current data and pass its own backend, frontend and visual acceptance before integration.
5. Implement W9 over W8 canonical state. Do not finalize an unapproved Developer visual design or introduce a second scheduler, graph database, memory copy or credential store.
6. At every merge boundary, W10 verifies migrations, route compatibility, tenant isolation, secrets, idempotency, events, loading/empty/error states, accessibility and responsive screenshots.
7. Run the complete milestone acceptance order in Part Three. A workstream is not VERIFIED from an agent completion message; it requires recorded tests, runtime evidence and visual review.

8 Reading and execution rule for the preserved parts

Parts Two through Four are preserved source contracts, not competing documents. References inside them to LIFE_OS_UI_IMPLEMENTATION.docx, LIFE_OS_UI_ROADMAP.docx or Next Implementation File (1).docx now refer to the corresponding preserved part of this unified contract. Their original wording remains so no context or provenance is lost.

If two preserved statements appear inconsistent, use the authority table and explicit status labels first. Do not delete one statement during implementation. Record the conflict, identify whether it is an approved target, current implementation fact, historical reference or unapproved proposal, and ask the user only when the distinction does not resolve the work safely.

Part Two Frozen UI implementation and visual contract

The following UI master is preserved in full, including its tables, task IDs, acceptance tests, status vocabulary and 25 embedded reference figures.

LIFE OS — UI IMPLEMENTATION

Cumulative frozen specifications · component responsibilities · figures · verification gates

Living master document • Oct 8, 2026 • No per-freeze filenames

STATUS: design specification. No application code has been modified or verified by this package.

How to use this implementation contract

This is the SINGLE cumulative UI implementation document. Approved future feature/workspace designs extend or amend this document in place. It is written for coding agents to implement against the existing Fivina/life-os repository, without rebuilding existing working Calendar/Kitchen engines or inventing parallel data ownership.

IMPLEMENTATION STATUS Design frozen. Application code was not changed as part of this package. A Codex completion claim is insufficient without tests, runtime validation and visual review.

Inputs	Authority
User's latest FREEZE	Controls approved visual/interaction decisions.
This UI Implementation document	Controlling detailed UI target, supersedes prior UI implementation fragments.
LIFE_OS_UI_ROADMAP.docx	Tracking and future queue, not permission to ship speculative features.
Photos/	Visual evidence and design direction, subordinate to written contract.
Repository Fivina/life-os	Current code and API/runtime implementation truth.
Next Implementation File (1).docx from original ZIP	Planned backend and agent runtime implementation sequence; no duplicate subsystems.

01 Non-negotiable migration principles

- UPDATE, DO NOT REBUILD: Calendar and Kitchen are existing implementations. Preserve valid components, hooks, API calls, stored data, authorization and workflows; change composition and behavior deliberately.
- No demo data hardcoded into production. Generated concept screens contain illustrative food names, dates, inventory amounts and claims. Always bind canonical API DTOs and honest loading/empty/error states.

- Do not delete any lower tool until equivalent behavior is implemented, discoverable and regression-tested in its new workspace.
- All canonical data modifications must use existing domain services, permission constraints, idempotency/version handling, and confirmation where required. An attractive button cannot bypass approval.
- Current repo is the implementation source of truth; planned routes and model integrations in Next Implementation File are dependencies, not currently shipped behavior.
- Treat navigation, action, assistant and inspection as different interaction categories: only navigation changes workspace; edit/log/approve is a contextual action; PHÉNGOS is one orchestrated assistant surface; inspect opens relevant detail.

02 Universal shell, copy, colors and visual grammar

02.1 Visual preservation

- Keep original cinematic space scene and near-black / dark navy translucent glass panels, restrained blue edge highlights, glowing horizon, narrow floating sidebar and existing circle construction.
- Do not replace the original orb with a newly designed blue chat bubble. Reuse shared sidebar PHÉNGOS visual/component/CSS tokens for the in-page launcher.
- Retain existing global sidebar and shared visual tokens; do not recolor previously frozen Calendar amber/yellow, Kitchen blue/cyan, Learning violet, universal Chat white, or global Life navigation without its own approval. For the APPROVED NEW workspaces, apply the contextual visual accents shown in the frozen figures: Fitness training/neon focus green, Finance amber/gold with semantic financial series colors, Movies purple/violet. Red is reserved for genuine alert/error or semantically negative financial values, never an ornamental default.
- The contextual PHÉNGOS launcher inherits the CURRENT WORKSPACE accent while retaining the ORIGINAL sidebar orb shape/ring: Calendar amber, Kitchen blue/cyan, Fitness green, Finance gold, Movies violet. Global Chat sidebar stays white-accented; white means luminous outline/theme, NOT a white chat background. For Finance/Movies, the GLOBAL Life sidebar destination remains active; do not invent Finance or Movies global icons.
- Dark glass remains readable; no giant opaque overlay that hides the space identity when unnecessary. Avoid gratuitous neon proliferation or animated glows that imply nonexistent AI activity.

02.2 Universal header rule

Title and icon are enough. REMOVE promotional/descriptive slogan subtitles beneath the title, including “Plan your time. Align your life.” and “Plan the meal. Know what you have.” Preserve meaningful functional instructions, labels, status, empty messages, unit information, warnings and accessible descriptions. Do not replace with another motto.

The repeated top-right PAGE DATE CHIP is removed from Calendar and Kitchen. In that same upper-right utility location place the domain-colored PHÉNGOS trigger. This does NOT delete operational dates inside the Calendar grid, date navigator, meal-journal day selector or event timestamps.

02.3 Layout invariants and glass cards

- One global sidebar, one contextual subnav per domain. No pre-workspace launcher pages, duplicate sidebar feature menus or permanent giant raw form sections beneath the dashboard.
- Cards use stable internal padding and minmax responsive grid tracks. Cards may resize by content state, but do not reorganize unexpectedly while focused/dragged/edited.
- An empty state is compact and truthful; no huge blank panel caused by one item in a card dimensioned for dozens.

- Cards have compact/normal/wide patterns; text truncation must show an accessible full value. Date, minute, quantity and nutrition values use correct units and user locale.
- An element's active state must be visible beyond color alone (text/icon/underline/aria-current); contrast, keyboard focus and reduced-motion settings remain supported.

03 Navigation and route consolidation [NAV-001, DEV-001]

Layer	Expected behavior	Do not do
Global sidebar	Home · Calendar · Chat · Learning · Fitness · Life · Kitchen, utilities Search and Settings; Developer separate technical module.	Do not navigate through a sidebar feature flyout to reach main domain.
In-domain subnav	One compact local strip with a domain's approved subcomponents.	Do not repeat global sidebar or create a standalone pre-dashboard menu.
Cards / actions	Open actual details or focused composer inside appropriate domain.	Do not use giant decorative card launchers for navigation.
Developer	Independent route/control plane; not Settings or Life.	Do not create second agent runtime or bury under Settings.
Legacy links	Keep old bookmarked `/self/assistant` and `/assistant` functional and resolve to Chat.	Do not discard user's existing conversations, IDs or draft state.

03.1 Specific duplicated navigation to remove

- The current `PhengosNavigation.tsx` sidebar main-domain `pn-feature-panel` expansion duplicates domain navigation: one click on the primary domain must go directly to actual workspace.
- `PhengosHome.tsx` currently contains a separate `phengos-domain-grid` domain-selection interstitial (“Calendar / components” etc). Bypass/remove this hop from main-domain navigation; retain the intentionally designed cinematic Home intro/idle state.
- Use one route model (App.tsx and phengosFeatures.ts) to preserve aliases, Back/Forward, active state and deep links; avoid ghost menu definitions that drift out of sync.
- Independent Developer controls may use their own inner navigation when designed, but cannot become Settings subitems by default.

03.2 Approved domain subnav map

Main module	Local destinations / notes
Home	Home dashboard itself (no redundant Overview).
Calendar	Overview · Daily List · Planning. Inside Overview, Day/Week/Month is a view selector.
Kitchen	Overview · Fridge · Meals · Recipes · Shopping. Approved tab strip K08.
Learning	Dashboard · Study · Exams · Study Log · Candidates; precise design deferred.
Fitness	Dashboard · Training · Progress · Nutrition; precise design deferred.
Life	Dashboard · Goals · Finance · Social · Movies; precise design deferred.
Settings	Preferences · Integrations · Agent Settings; utility, not Developer.
Developer	Standalone control plane; exact UI/mode gating not yet approved.
Notebook	Existing route remains until final placement decided.

04 PHÉNGOS and Chat [CHAT-001, PHENGOS-002]

04.1 Distinct surfaces, one runtime

The GLOBAL Chat sidebar icon is the existing PHÉNGOS orb with WHITE glow/accent and navigates intentionally to the dedicated Chat workspace. There, the user can access general PHÉNGOS conversation, existing threads and registered domain specialists (Calendar, Fitness, Kitchen, Learning, Life as supported by actual backend role configuration). The global Chat workspace is not a floating overlay and is not a second chatbot backend.

A PAGE-LEVEL PHÉNGOS launcher lives in the top-right header utility slot of Calendar and Kitchen. Its orb has the same design/construction but inherits that page's color; this is context and not a new autonomous agent instance. Clicking opens in-page contextual chat. A single shared conversation/thread service and agent routing remain canonical.

04.2 Exact collapsed → expanded interaction

State	Required UI behavior
Closed	Top-right launcher visible, with orb and concise PHÉNGOS label/affordance; NO bottom bar in Calendar/Kitchen.
Opening	Anchor a compact floating dark-glass chat window near the top-right trigger. Position within viewport; opening must not change domain route or dashboard width.
Expanded	Overlay sits above content with its own scroll region, PHÉNGOS thread messages, typed composer and close/minimize. Underlying dashboard remains full width; temporary occlusion is intentional.
Message sent	Keep same page, active subnav/date and scroll; submit using existing chat stream tools and attach validated current domain/entity context where supported.
Proposal returned	Show preview/explanation and confirmation controls. Do not claim a canonical mutation succeeded until domain service confirms.
Collapsed/closed	Return focus to PHÉNGOS trigger, preserve thread/message history and unsent draft; reopen the same context when appropriate.
Navigation away	Thread persists in canonical store; transition to intentional global Chat only when chosen, not because user used page composer.
Narrow/mobile	Floating window becomes a viewport-contained dialog/sheet OVER the current page; no forced domain change or unusable underlying columns.

04.3 Geometry and accessibility

- At desktop, prefer approximately 360–440 CSS px chat width and a bounded height $\leq 75\text{--}80\text{vh}$, adjusted to actual available room; floating layer overlays, never reserves a grid column.
- Position within safe viewport margins, avoid covering launcher close button, use scroll inside message pane; the page behind remains at original width and scroll state.
- Window header has labeled Close and Minimize. Keyboard: trigger Enter/Space opens; Escape closes unless an explicit confirmation dialog owns Escape; focus moves to composer/first relevant control and returns on close.

- Accessible role, descriptive title, focus containment when modal on mobile, safe inert/aria semantics if sheet-style; announcements for new responses and confirmation outcomes without aggressive live-region spam.
- No repeated bottom composer; only the expanded window composer is active. No focus mode, no extra permanent full chat column, no route change on send.
- One canonical conversation history with multiple addressable threads; changing specialist context cannot overwrite or falsely relabel previous messages.

04.4 Relevant current code and backend boundaries

Code owner (inspect at implementation time)	Responsibility
<code>apps/web/src/features/phengos/navigation/PhengosAssistantBar.tsx</code>	Remove from Calendar/Kitchen composition; transfer send semantics to reusable top-right trigger/window; preserve other consumers pending audit.
<code>apps/web/src/layouts/AppShell.tsx</code>	Own shared overlay portal/trigger placement without duplicating domain state.
<code>apps/web/src/features/assistant/AssistantPage.tsx</code>	Reuse existing thread list/streaming/agent selection in full Chat; no new second assistant.
<code>apps/web/src/app/App.tsx</code> ; <code>phengosFeatures.ts</code> ; <code>PhengosNavigation.tsx</code>	Canonical routes, selected state, legacy /self redirect, sidebar orb and white Chat accent.
<code>apps/web/src/services/api.ts</code> ; <code>backend/app/api/routes_assistant.py</code>	Audit typed context payload/role names and tool approvals before extending; no uncontrolled client mutation.

04.5 Main Chat workspace — migration from Self [FROZEN]

This is an UPDATE of the existing assistant experience, not a new chatbot application and not a separate specialist backend. Figure CH01 shows the two-region Chat Hub idea (thread list + main conversation). Figure CH02 shows the more detailed dedicated Chat page with a general/specialist agent selector. Both are conceptual screenshots whose sidebar captions, logo art, and old bottom workspace bars are NOT authoritative.

A. Global sidebar replacement

- Replace the visible sidebar label “Self” with “Chat” at the exact existing primary destination position; the primary entry goes directly to the full Chat workspace (suggested canonical path `/chat`, checked against the live route table). Remove the legacy Self entry from the rendered primary sidebar: there must NOT be both “Self” and “Chat”.
- Render the existing shared PHÉNGOS sidebar orb artwork/component, not a speech-bubble icon or new generated logo. In the Chat destination its active highlight, rim/glow, hover and selection transition use luminous WHITE. Retain the dark navy/glass surface; WHITE is the semantic accent, not a white page background.
- Leave old `/self/assistant`, `/assistant` and existing deep links backward compatible through an intentional redirect/adaptation to the canonical Chat screen. Preserve thread identifiers, drafts, role context and link navigation where available; do not create redirect loops or destroy history.
- Main Chat destination must be reached in one click from any workspace without an intermediate agent landing menu, sidebar flyout, or Home → domain selector. No Focus Mode or duplicated secondary Chat app.

B. Primary Chat page: visible regions and hierarchy

- LEFT conversation list within the Chat workspace (not another global sidebar): grouped/recent persisted threads, selected state, clear New chat action, search/filter where backend supports it,

names/previews/timestamps sourced from real thread metadata. Empty state offers New chat without inventing past conversations.

- CENTER main conversation canvas: scrollable message history, user/PHÉNGOS messages, agent/tool or actionable result cards only when grounded, inline approval/reject/review UI for mutations, loading/error/retry states, one bottom composer in the FULL Chat page. Conversation remains usable while tool or reasoning activity is pending.
- Agent/context selector IN the Chat page header or conversation header: General / PHÉNGOS (default) or explicitly selected available domain specialist. Examples from the approved exploration include Calendar, Fitness, Kitchen, Learning, Life; only advertise actual configured roles/capabilities returned by the current backend. Selection does not create permanent additional global sidebar destinations.
- General PHÉNGOS may dispatch a request to appropriate existing specialist tools/agents using existing validated orchestration and permissions; display useful source/agent attribution where reliably known. A user-selected specialist narrows the working scope; it must not grant out-of-domain privileges.
- Thread history is shared by the canonical conversation service but consists of MULTIPLE independently addressable threads. A new chat starts a new thread; selecting a thread resumes that thread and its recorded role context. Do not force a single endless cross-domain transcript and do not duplicate stores per specialist.
- Changing agents within or between conversations must make the new scope apparent and cannot retroactively relabel earlier turns, merge conversations, discard unsent text, or silently change historical ownership. If live backend lacks role transitions within a thread, require an explicit new/specialist thread transition rather than simulating unsupported switching.
- The Chat hub can intentionally open an existing contextual conversation from Calendar/Kitchen, if a canonical thread ID is available, without erasing prior messages. Clicking global Chat is an intentional route change; clicking the in-page PHÉNGOS launcher is NOT.
- Respect the earlier request to speak to PHÉNGOS: preserve a voice-capable input affordance/space only if it is actually supported in the current implementation; simple turn-based voice belongs to the later voice roadmap. Do not implement full-duplex Moshi or claim voice is active as part of this UI migration.
- Small standalone information labels are functional, not slogans. Omit marketing subtitles such as “Plan your time. Align your life.” and decorative Chat taglines.

C. Contextual assistant vs full Chat — one behavior contract

- Calendar and Kitchen trigger: upper-right page utility position formerly occupied by decorative date chip; the identical PHÉNGOS sidebar orb structure takes the ACTIVE DOMAIN glow (Calendar yellow/amber; Kitchen blue/cyan) while full Chat nav stays WHITE.
- Clicking the page trigger opens a SMALL floating dark-glass chat window over the full-width unchanged page. No bottom bar and no reserved/docked chat column. Window may occlude cards while open; minimize/close returns exactly to the same scroll, selected date/tab and grid measurements.
- Context is scoped by the active page and user-selected entity/date/recipe only when validated by backend typed contract. Show “Calendar context” / “Kitchen context” truthfully; never fabricate information or assume every prompt is a command.
- Inside the popup, one message composer, message history for the attached thread, minimized/collapsed state and an optional explicit “Open in Chat” action to navigate to the main Chat hub. No automatic navigation to Self/Chat on submit.
- Existing approval/proposal rules apply to every domain mutation: assistant may inspect, explain or propose; writing changes only through authorized typed services with proper confirmation and accurate action status.
- Window styling: dark smoked-glass, thin appropriate accent edges, restrained light falloff and smooth motion; never white chat panel fill, never a new bright-blue PHÉNGOS icon merely because a design board shows one.

D. Visual implementation / authority of chat figures

- CH01 is the approved concept DIRECTION for conversation list + central chat + General/specialist selection; replace its illustrative thread contents, tab labels and blue PHÉNGOS art with real data and frozen white Chat theme.
- CH02 is the approved DIRECTION for a Chat workspace with agent selector, contextual tools and durable conversation history; its old Calendar screenshot and any marketing copy do not amend the later Calendar/Kitchen freeze.
- CH03 illustrates a SHARED PHÉNGOS conversation system, context-aware routing and optional domain-scoped threads; diagram arrows are conceptual, not proof of concrete runtime routes or permission delegation.
- CH04 illustrates a bounded context package (active workspace, selected item/date, user preferences, permitted data/tools). Pass only validated minimum needed to existing backend; never use client-side role labels as authority or transmit unrestricted private domain state.

E. Agent/code migration and acceptance gate

- Inspect actual current HEAD before editing:
apps/web/src/features/phengos/navigation/PhengosNavigation.tsx, navigationModel.ts, phengosFeatures.ts, apps/web/src/app/App.tsx, apps/web/src/features/assistant/AssistantPage.tsx, navigation/PhengosAssistantBar.tsx and backend assistant routes/agents/tools + conversation/thread services. Use existing API/role primitives where present; no parallel runtime.
- Acceptance: exactly one Chat global sidebar destination labelled “Chat”, with WHITE orb highlight; no visible Self. Navigating to Chat shows the conversation list and main conversation; new/resumed/specialist threads work with canonical stored IDs and tenant isolation.
- Acceptance: switching general/specialist context is explicit and truthful, existing history is not lost/mixed/relabelled, only real registered agents show; General mode can route through existing validated orchestration without unauthorized writes.
- Acceptance: Calendar/Kitchen launchers use page color and same orb, open floating overlap, do not change route/grid/card dimensions, preserve tab/day/scroll/draft; bottom chat bars are absent on both.
- Acceptance: keyboard/focus, Escape, overlay stacking, narrow screen sheet behavior, announcement of actual result state, error/disconnect/retry, reduced motion and cross-domain browser Back/Forward are verified before marking IMPLEMENTED/VERIFIED.

05 Calendar [CAL-001] — existing implementation update

05.1 Preserve what already works

- Current Calendar grid/timeline with day/week/month switching, previous/today/next date navigation and scheduled event rendering (Figure C01).
- Right-side insight rail: contextual Today items, Attention, PHÉNGOS-related actionable summary, Week Allocation and small commitment month; deduplicate content by making cards summaries linking to detailed workspaces.
- Planner and calendar backend projections, standing rules, candidate actions and canonical event/commitment data; do not replace with client-calculated fake scheduling.

05.2 Move existing lower tools to the appropriate workspace

Destination	What belongs here	Interaction rule
Overview	Day/week/month grid + insight rail, event details.	Click event → in-place detail/editor; no extra dashboard/launcher.

Destination	What belongs here	Interaction rule
Daily List	Today's canonical commitments and plan blocks, status, attention queue, Why here?, start/complete/partial/skip/remove placement where authorized.	Chronological execution view; don't duplicate summary cards as raw lists.
Planning	Planning pool, intentions, plan proposals, week map, standing calendar rules, replan tools and variant feasibility.	Intentions/commitments create/edit via contextual composer, no giant fixed forms below main grid.

05.3 PHÉNGOS replacement and date

DELETE the bottom PHÉNGOS composer from the Calendar page. REPLACE the generic upper-right date/status chip with the page-accent (amber/yellow) orb PHÉNGOS launcher. Calendar's functional date navigation, selected date, visible week and event times stay. Figure C02 illustrates the floating window over the unchanged working grid; any leftover date chip or subtitle in a concept image is an artifact to correct, NOT a target.

The chat can discuss a selected time block, report overload or propose a replan, but scheduling changes must still follow canonical Planner/approval contracts. Never hardcode an "I updated your plan" success from the concept image.

05.4 Existing interaction defects and migration

- The current lower `Save edit` action is not a proper edit flow in the inspected source; build a real editor that allows changing a value, submits changes and reports version/conflict errors.
- Current Add Commitment / Add Intention forms use example seed values. Replace with empty/contextual defaults and focus in relevant workspaces.
- Do not destroy event selection, scroll position, selected week/day or ongoing task when opening/closing PHÉNGOS.
- Retain deep-link compatibility, route behavior, keyboard navigation, and accurate server-sourced attention status.

05.5 Calendar component mapping

Existing code paths	Agent operations
<code>apps/web/src/features/calendar/CalendarPage.tsx</code>	Decompose permanent lower tool stack into Overview/Daily List/Planning; preserve mutations with test parity.
<code>apps/web/src/features/calendar/CalendarWorkspace.tsx</code>	Keep grid and insight rail; adapt header utility slot and details interaction.
Calendar CSS and shared shell	Do not shrink grid when chat opens; remove bottom chat reservation; maintain breakpoint behavior.
Planning route/service hooks and api client	Keep underlying planner authority, not a second scheduling engine.

06 Kitchen [KIT-001] — existing implementation update

06.1 Scope: preserve, relocate, enhance

The existing KitchenOverview and KitchenPage already connect Chef, inventory, meal planning, nutrition, receipts, recipes and shopping (Figures K01 and K02). The target is a cohesive update, NOT replacement with a new app. The old long "Kitchen tools" stack ceases to be the default scroll destination; its capabilities move to the five approved subworkspaces, and service integrations remain.

The agent/Planner prepares a relevant meal/next action through existing canonical services. Kitchen displays the result and lets the user understand, accept, cook, inspect or correct it. The UI must not independently invent suggestions or write unapproved changes. Any “PHÉNGOS suggestion” strip displays a real backend-backed proposal or a truthful empty/no-plan state.

06.2 Kitchen workspace navigation strip

The approved strip is shown in Figure K08 and throughout K03–K07. Render immediately beneath the Kitchen title, fully within Kitchen rather than as a second global nav. Items in order: ****Overview · Fridge · Meals · Recipes · Shopping****. Active item has a narrow luminous blue underline/highlight and subtle edge reflection; inactive items remain dark glass with readable labels and monochrome icons. Do not insert History or Manual Entry as sixth/seventh tabs. They live within Meals/Recipes/Fridge.

- Selecting one tab updates the active Kitchen workspace without forcing an intermediate menu or unrelated sidebar expansion.
- Support direct/deep URLs (e.g. one canonical route or query/hash mapping per subcomponent), browser Back/Forward and selected-state on refresh.
- The state change uses the slow, elegant fade detailed in §08; no hard cut or flashy pulsing.

06.3 Overview — the central daily surface (K03)

The visual reference K03 is approved for card composition, material, information hierarchy and top navigation. Its blank void under Today’s meal is an identified concept defect and **MUST NOT** be copied into production. All user data in the image is illustrative.

Card	Canonical content / actions	Sizing intent
Today’s meal	Agent-prepared or selected current meal; real recipe/servings/availability; Start cooking / View recipe; fallback no prepared meal state.	Primary visual anchor, not two grid rows tall when content is short.
Nutrition today	Consumed versus target from Kitchen/Fitness nutrition services; units and confirmed-source distinctions.	Shares first main row height with Today’s meal and journal.
What I have eaten today	Chronological actual meals, not unconsumed planned meals; View all jumps to Meals history.	Visible, compact 2–4 rows before continuation.
Fridge preview	Item count, ingredients and storage/expiry relevance; View all opens Fridge.	Content-bounded; one ingredient does not create a giant empty cell.
Expiring soon	Only actionable expiring lots, truthful expiry state and quantity.	Compact 2–4 rows; collapse/empty state on zero.
Recent recipes	Recently cooked/used/saved recipes; history and repeat selection.	Horizontal content cards, bounded height, no filler tile count.
Shopping connection	Actual active needs/aggregated list, next 2–3 days as optional forecast; Open list goes to Shopping.	Compact; no forced 7-day display.
Quick actions	Log meal, Add ingredient, Scan receipt, Open cooking/recipe as supported.	Contextual buttons with a working handler, not extra main destinations.

06.4 Exact dashboard reflow requirement: fix blank under Today’s meal

Implement a cohesive aligned 12-column CSS Grid or equivalent with content-aware rows. Suggested desktop starting composition (adjust for screenshot parity, not hard pixel constraints):

- Row A: Today's meal spans 5/12 columns; Nutrition 4/12; Food journal 3/12. Cards align at top AND bottom, share a content-appropriate min-height rather than an excessively tall fixed height.
- Row B: Fridge spans 5/12, Expiring 4/12, Quick actions 3/12. Each row wraps and fills its own useful content.
- Row C: Recent recipes spans 7/12; Shopping connection spans 5/12. Never let the largest hero span two rows unless deliberately filled with content.
- Alternative local adjustment: if the meal image becomes wider, redistribute Row A as 5/4/3 or 6/3/3 but do NOT leave orphaned empty space below it. Agent must compare rendered screenshot at actual laptop/wide widths.
- Use ``minmax(0,1fr)`` tracks, ``align-items:stretch``, contained card body scroll if needed, and ``grid-auto-rows:auto`` or explicit auto row sizing. Do not combine a 2-row hero span with one-item adjacent utility cards.
- At width below roughly 1280px, reflow to two columns or stacked cards as appropriate. At narrow/mobile, one column. These are implementation starting breakpoints, not new exact freeze constants.

06.5 Fridge workspace (K04)

- Primary content: searchable inventory cards (category filter, quantity, unit, expiry state, staples/low stock). Select item → contextual item details/editor in workspace (not a giant permanent form on Overview).
- Add ingredient, edit quantity, remove/consume, set staple thresholds and attach expiry. Reuse existing inventory endpoints with optimistic version/error states.
- Receipt scan uses existing receipt upload/process/review/confirm flow; recognition requires review before inventory mutation.
- Expiry/low-stock/staple summaries should be derived from canonical lots and thresholds. Do not invent static freshness labels from mockup data.

06.6 Meals workspace (K05)

- Today's meal and actual eaten-food journal are primary. Keep planned/selected/cooked/eaten distinctions visually explicit.
- Browse meals by day with chronological entries and a later-month history/search view; user can find recipes from last month and repeat an authentic past meal.
- Manual meal logging is a compact interaction inside Meals. Repeat from recipe/history is one-click prefill followed by approval/check, not automatic double-log.
- Optional "Maybe next" and next 2–3 days can exist but must not dominate; no required weekly meal scheduling.
- A selected recipe offers cooking session with ingredient steps and progress; persisted `ActiveWorkspace` state should restore/rejoin after route change/reload once verified.
- IMPORTANT backend gap: repository has `MealHistory` creation and completion, but inspected `routes_kitchen.py` do not expose dedicated historical `MealHistory` list/GET. Add/read an owned paginated history contract before claiming full history works; do not spoof by scraping plan list.

06.7 Recipes workspace (K06)

- Saved recipes, all recipes, filters/search, selected recipe detail, steps/ingredients/nutrition and quick Cook/Save action.
- Recipe creation/editing is within this workspace, not a giant permanent Overview form. Preserve existing recipe API and validate edits/version handling.
- Recent and favorite/repeat information should derive from actual recipe and meal history; no fake "used 3x" figure.
- Recipe imagery uses current media subsystem/cache and the media policy from Next Implementation File; show designed fallback until ready. Do not generate a new food image on every visit.

- Some mockup features such as import-from-photo and expanded difficulty/competency metadata are DESIGN REFERENCES/PLANNED dependencies unless verified in code. Show correctly gated actions until service exists.

06.8 Shopping workspace (K07)

- Prioritized real shopping needs, grouped by reason; indicate item source (selected recipe, low staple threshold, explicit/manual, upcoming short-horizon meal) and due state.
- Aggregation comes from backend canonical needs; mark purchased with the existing purchase flow, optionally linking actual inventory mutation only through service.
- Manual add as focused input within Shopping; no permanent lower-page admin form.
- Next 2–3-day forecast is OPTIONAL and should not create duplicate speculative needs each time a view opens.
- History only if backed by persistent purchase records/read API; do not invent purchasing history.

06.9 Kitchen service mapping and planned additions

Existing / planned capability	Agent instruction
KitchenPage.tsx / KitchenOverview.tsx	Refactor layout and route/page grouping around existing queries and mutations; preserve core data flow.
GET /api/v1/kitchen/status; inventory; recipes; recommendations; meal-plans	Use normalized existing data as source of truth. Keep all endpoint behavior and user scope.
POST /api/v1/kitchen/meals/manual; recipes/{id}/complete	Write meal history using canonical services, not local fake state.
GET /api/v1/kitchen/nutrition/progress	Bind Today nutrition to confirmed meal records and target; display missing targets honestly.
GET /api/v1/kitchen/shopping-list; shopping-needs; shopping-forecast; purchases	Bind shopping card, short-horizon forecast and checklist with correct side effects.
POST/PATCH /api/v1/kitchen/cooking-sessions	Use persisted workspaces; add/verify restoration UI path on reload.
MealHistory list/search read API	NEEDED for rich Food Journal/history; define paginated bounded query backed by existing MealHistory, with ownership and tests.
Food catalog, Open Food Facts / USDA, media semantic cache	PLANNED in Next Implementation File; integrate provider-neutral adapters, provenance and caching; do not rebuild under UI package.

07 Existing data and planned features: important distinctions

Evidence type	Can UI claim this works now?	Display treatment
Source-coded and validated by current route	Only after runtime test	Show actual server data with loading/error/empty states.
Present in Next Implementation File as planned future backend	No	Mark future dependency and keep action disabled/hidden until available.
Present only in concept image	No	Visual structure only; substitute correct real data and behavior.
User approved interaction	Design frozen, not code complete	Implement and verify before labeling shipped.

Examples: meal history is canonical data already written, but a usable browse API may still be missing; Open Food Facts/USDA and semantic food media belong to the user’s next backend implementation file; the agent preparing meals is an orchestration behavior and must not be recreated as an autonomous frontend recommendation engine. Recipe cards should gracefully handle no media and no history.

08 Motion and responsive presentation [MOTION-001]

08.1 Slow, elegant contextual Kitchen tab transitions

- The active Kitchen tab transitions through a subtle luminous blue underline/border and a small translucent blue wash. Highlight brightens gently then settles; do not make it pulse indefinitely.
- Recommended starting durations: tab text/underline opacity about 220–320ms; glass background tint 280–420ms; content view crossfade 160–260ms. Use one shared easing curve (e.g. cubic-bezier(0.22,1,0.36,1)); adjust after actual UI visual QA.
- For crossfade, use opacity and at most a 4–8px translate; avoid width/height animation that causes dashboard reshuffle. Never transition canonical data values as if the backend is “thinking”.
- On `prefers-reduced-motion: reduce`, eliminate decorative transitions; keep instant visible active state and stable focus/ARIA.
- Active item is still recognizable without glow (underline, weight, icon and aria-current). Keyboard focus ring is distinct from selected state.
- Navigation tab changes should not trigger double loading, replay stale animation on every background cache refresh, or remount an active editor unexpectedly.

08.2 Floating chat animation and behavior

- Open chat with a gentle opacity/scale entrance anchored near trigger; the Calendar/Kitchen underlying grid must remain pixel-identical before and after opening.
- Opening should not move/resize the underlying cards, bottom-scroll the page, or obscure the window’s close/submit controls.
- Restoring an existing thread should preserve scroll state and draft; loading states should be truthful and accessible, not a synthetic glow pulse.
- On mobile, overlay may cover workspace temporarily. The user remains on the same route; keyboard composer must stay visible when the software keyboard is open.

09 Visual register and figure authority

ID	Full resolution file under Photos/	Role	Mandatory correction / note
N01	N01_sidebar_duplicate_flyout_CURRENT.png	Evidence	Remove duplicate sidebar feature flyout for selecting a main domain.
N02	N02_domain_interstitial_CURRENT.png	Evidence	Remove unnecessary domain-selection page before actual workspace.
C01	C01_calendar_existing_CURRENT.png	Existing screenshot	Preserve grid/rail functionality, remove bottom chat.
C02	C02_calendar_phengos_floating_CONCEPT.png	Approved interaction direction	Shows overlapping chat; move trigger to top-right and remove ANY old date chip or subtitle artifacts.

ID	Full resolution file under Photos/	Role	Mandatory correction / note
K01	K01_kitchen_existing_CURR ENT.png	Existing screenshot	Preserve existing working services, not the outdated long layout.
K02	K02_kitchen_old_tools_CURR ENT.png	Evidence	Rehome lower manual forms; do not delete capability.
K03	K03_kitchen_overview_TARGET .png	Approved dashboard direction	Fix identified blank under Today's meal via 12-col repack. Illustrative dish and numbers.
K04	K04_kitchen_fridge_TARGET.p n	Approved target	Ingredient editor can be contextual; sample inventory is illustrative.
K05	K05_kitchen_meals_TARGET.p n	Approved target	Actual journal + light near-term meals; invented history data not canonical.
K06	K06_kitchen_recipes_TARGET.p n	Approved target	Recipe library/detail structure; photo import/progress may need backend work.
K07	K07_kitchen_shopping_TARGE T.png	Approved target	Short-horizon, priority groups; invented items/history not canonical.
K08	K08_kitchen_subnav_detail_US ER.png	User-supplied detail	Authoritative one-strip tab order and soft fade tone; reuse in K03–K07.
CH01	CH01_chat_hub_genera l_threads_CONCEPT.pn g	Earlier reviewed Chat exploration (concept crop)	Concept crop: thread list, central chat, General/specialist tabs. Correct illustrated PHÉNGOS icon + labels to frozen sidebar orb WHITE Chat accent; sample dialogue illustrative.
CH02	CH02_chat_hub_agent selection_CONCEPT.pn g	Earlier reviewed Chat exploration (concept crop)	Concept crop: full main Chat page, agent mode switch, context panels, action cards and thread history; not proof of APIs or approved voice implementation.
CH03	CH03_chat_continuity_ model_CONCEPT.png	Earlier reviewed Chat exploration (concept crop)	Concept crop: one shared assistant/runtime, optional role/context-specific threads. Never duplicate per-agent storage or blindly auto-merge threads.
CH04	CH04_chat_context_mo del_CONCEPT.png	Earlier reviewed Chat exploration (concept crop)	Concept crop: bounded workspace context; old screen details and old bottom bar are

ID	Full resolution file under Photos/	Role	Mandatory correction / note
			superseded. Use validated typed context and canonical authority.
CH05	CH05_original_chat_system_design_board_HISTORICAL.png	Historical source	Full original chat catalog; HISTORY ONLY. Old bottom dock, old orb/chat colors, fixed single endless chat claims and launcher styling are superseded; use CH01 and active §04 contract.
CH06	CH06_original_chat_workspace_design_board_HISTORICAL.png	Historical source	Full original workspace/chat exploration; HISTORY ONLY. Old date tile, bottom chat bar, generic marketing subtexts and docked sidechat are superseded; use CH02–CH04 and active §04 contract.

READ EVERY FIGURE WITH THIS RULE: source screenshot = existing implementation evidence; approved target = composition and interaction direction; figure text/numbers/photo content = illustrative until backed by repository. The written contract overrides image artifacts. Do not claim image pixel-fidelity if implementing responsive UI has different actual data.

10 Implementation sequencing and migration safety

Phase	Work / dependencies	Exit gate
A	Audit actual current frontend routes/screens, shared PHÉNGOS comp, APIs, tests and server version.	No missing capability list; screenshot baseline captured.
B	Unify global sidebar destinations and bypass flyout/interstitial while retaining Home intro/legacy links.	Direct-route and history tests pass.
C	Introduce shared page-level PHÉNGOS launcher and floating chat component; replace Calendar/Kitchen bottom docks and header date chips.	Overlay never changes underlying layout; single composer and canonical conversation continuity.
D	Recompose Calendar existing tools into Overview/Daily List/Planning; repair bad edit interaction.	Calendar events/plan proposals/edits and duplicate-state tests pass.
E	Recompose Kitchen Overview card grid and build Fridge/Meals/Recipes/Shopping views using existing queries.	No blank under Today's meal; actions work; no raw tool stack.
F	Add audited backend read contracts for Kitchen meal history when needed; integrate food/catalog/media planned milestones when available.	No falsely working affordances; owned APIs and provenance tests.

Phase	Work / dependencies	Exit gate
G	Visual and accessibility QA; screenshot comparisons at real widths and deep link refresh; rollback plan.	User-accepted UI and all regression tests pass.

11 Acceptance tests (required before VERIFIED)

11.1 Navigation and assistant

- [] Each primary sidebar destination opens its real workspace in one action with no flyout or intermediary domain page.
- [] Global Chat appears with existing orb in WHITE accent; old Self/assistant URL remains accessible to its threads.
- [] Calendar page PHÉNGOS orb uses amber accent; Kitchen page PHÉNGOS orb uses Kitchen blue accent; both are top-right in former date-chip utility slot.
- [] Neither Calendar nor Kitchen displays a bottom PHÉNGOS bar after migration.
- [] Opening chat overlays without changing page width, card layout, grid column widths, scroll or route.
- [] Chat close/minimize/reopen preserves conversation, draft and focus return, with one active composer.
- [] Calendar selected week/day and Kitchen active tab survive chat interaction; role/context message bound to user and active entity only.
- [] Proposed writes require actual validation/confirmation before success toast; no fabricated tool outcomes.
- [] Sidebar name Self is REPLACED by Chat, not duplicated; icon retains original PHÉNGOS orb at white Chat glow and opens canonical main Chat hub.
- [] Main Chat renders real persisted conversation list, new thread, resume and explicit General or specialist selector from actual backend capabilities.
- [] General auto-routing respects tool permissions; specialist selection cannot rewrite previous turns, merge threads or silently switch stored roles.
- [] Page-level overlay can optionally open its same canonical thread in the Chat hub, and Calendar/Kitchen no longer navigate to legacy Self when sending.

11.2 Calendar

- [] Calendar grid day/week/month, Today controls, event details, insight rail and deep links work.
- [] Daily List/Planning expose all current lower functions with parity; no raw stack under Overview.
- [] Save edit actually changes editable value, handles conflict/error, and has tests.
- [] No example-form defaults or ornamental subtitle.

11.3 Kitchen

- [] Five Kitchen tabs appear in exact order and switch/deep-link without a second global menu.
- [] Kitchen Overview matches approved K03 hierarchy but has NO stranded blank under Today's meal at realistic content counts.
- [] Fridge inventory search/edit/expiry/staples/receipt review work through real APIs.
- [] Meals logs actual consumed entries; history list is available only with real backed read API; optional next 2–3 days never forces weekly planning.
- [] Recipes browse/detail/cook/create preserve existing functions; media has placeholders/cache provenance.
- [] Shopping list and purchase flows use canonical needs and short-term sources; no invented history.
- [] No sample fixture meal/ingredient data hardcoded or mislabeled as actual personal state.

- [] Kitchen tabs fade gently, focus and reduced motion work; no layout shift/flicker.

11.4 System QA

- [] Automated existing frontend/backend tests re-run; reported actual commands/results and any failures.
- [] Desktop wide, laptop, tablet and phone sizes verified with screenshots; no content clipping or inaccessible overlays.
- [] Keyboard-only, screen reader landmarks and names, aria-current, focus/Escape, Back/Forward and reload verified.
- [] No duplicate API requests/events, silent mutation, broken permissions or orphaned legacy routes.
- [] Known backend dependencies logged without falsely claiming implementation or changing the source of truth.

12 Source and file ownership index

Component	Likely owner / inspect before editing
Router / compatibility	apps/web/src/app/App.tsx; apps/web/src/features/phengos/phengosFeatures.ts
Sidebar / duplicate menu	apps/web/src/features/phengos/navigation/PhengosNavigation.tsx; navigationModel.ts; navigation.css
Shell and global chat overlay	apps/web/src/layouts/AppShell.tsx; apps/web/src/features/assistant/AssistantPage.tsx; navigation/PhengosAssistantBar.tsx
Calendar	apps/web/src/features/calendar/CalendarPage.tsx; CalendarWorkspace.tsx; calendar CSS / hooks
Kitchen	apps/web/src/features/kitchen/KitchenPage.tsx; KitchenOverview.tsx; kitchen CSS / hooks
Shared DTOs	apps/web/src/services/api.ts; apps/web/src/types/api.ts
Kitchen backend	backend/app/api/routes_kitchen.py; backend/app/domains/kitchen/service.py; schemas.py
Assistant/backend context	backend/app/api/routes_assistant.py; agent runtime / tool registry; inspect exact paths in current HEAD.
Backend roadmap dependencies	Next Implementation File (1).docx food/nutrition, media, agent runtime contracts
Main Chat / Self migration	assistant/AssistantPage.tsx, routes in app/App.tsx, phengos/navigation/PhengosNavigation.tsx and navigationModel.ts, backend assistant thread/role APIs. Verify names against current HEAD.

13 Change log within this one document

Date	Approved cumulative content
2026-10-08	Navigation and Developer independence; Chat sidebar white / dedicated main workspace; Calendar update; PHÉNGOS top-right colored orb replacing date chip; no bottom bar; Kitchen five-tab update with images; universal no-slogan rule; subtle fade.
Future	Append one dated row on next explicit FREEZE, modifying sections/images without changing filename.

Date	Approved cumulative content
2026-10-08 · chat audit	Explicit Self→Chat sidebar replacement, main Chat workspace and history/agent selection, Chat→context handoff, 4 concept reference plates and regression tests; no prior Calendar/Kitchen freeze changed.

14 Photo reference plates — full-sized originals remain in Photos/

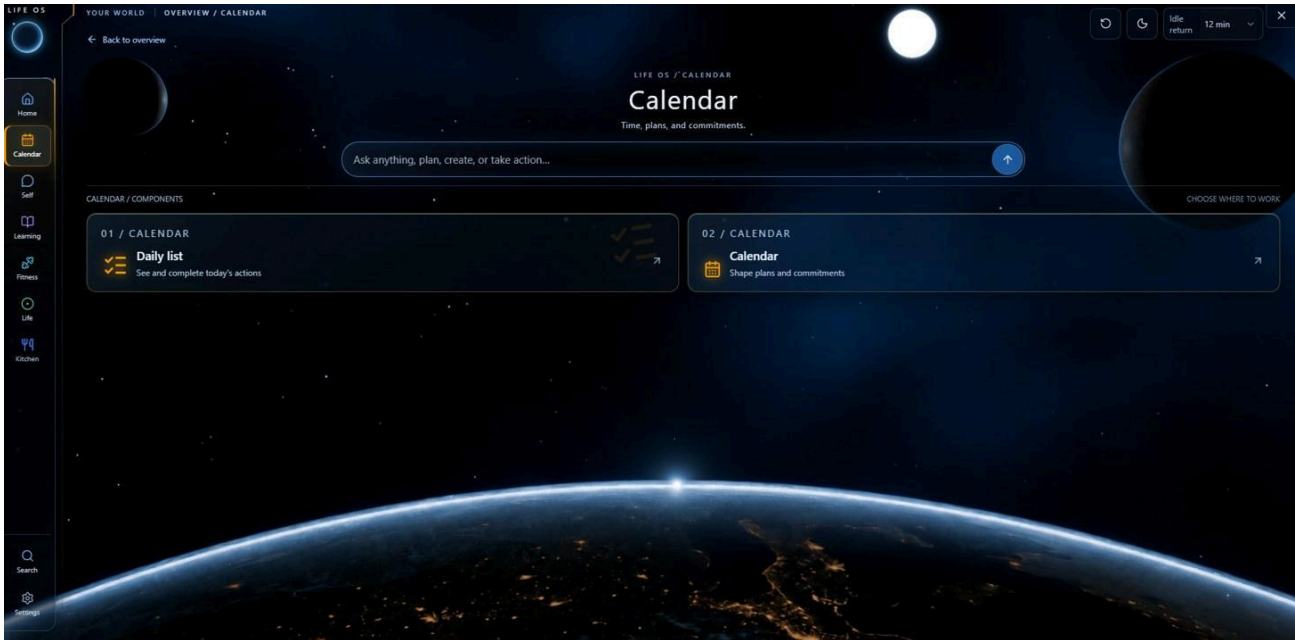
Figures below are compressed readability previews embedded in this document. Coding agents should open the full-resolution same-named files in Photos/ for text, card density and layout details. Every concept image is subordinate to the written rules above.

FIGURE N01 | Photos/N01_sidebar_duplicate_flyout_CURRENT.png



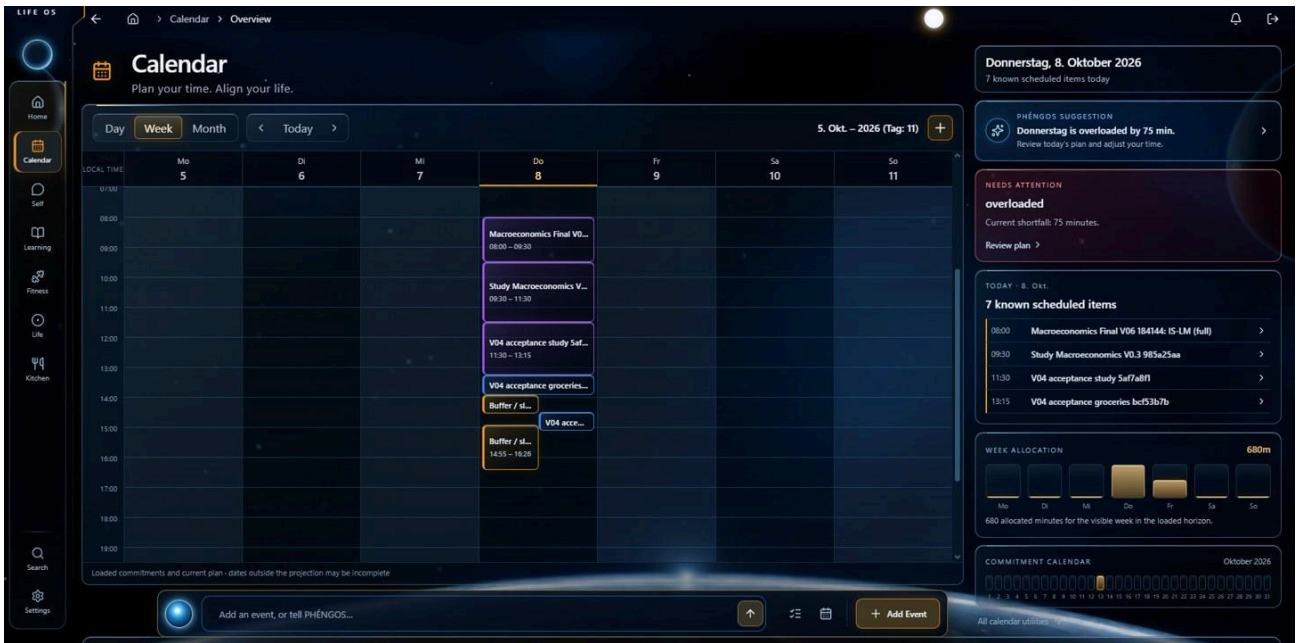
Existing duplicate sidebar panel (remove main-domain feature flyout; not the whole sidebar).

FIGURE N02 | Photos/N02_domain_interstitial_CURRENT.png



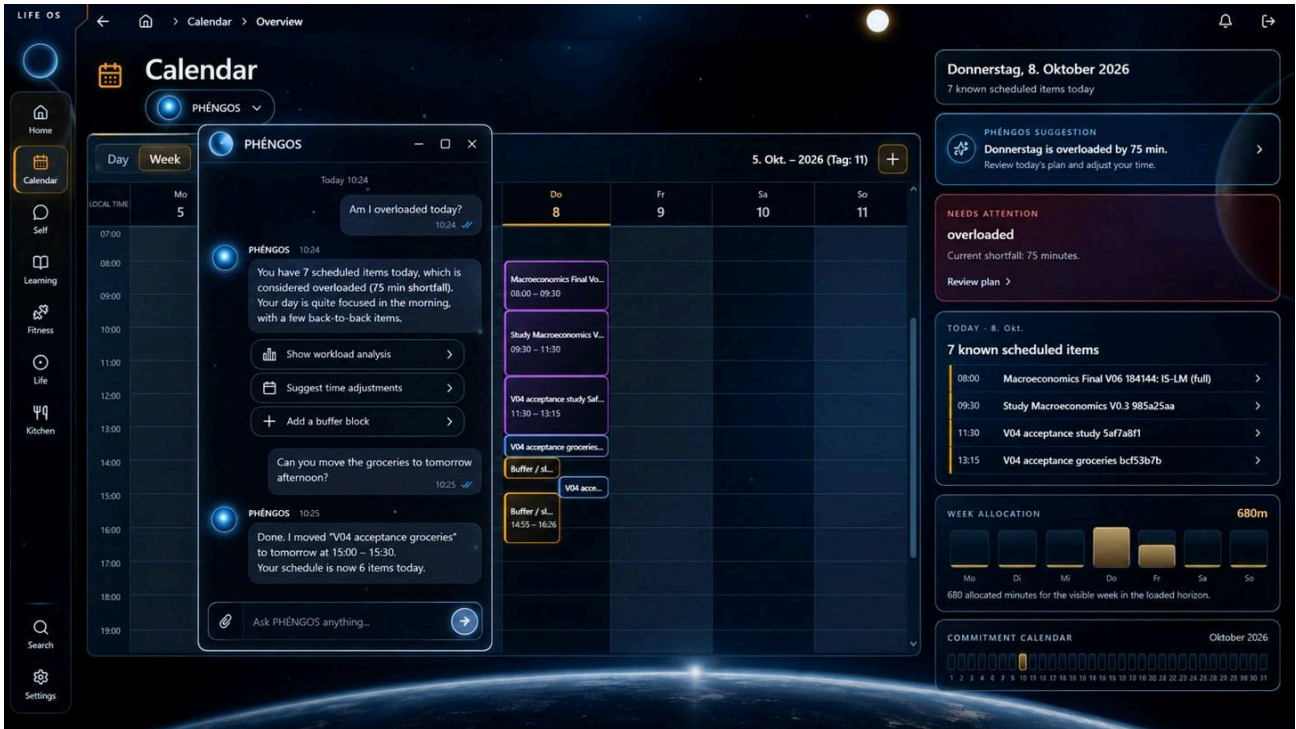
Existing redundant domain-launcher page; bypass while keeping real Home design.

FIGURE C01 | Photos/C01_calendar_existing_CURRENT.png



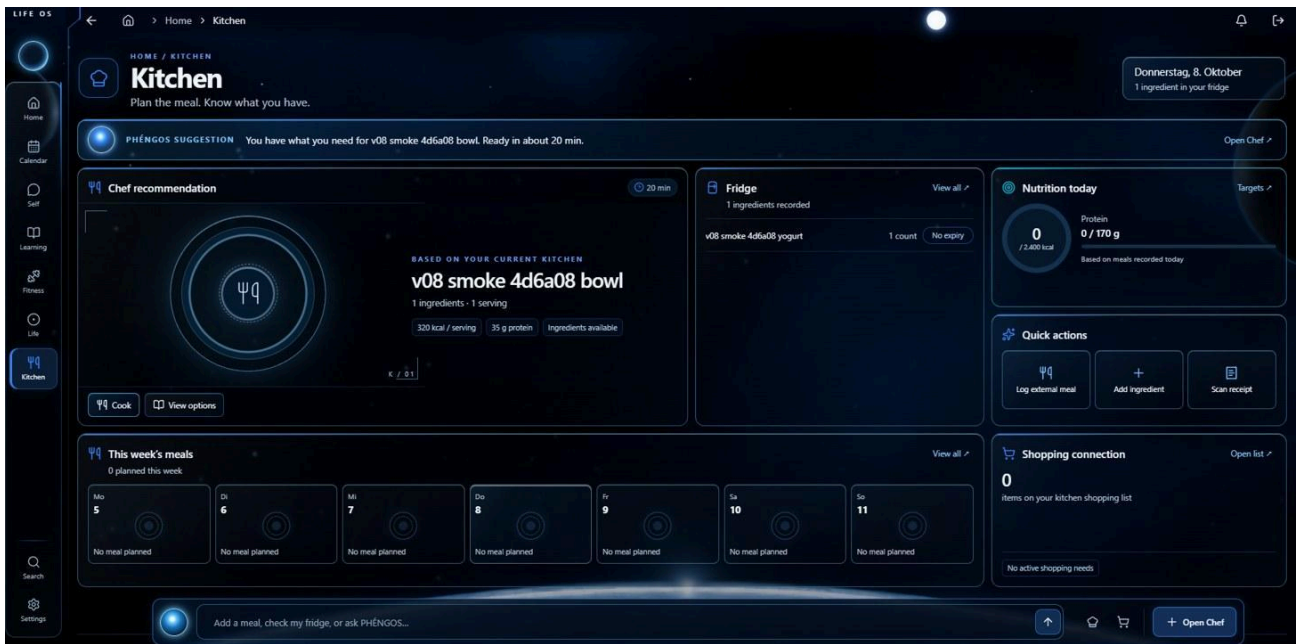
Existing Calendar screenshot showing the working grid and right insight rail. Update, do not rebuild.

FIGURE C02 | Photos/C02_calendar_phengos_floating_CONCEPT.png



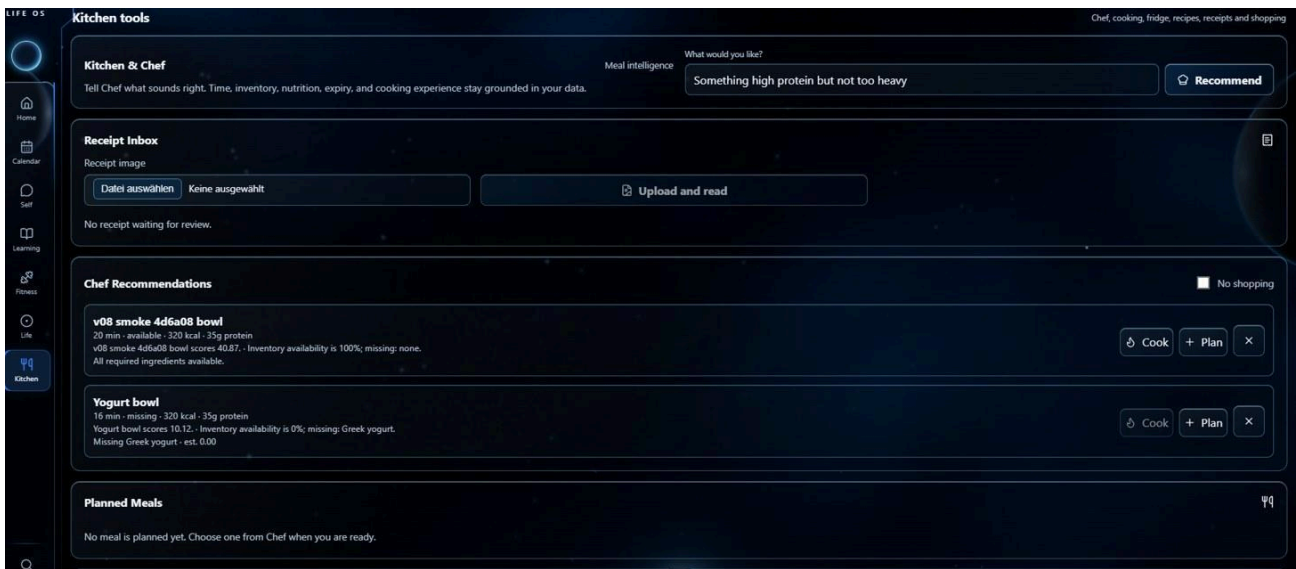
Floating contextual chat over Calendar. Concept still has old header artifacts; relocate trigger top-right, remove bottom bar/date chip/subtitle.

FIGURE K01 | Photos/K01_kitchen_existing_CURRENT.png



Kitchen current overview baseline. Preserve service-backed value.

FIGURE K02 | Photos/K02_kitchen_old_tools_CURRENT.png



Current raw tool stack; migrate functions to focused subpages, never silently delete tools.

FIGURE K08 | Photos/K08_kitchen_subnav_detail_USER.png



User-provided local Kitchen nav strip reference; approved order and restrained fading active tab.

FIGURE K03 | Photos/K03_kitchen_overview_TARGET.png



Approved Overview layout. Fix underfilled space under Today's meal with aligned content-aware grid.

FIGURE K04 | Photos/K04_kitchen_fridge_TARGET.png



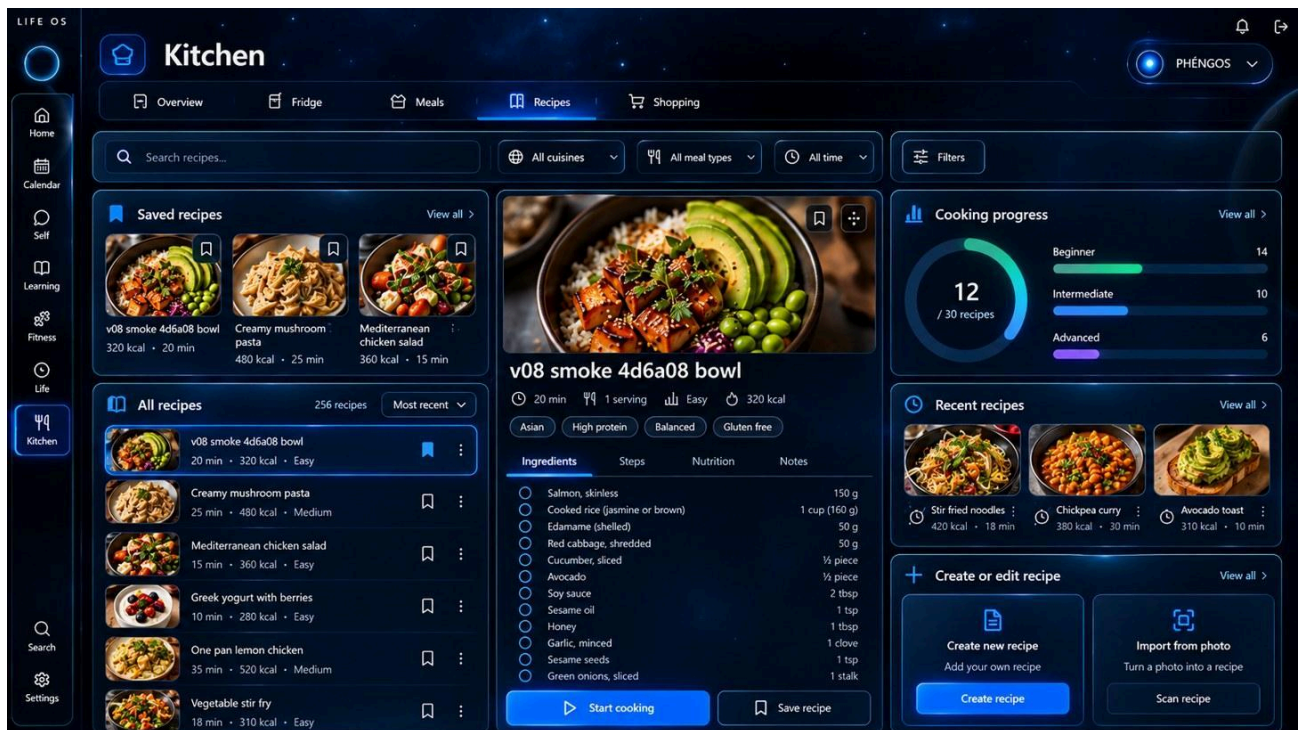
Fridge cards/detail and focused manual changes. Fake food records remain illustrative.

FIGURE K05 | Photos/K05_kitchen_meals_TARGET.png



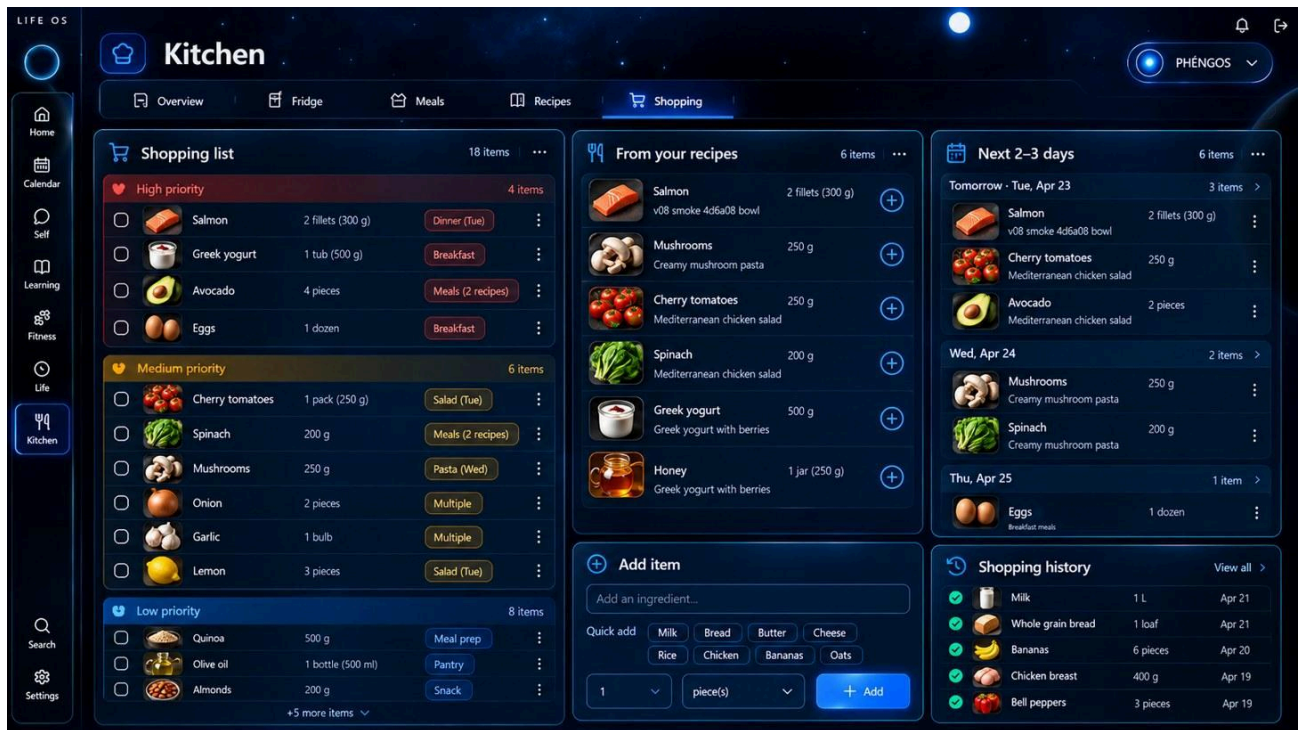
Meals journal, current meal and optional next meals. No mandatory weekly planning.

FIGURE K06 | Photos/K06_kitchen_recipes_TARGET.png



Recipe library/search/detail and contextual creation. Disable unimplemented media/import paths.

FIGURE K07 | Photos/K07_kitchen_shopping_TARGET.png

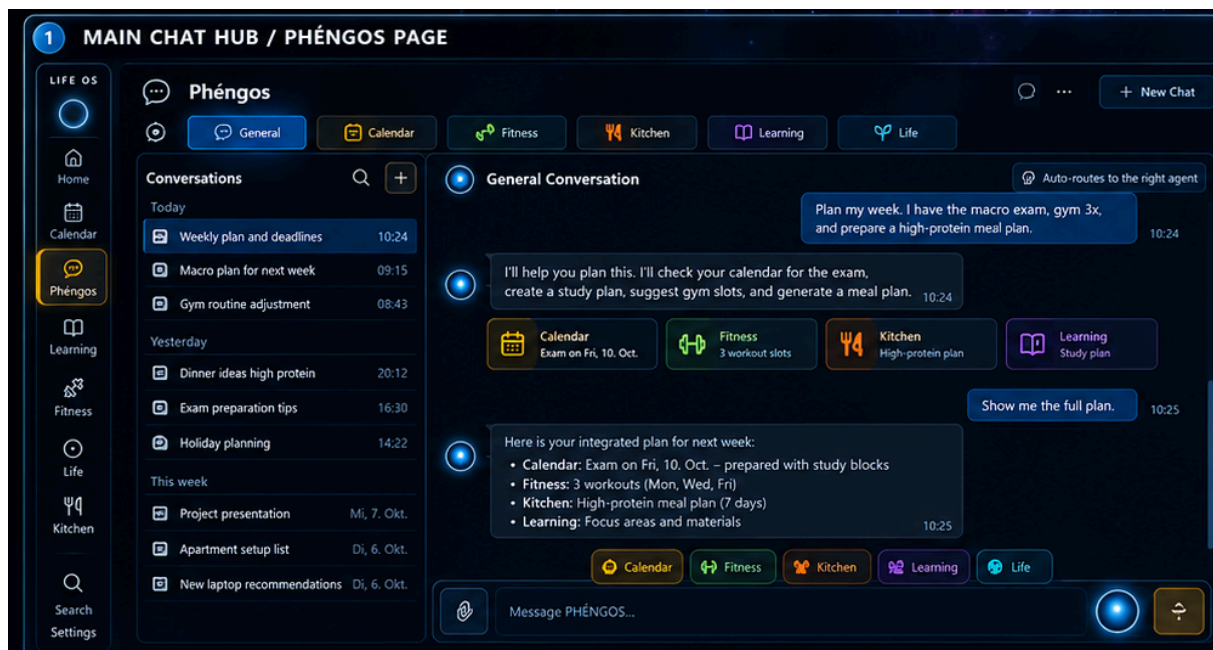


Shopping needs/priorities, short horizon and manual add within workspace.

15 Main Chat design plates — reviewed concepts, not route screenshots

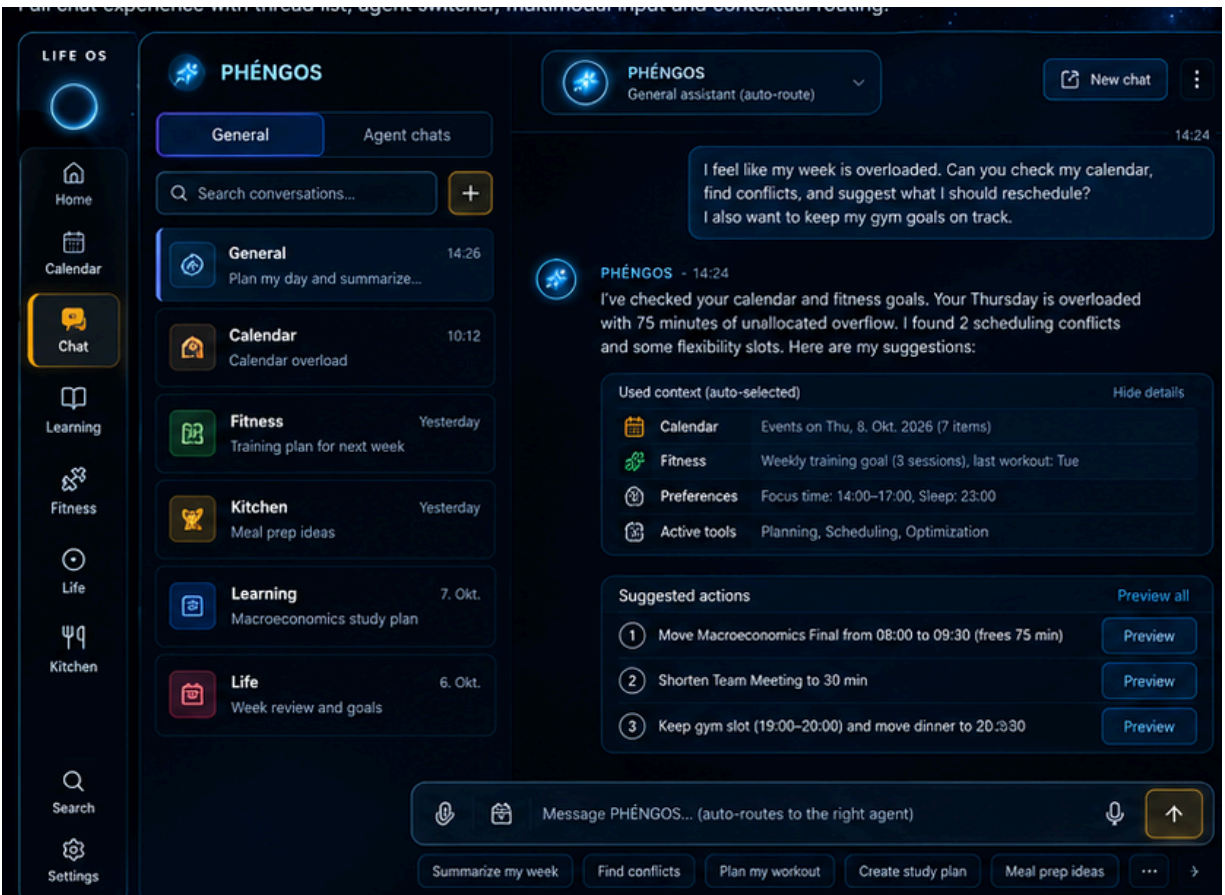
These four images are carefully cropped from earlier conversation design boards to isolate the full Chat experience and its shared context model. They are REFERENCE-ONLY. Each figure is subordinate to §04.5. Do not reinstate old bottom bars, promotional text, blue orb colors, or per-domain chatbot backends shown anywhere in earlier full design boards.

FIGURE CH01 | Photos/CH01_chat_hub_general_threads_CONCEPT.png



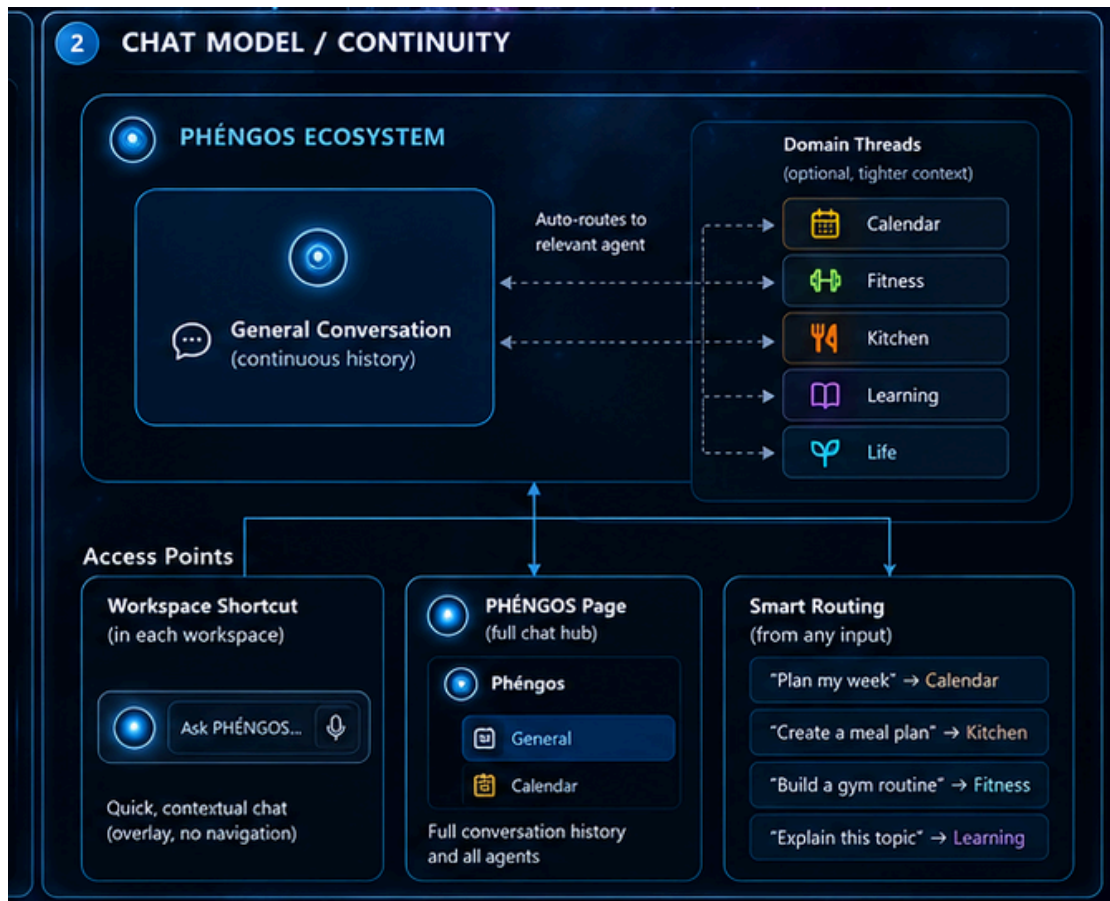
General conversation + thread history + available specialist selector. Source design used legacy icon styling; actual Chat uses WHITE sidebar-orb theme.

FIGURE CH02 | Photos/CH02_chat_hub_agent_selection_CONCEPT.png



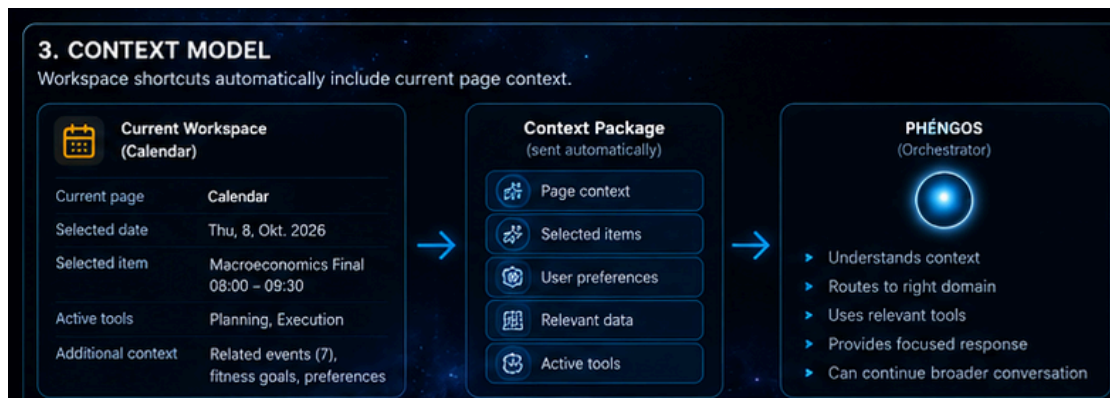
Dedicated main Chat page with agent-mode picker, canonical history, actionable tool cards and contextual results; example content not real data.

FIGURE CH03 | Photos/CH03_chat_continuity_model_CONCEPT.png



One PHÉNGOS ecosystem and multiple addressable domain-scoped threads. Diagram does not authorize distinct stores or autonomous agent systems.

FIGURE CH04 | Photos/CH04_chat_context_model_CONCEPT.png

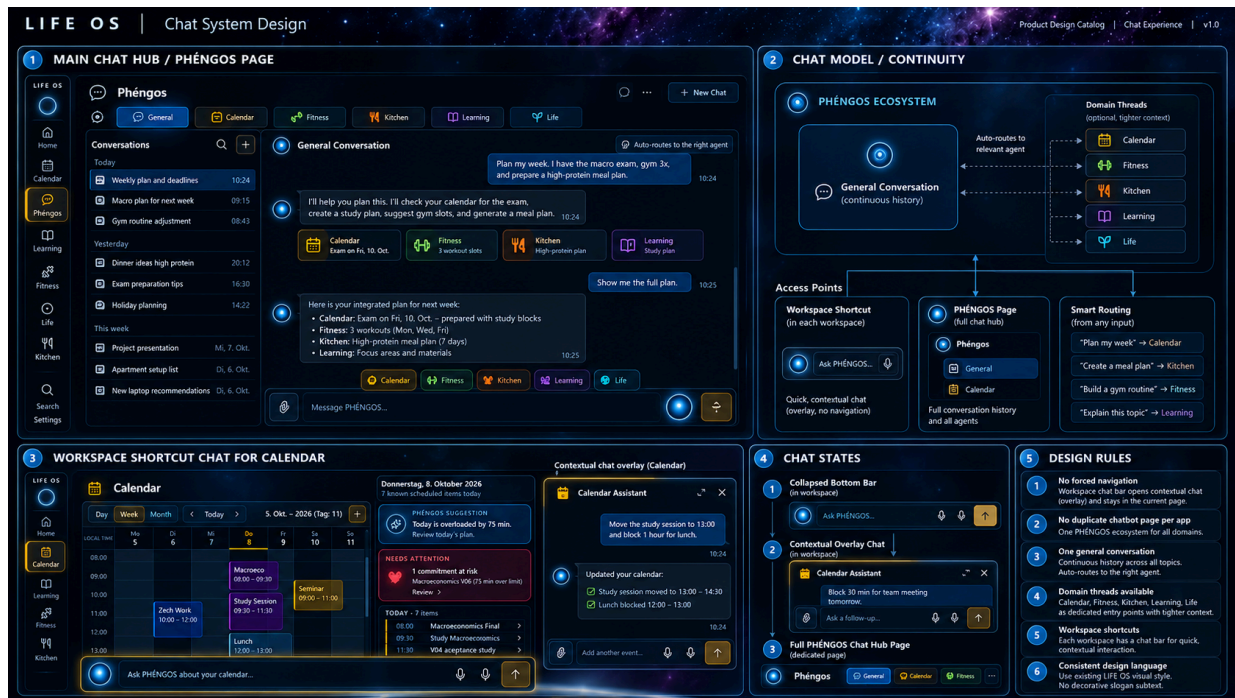


Validated bounded workspace context for prompts; do not expose unrestricted records, tokens or unvalidated client claims.

16 Original Chat exploration boards — historical only

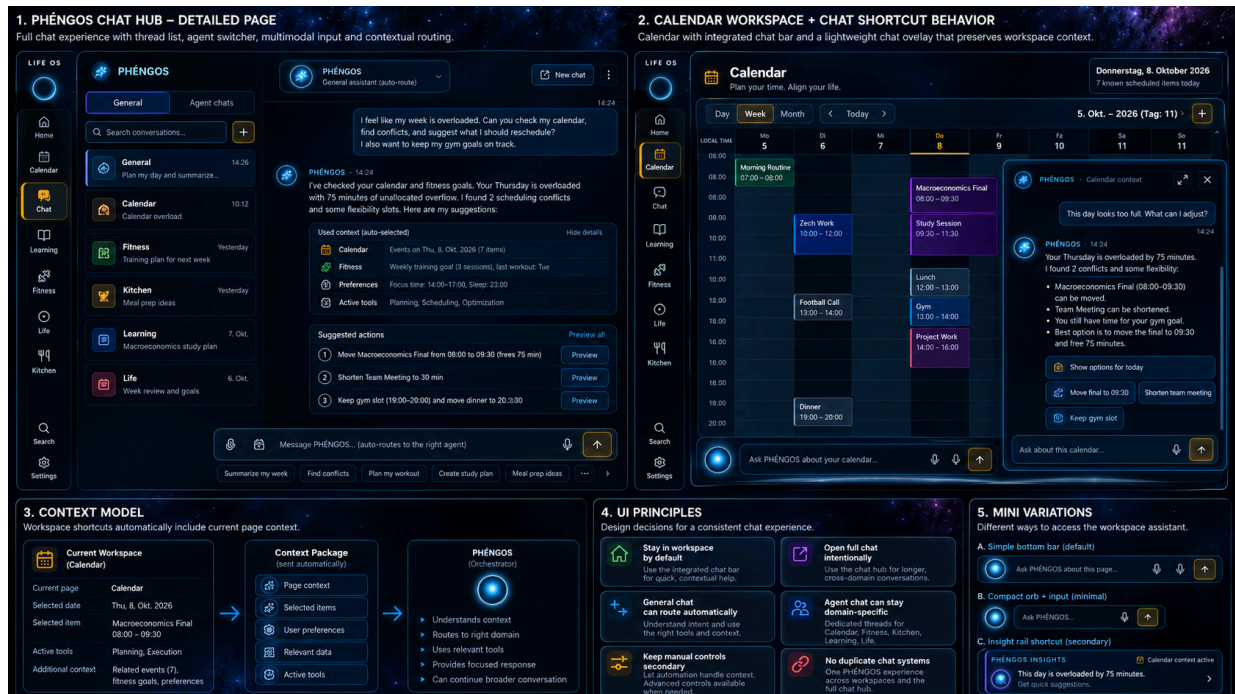
The two uncropped source boards below record the earlier discussion and remain available in Photos/. They are preserved for traceability, not for direct replication. Their superseded visual and behavioral details must NEVER override the latest frozen contract in §§02–06.

FIGURE CH05 | Photos/CH05_original_chat_system_design_board_HISTORICAL.png — HISTORICAL SOURCE, NOT IMPLEMENTATION TARGET



Prior general Chat/thread and multi-agent catalog. Keep only compatible composition patterns; remove outdated page-level bottom bars and old non-white global Chat visual accents.

FIGURE CH06 | Photos/CH06_original_chat_workspace_design_board_HISTORICAL.png — HISTORICAL SOURCE, NOT IMPLEMENTATION TARGET



Prior expanded Chat/Calendar catalog. The active freeze instead requires top-right in-page PHÉNGOS launcher, floating overlapping window, no generic date card or bottom bar.

17 Fitness UI [FIT-001] — FROZEN target update

Approval: latest user FREEZE. Preserve existing FitnessPage.tsx, workout services, sessions, progression, BodyMeasurement, FitnessGoal and recovery. Recompose them into the approved visual language, not an independent app redesign. Treat sample stats, training details, and dates in screenshots as non-canonical illustrative values.

17.1 Dashboard structure and navigation

Area	Required data and behavior	Ownership
Today's training	Active/current scheduled plan, template name, exercises, sets/reps, start/resume session; agent may prepare plan but actions use canonical session commands.	Fitness status + workout APIs; Calendar plan projection
Neon human model	Front/back body render; deterministic highlighted regions based on TODAY'S EXERCISE IDS, not on a generic Push/Pull/Legs label. Click region to reveal contributing exercises/sets.	Fitness exercise catalog; proposed typed muscle-region projection
PHÉNGOS training report	Explain what training has been prepared, why muscles are highlighted, readiness and recent training context, timestamp, provenance and approved action status.	Existing agent/runtime orchestration; typed Fitness projections
Readiness / consistency	Use real recovery and sessions; unavailable sleep/HRV must show "not connected", NOT sample values.	Fitness readiness/status
Calories and macros	Use Fitness nutrition-target contract plus canonical Kitchen consumption; distinguish target vs confirmed consumed.	Typed Fitness → Kitchen contract
Progress graph	Metric selector with observed data, today marker, dashed conditional forecast, goal band/date only if supported.	Body trends, goals and proposed projection
Training workspace	One compact subworkspace surface: Program, Session Log, Exercise Library, Measurements/Progress; detail transitions internal, NOT extra global navigation.	Existing workout templates, sessions, exercise APIs

The broad frozen Fitness destination hierarchy remains Dashboard / Training / Progress / Nutrition. Within Training, local views like Program, Logbook, Exercise Library and Notes are content modes, not another layer of global navigation. Do not display both a large top subnav and a second redundant card-navigation ribbon. Prefer an internal switcher only when a distinct real workspace exists.

17.2 Human Neon Model — deterministic system contract

8. PHÉNGOS or the approved agent/runtime selects or prepares a training plan; accepted/persisted Fitness template and selected exercises become the canonical snapshot for today.
9. Backend resolves every exercise_id against a controlled anatomical taxonomy: chest/pectorals, anterior/lateral/rear deltoids, biceps, triceps, forearms, lats, upper/mid back, core/rectus abdominis, obliques, glutes, quads, hamstrings, calves; include front/back render region keys.
10. Compute union of regions from TODAY'S exercises with primary/secondary/assisting muscle roles. An arbitrary shoulder+chest exercise combination MUST highlight only those mappings; it must not light up the entire push template automatically.
11. Map per-region weights from approved per-exercise mapping and optional known session sets/volume. Default to discrete primary / secondary / non-target states if no defensible quantitative intensity exists; do not turn subjective "activation" into biomedical measurement.
12. Send a stable typed render snapshot (training date, workout/template ID, exercise IDs, muscle role list, region keys, source/version, observation/planned status). UI renders a single canonical SVG/mesh with selective neon overlays; no per-exercise AI image generation for anatomy states.
13. If the agent changes the proposed plan, update the highlight preview and explanation. If mutation needs confirmation, separate proposal from approved training; update persisted canonical state only through the existing workflow/approval path.
14. On selecting a highlighted muscle, show the exact exercises and primary/secondary contributors. Front/back toggle must preserve highlight semantics. Empty/no plan state shows dim body and "No training selected" rather than invented glow.

Examples (illustrative only): chest+shoulders → pectorals and relevant deltoids illuminated; pull with rows and curls → back/rear deltoids/biceps; leg session → quadriceps/hamstrings/glutes/calf regions only where actual exercises map. Muscle mapping is based on exercise records rather than the session nickname. Glow color inherits Fitness green; global Chat remains white, page-level orb Fitness green.

17.3 Training workspace and manual inputs

- Program: existing WorkoutProgram, WorkoutTemplate, WorkoutTemplateExercise and exercise library. Exercise card: trusted instructional image when licensed/verified, target sets×reps/load/rest, recent achieved values, start/log/edit; generated decorative sketch is NOT technique instruction (Next Implementation File §8.3).
- Active session: existing WorkoutSession/ExerciseSet and mutation routes support set-by-set logging, rest timer, completion or abandon; never replace this with one "Mark day complete" write if sets are required.
- Session Log: dated completed workout/sessions, exercises, sets and trend; missing history pagination is only a proposed API extension after checking current endpoints.
- Measurements / Progress inputs: actual dated body weight (kg), body-fat %, lean mass (kg), muscle mass (kg), waist/other data only if schema supports it, source and optional notes. Fitness already has POST /body-measurements and list/latest/trends; do not create a duplicate measurement table.
- Goals editor: active FitnessGoal can set target weight, target body-fat, target lean mass and direction. The user must be able to choose metrics independently and create a goal without implying weight loss. Additional target dimensions or target_date require separate user/backend decision.
- Corrections: inspect whether current backend supports update/delete of erroneous entries; avoid displaying editable historical records until a verified endpoint exists. Proposed edit API requires audit/provenance and approval.
- PHÉNGOS receives approved typed metrics and source timestamps; it cannot silently rewrite measurements or substitute speculative estimates for user data.

18 Fitness progress graph — mandatory screenshot correction

MANDATORY CORRECTION OF FIGURE FIT01: The mockup has the axes reversed. Production charts put the SELECTED PROGRESS METRIC / VALUE on the VERTICAL LEFT Y axis; DATE / ELAPSED TIME on the HORIZONTAL BOTTOM X axis, increasing left to right. Progress does NOT automatically mean weight loss. This written rule overrides FIT01, FIT02 and every illustrative screenshot, including a screenshot of a Strength Index with time on the left.

Metric selector: General Goal Progress (% of selected goal attainment, only with approved normalization); Strength/Performance (meaningful chosen exercise/aggregate and units); Body Fat (%); Body Weight (kg); Lean Mass (kg), only when genuine dated observations exist. Left Y axis dynamically shows that metric with correct units and relevant scale; bottom X axis always shows actual observation dates/months and forecast dates. A lower weight or body-fat value is not necessarily better. Never flip, reverse, or re-label axes simply because a target moves downward.

Display element	Mandatory contract
Observed points	Timestamped canonical measurements, workout performance or validated normalized goal score. Actual recorded points connected by solid line; do not interpolate fake readings.
Current marker	Last reliable observation, with value and measurement timestamp; show stale-data flag where appropriate.
Goal marker / band	Explicit goal value/target range from FitnessGoal or separately user-approved composite-goal definition; target can represent gain, maintenance, strength, body fat or weight.
Forecast	Dotted line from latest actual sample. Must be labeled PROJECTED, provide model/method, assumption/sample basis, uncertainty interval or an “insufficient data” state.
Estimated goal achievement	Only show date or time range if forecast crosses goal in supported domain/valid horizon; never reuse mockup’s March 2027/date/95 index as actual.
Normalized General Goal Progress	Proposed—not automatic: decide with user whether to normalize (current-baseline)/(target-baseline), direction-aware and clamped, or build a multi-goal composite with explicit weights. Avoid dividing by zero or combining unrelated unit scales.
Metric switching	Independent Y axis and meaningful tick labels for kg, %, strength units or normalized index; X stays chronological dates. Preserve filter/time range and mark missing periods.

Illustrative possible method, NOT a frozen backend algorithm: for goal value G, baseline B and observed x(t), directional fraction $f(t)=(x(t)-B)/(G-B)$ with safe handling when B=G; progress is 100f, optionally bounded 0–100 for display. For increasing strength targets and decreasing body-fat targets the same direction-aware expression works. This computes attainment relative to an explicit baseline, not health or fitness quality. User must approve the composite/general index and forecasting definitions before backend changes.

Forecast proposal pending user approval: use dated observations for the selected metric, robust simple slope/median trend with minimum samples and a short bounded horizon; projected date only when metric and slope imply a plausible crossing. Record forecast version, sample count, baseline, assumptions and confidence. Prefer no forecast over a precise unsupported date. Future models remain hypothetical until discussed with the user.

18.1 Technical backend comparison / needed user decisions

Status	Existing evidence / planned backend	Action
EXISTING	backend/app/domains/fitness/service.py: add_body_measurement, body_trends, create_goal/active_goal, programs, templates, sessions, progression, readiness; api/routes_fitness.py exposes endpoints.	Reuse. UI refactor + graph from existing observations.
EXISTING	Exercise.primary_muscle_group and WorkoutTemplateExercise; sessions/sets present.	Basic muscle mapping can start with existing primary muscle field; cannot infer weighted secondary muscle roles safely.
PLANNED in Next Implementation File	§8 food/training media caching and wger-first exercise assets; workflow protocol example fitness.create_training_plan@v1.	Do not duplicate media provider or claim all agent protocols shipped.
DISCUSS WITH USER → BACKEND EDITOR	FIT-BE-01 canonical region taxonomy + weighted primary/secondary exercise mapping and today focus projection.	Confirm taxonomy detail, exercise data sourcing, weighting before Next Implementation File amendment.
DISCUSS WITH USER → BACKEND EDITOR	FIT-BE-02 goal trajectory estimator: selectable metric, observed/forecast separation, normalization rule, expected-goal date and uncertainty.	Confirm goal types, minimum history, confidence/forecast semantics; no fixed “strength index” invented.
DISCUSS WITH USER → BACKEND EDITOR	FIT-BE-03 measurement corrections, optional waist/other fields and extended/paginated history only if current APIs insufficient.	Confirm which metrics, edit policy, source provenance and data retention.

19 Fitness acceptance and real screenshot mapping

- Today chest+shoulders exercise selection illuminates those regions; a mixed or custom routine illuminates its own deterministic muscle union; back/front panels remain consistent.
- Changes from one plan to another update body mapping without manual hard-coded split names; agent report references the actual chosen exercises and readiness evidence.
- Metric form persists actual samples and refreshes chart; agent sees data via typed service read, not duplicated private memory.

- Progress Y-left and Time X-bottom remain correct for all selectors and at all widths. FIT01 screenshot contrary axis labels are not implemented.
- No input or derived metric is filled with screenshot numbers. Missing/unconnected sleep/HRV/scale data must be labeled unavailable.
- Fitness orb top-right, no bottom chat composer or promotional subtext, no duplicate global navigation.

20 Finance UI [FIN-001] — FROZEN target update

Finance lives under Life. Do not create a new Finance global sidebar destination. Preserve FinanceTransaction, budgets, safe-to-spend, recurring analysis, manual transactions and existing receipts/import workflow. Finance UI is account/balance FIRST, not merely a transaction administration screen. Finance design imitates the useful hierarchy of a personal finance cockpit but must retain LIFE OS styling and its own data contracts.

20.1 Finance overview layout and required widgets

Hierarchy	Source and interaction	Implementation caution
Top row: bank position	Total available/current account balances when bank source is verified, income this month, spending this month, safe-to-spend, net income-expenses. Cards drill into account/analytics/transactions.	Current finance overview is transaction-derived, not live balance. Show “Bank connection pending”, last sync, coverage and as-of when missing.
Primary 6/12-month graph	Monthly income bars, spending bars and total balance line for recent months; range 6M / 12M / all where enough data; legend with separate scales.	Monthly income/spend can derive from canonical FinanceTransaction; historical balances require BankBalanceSnapshot or verified opening-balance/backfill.
Small debt row	Compact Klarna and credit-card tiles: outstanding, limit/plan where meaningful, next due, minimum/due and payment status. Click opens Debts.	Debt cards must not dominate chart; all amounts source from verified liability records. Do not infer card balance from outgoing charges.
Upcoming bills	Timestamped recurring commitments / upcoming recognized payments with amount, expected date, confidence, edit/review.	Existing recurring detection is likely, not contractually guaranteed; avoid claiming bills as posted.
Budget/category widgets	Budget spent/remaining and spending breakdown by category; labels link to filtered Transactions.	Use category totals with canonical data and correct direction/sign.
Recent transactions	Small recent list linking to Transactions review; no giant duplicate transaction table on Overview.	One canonical editable table lives in Transactions.
Financial brief	Agent summarizes data coverage, known obligations, trend and risks; source/last	Never label synthetic or unsynced values “live”.

Hierarchy	Source and interaction	Implementation caution
	synchronization; no autonomous money movement.	

Temporal semantics: transaction “monthly income” and “monthly spending” use actual posted records for selected time window; “net this month” = income–expense excluding own-account transfers. “Total bank balance” comes from current synced balances, not the sum of transaction inflows; “safe to spend” is a separate calculation with assumptions, protected budgets and future obligations. Values shown in FIN01/FIN02 are illustrative.

20.2 Single Finance workspace hierarchy

ONE persistent context-level Finance navigation strip: Overview · Transactions · Budgets · Debts · Analytics. It follows the same shared dark-neon-fade component as Kitchen rather than building a Finance-only tab CSS system. Manual Entry is an action/drawer in Transactions (or an appropriately focused panel), NOT a sixth permanent tab or competing bottom ribbon. FIN01/FIN02 illustrate both a bottom Finance ribbon and a Manual Entry top tab; BOTH duplicate controls are non-authoritative and must not be copied. A single route can support scoped internal content without another second-tier site navigation.

- Overview = one-screen money position and temporal analysis with limited previews and deep links.
- Transactions = searchable sortable canonical transaction table, category correction, review-required queue, transaction detail; manual add opens focused form/drawer; CSV import only temporary migration fallback until verified bank integration works (Next Implementation §6.2).
- Budgets = existing category budgets, protected-budget status, adjust goals/categories, no duplicate overview forms.
- Debts = Klarna plan/installments and credit card balances with next due and payment ledger, explicit source/verification; liabilities are not safe-to-spend duplicate charges.
- Analytics = 6/12-month trends, category movement, recurring/merchant patterns, trajectory assumptions; forecast not shown as certain future balance.
- Bank integration status/action in relevant account/Settings view. Finance is reached through Life; the Life navigation selection remains highlighted.

21 Finance backend alignment + editor/user decisions

Status	Verified plan / route	Implementation/editor action
EXISTING	backend/app/domains/finance/service.py: overview, safe_to_spend, transaction listing, budgets, recurring; backend/app/api/routes_finance.py route family.	Compose dashboard from existing read models and add only bounded read projections if needed.
PLANNED already frozen	Next Implementation File §6: provider-neutral BankDataProvider, Plaid qualification gates for TARGOBANK/DE-EU, BankConnection, BankAccount, BankBalanceSnapshot, sync cursor, finance typed tools.	Integrate as it ships. If Plaid fails, editor must request orchestration approval to evaluate Tink then Yapily; no direct TARGOBANK PSD2 implementation.
DISCUSS WITH USER → BACKEND EDITOR	FIN-BE-01 liabilities: Klarna installments and credit card	Discuss provider/manual data entry and debt ownership before

Status	Verified plan / route	Implementation/editor action
	statements, outstanding vs paid, due dates, amount, limits and source; detect double-counted repayment transactions.	modifying Next Implementation File.
DISCUSS WITH USER → BACKEND EDITOR	FIN-BE-02 historical monthly time-series + cashflow/forecast projection where current overview is insufficient; bank balance snapshot retention/backfill.	Discuss date horizon, assumptions, what constitutes actual balance, whether user approves projection at all.
READ MODEL ONLY if sufficient	Source-aware upcoming payments and categorized review queue can be composed from recurring, budgets and existing transactions.	Start with current APIs; do not create separate contract services if existing projections are enough.

- No invented account figures, monthly balances, bank sync or debt balance in UI. Connected bank state includes as-of and partial coverage.
- Charts must not combine two units on one axis without axis labels. Historical balance lines require verified snapshots; do not reconstruct cash on hand from an arbitrary partial CSV.
- Debt payment reconciliation must distinguish statement balance, liability balance, outgoing transaction and income/spend; no counting debt repayment as new consumption twice.
- PHÉNGOS Finance orb in page header uses chosen subdomain accent (the gold/amber concept); global Chat accent remains white. No sticky date chip; meaningful analysis period selector inside workspace is allowed.
- No financial transfers, bank revocation or debt settlement from UI/agent without existing explicit write/approval workflow.

22 Movies UI [MOV-001] — FROZEN target update

Movies lives under Life. The frozen concept MOV01 is the preferred overview. REMOVE any account-style name, avatar/profile banner, biography or unnecessary Profile tab: this is a personal local LIFE OS library, not a social movie account. Preserve a poster-first layout, favorites, summary star rating, watched/diary/watchlist counts and activity chart.

22.1 Exactly one Movies top navigation strip

FROZEN Movies top navigation: exactly Overview · Watchlist · Logs, implemented with the same shared dark-neon-fade strip as Kitchen/Finance/Fitness. No Profile tab and no extra Library/Lists as fourth global subtab. MOV02 concept renders Library Catalog / Watchlist / Diary Log / Lists; these four labels are NOT authoritative. Full searchable catalog and saved lists can be reached from relevant Overview/Watchlist work areas without duplicate global or local navigation.

View	Contents	Interaction
Overview	Summary: average personal star rating, watched count, diary entries, watchlist count, activity/time histogram; pinned favorite poster row, recent activity, watchlist preview, compact lists preview; film details and search.	Click card opens movie detail drawer or routed detail; recent/diary/watchlist snippets deep-link to relevant tab. Library poster grid can expand inline/secondary view within Overview, not new main sidebar.

View	Contents	Interaction
Watchlist	Full responsive 2:3 poster grid with search/filter/sort, priority/status, remove/add; ratings from user only if watched, distinguish provider rating if present.	Keyboard-accessible poster actions and detail drawer; no nested duplicate import page.
Logs	Diary entries with dates, star rating, notes, rewatches, watched history filters, add/log viewing action.	Search within logs, rating/review editing only if backend supports confirmed mutation; otherwise show read+new log flows.
Contextual extras	Saved/curated movie lists and full library catalog are valid secondary panes from Overview; Settings owns full import workflow.	Do NOT add a fourth main tab merely because MOV02 concept rendered it.

22.2 Star rating and summary widgets

- Average personal rating uses rated MovieViewing records only, converting stored rating/rating_scale to five-star display safely; user's own 0–5 with half stars is distinct from provider popularity scores.
- Activity-over-time counts confirmed watched/viewing records grouped by local month/year; zero data shows neutral empty chart, not fabricated month bars.
- Optional compact distribution histogram counts each star rating bucket. Decision needed: average per diary viewing versus latest rating per unique film; until decided, label calculation and avoid cross-view contradictions.
- Favorites row is user-selectable/pinned in desired order; do not infer “favorites” from merely 5-star ratings if explicit favorites haven't been stored.
- Recent activity is ordered by actual watch history/created time with real notes and dates. Duplicated same film through rewatches remains distinct viewing records.
- Lists preview shows named collections only when backend exists; do not make fake hard-coded lists look saved.
- Film posters use canonical TMDb metadata/provenance and 2:3 aspect with lazy loading and safe placeholders; no AI-generated fake movie posters (Next Implementation §10.7).
- “PHÉNGOS suggestion” is an agent-produced, provenance-backed optional summary/action that uses existing recommendation service, not an invented automatically generated recommendation from a static frontend widget.

23 Movies backend alignment + open backend queue

Status	Available / planned	Agent implementation obligation
EXISTING	Movie/MovieExternalId/MovieWatchlistItem/MovieViewing/ImportBatch in models; services for watchlist, viewings/ratings/notes, recommendations, Letterboxd preview/confirm; apps/web/src/features/movies/MoviesPage.tsx.	Preserve and recompose into real Overview/Watchlist/Logs. MovieViewing.notes already exists—do not request new notes model.

Status	Available / planned	Agent implementation obligation
PLANNED in Next Implementation File	§10.1–10.10: one-time Letterboxd export baseline, TMDb catalog/posters, 2:3 poster grid, detail drawer, deterministic agent tools, link to Settings import.	Do not claim Letterboxd live sync. Full import belongs in /settings/integrations/letterboxd; Overview link may open that Settings page.
DISCUSS WITH USER → BACKEND EDITOR	MOV-BE-01 custom named lists with ordered membership and explicit pinned favorites; choose limits/private-only/order/history requirements.	Get user agreement before adding tables/routes and amending Next Implementation File.
DISCUSS WITH USER → BACKEND EDITOR	MOV-BE-02 bounded monthly watch-history stats + rating summary/distribution projection; first determine existing endpoints/data adequacy.	Agree whether repeated viewings count and which rating defines average; add read projections only when needed.

- MOV01 preferred approved dashboard; MOV02 preferred catalog/look and workspaces; do not literally implement its fourth top navigation tab or its in-page import wizard.
- Explicit page top-right PHÉNGOS orb with Movies purple accent; no bottom chat dock; Chat sidebar stays white. Life remains global active main domain; Movies has NO new main sidebar icon.
- No decorative subtitle or invented user profile. Overview is a LIFE OS tool, not social account.
- TMDb provider tokens and Letterboxd ZIP stay server-side. Ambiguous import/catalog matches require explicit resolution; no automatic duplicate film insertion.

24 Backend-editor handoff: proposed additions require USER discussion

EDITOR STOP RULE: New backend items below are **PROPOSED**, not approved by the UI FREEZE. Before editing Next Implementation File (1).docx or implementing a new service/schema, the editor **MUST** discuss each new feature and its contract with the user. Do not silently merge backend changes into a UI task.

Ticket	Owner / proposed new read or mutation contract	Discussion required before implementation
FIT-BE-01	Fitness: taxonomy + primary/secondary muscle role mapping; get_todays_muscle_focus(plan/session_id)	Map anatomy granularity; intensity; admin editing/source
FIT-BE-02	Fitness: goal trajectory metric/time series and optional estimated crossing	Agree metrics, goal meanings, normalization, projection uncertainty, minimum data
FIT-BE-03	Fitness: optional additional measurements & correction/provenance methods	Fields and ownership, history edit policy, scale integration

Ticket	Owner / proposed new read or mutation contract	Discussion required before implementation
FIN-BE-01	Finance: typed liabilities, Klarna installments and card statement/payment schedule	Data entry vs provider origin; payoff accounting/double-counting
FIN-BE-02	Finance: bounded historical income/spend/balance/forecast read projection	Balance snapshots, time bucket, bank provider readiness/assumptions
MOV-BE-01	Movies: ordered lists + user-pinned favorites	Count/order, explicit membership and UI CRUD behavior
MOV-BE-02	Movies: rating distribution/time-series stats where needed	Rerates vs rewatches, same film across logs, sample rules

25 Implementation guidance and regression acceptance

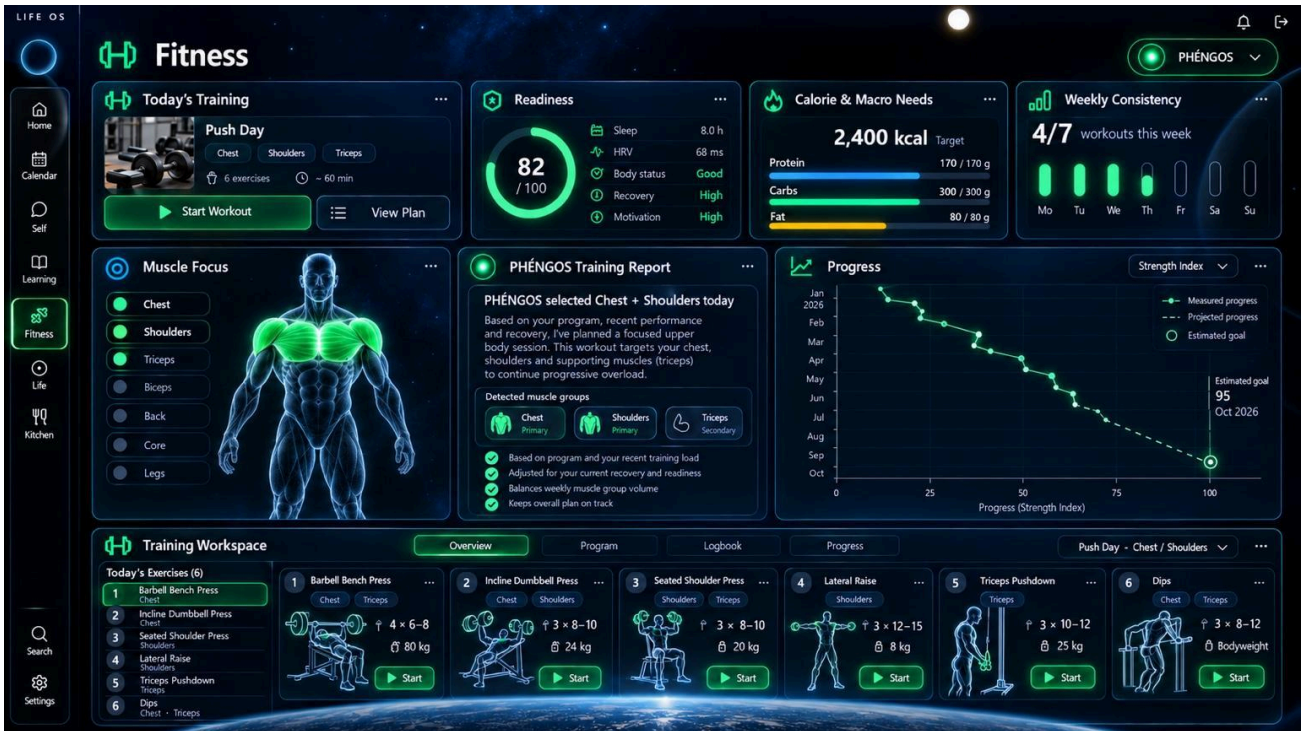
- Implementation source precedence: latest explicit user FREEZE > this UI implementation document (target UX) > Next Implementation File (1).docx (frozen backend/orchestration direction) > current GitHub master (what has actually shipped). No contradiction may silently overwrite another source. An editor must ASK THE USER before amending the Next Implementation File for any proposed model, endpoint, projection formula, liabilities, favorites or anatomy taxonomy. Existing backend endpoints must be reused wherever sufficient.
- Implement shared presentational shell, route and PHÉNGOS using frozen NAV-001/CHAT-001/PHENGOS-002/COPY-001/MOTION-001; these three domains inherit without creating additional permanent sidebars.
- Fitness verifies each custom exercise set maps to muscle regions and report; history/backdated measurements update correctly; all metric selectors use progress LEFT Y and time BOTTOM X. No invented goal/fitness judgments.
- Finance verifies empty/unlinked bank vs synced bank, correct monthly net and category totals, forecast caveats, small debts, single canonical Transactions editable list and protected sensitive banking fields.
- Movies verifies star conversion and stats, 2:3 poster grid, watchlist add/remove, diary log, rewatches, import link to Settings only; no new Profile route or fourth movie navigation destination.
- All domains: no screenshot data as default production values, no permanent raw tool stack under primary dashboard, keyboard/tab/focus/accessibility, reduced motion, state preservation on chat open and page navigation.
- Run backend tests, API contract tests, route-refresh tests and screenshots against real sample fixture data in a segregated test environment. Codex “completed” statements without runtime evidence are insufficient.

26 Frozen visual reference plates — Fitness, Finance, Movies

The following exact approved user-supplied images are bundled at full resolution inside Photos/. Figure captions below identify what is binding, and what the generated image got wrong. The written contract overrides image mistakes. All poster titles, exercise numbers, dates, cash amounts, body values and PHÉNGOS reports are illustrative.

FIGURE FIT01 — Fitness dashboard (approved)

PHOTO: Photos/FIT01_fitness_overview_FROZEN.jpeg

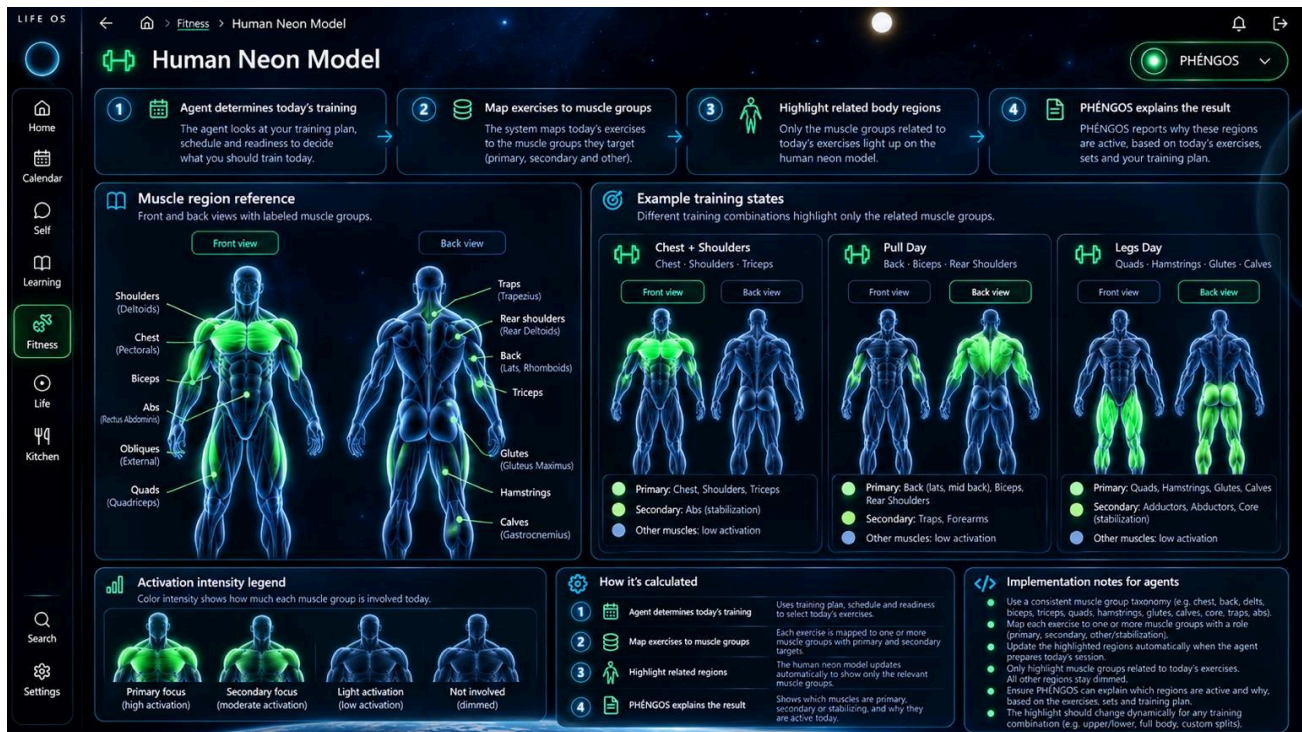


MUST MATCH — Preserve dark cosmic canvas, Fitness green, body focus, prepared workout, readiness, macro summary, training explanation, and compact + workout workplace.

IMAGE ERRORS / NON-AUTHORITATIVE — GRAPH AXES MUST BE FIXED: photo puts Time vertically and Progress horizontally; actual implementation puts Progress on LEFT Y axis and Time on BOTTOM X axis. No automatic weight-loss goal. Generic “Push Day” muscle selection may not override exercise IDs. Screenshot fake readings not canonical.

FIGURE FIT02 — Human Neon Model and interaction catalog (approved)

PHOTO: Photos/FIT02_human_neon_behavior_FROZEN.jpeg

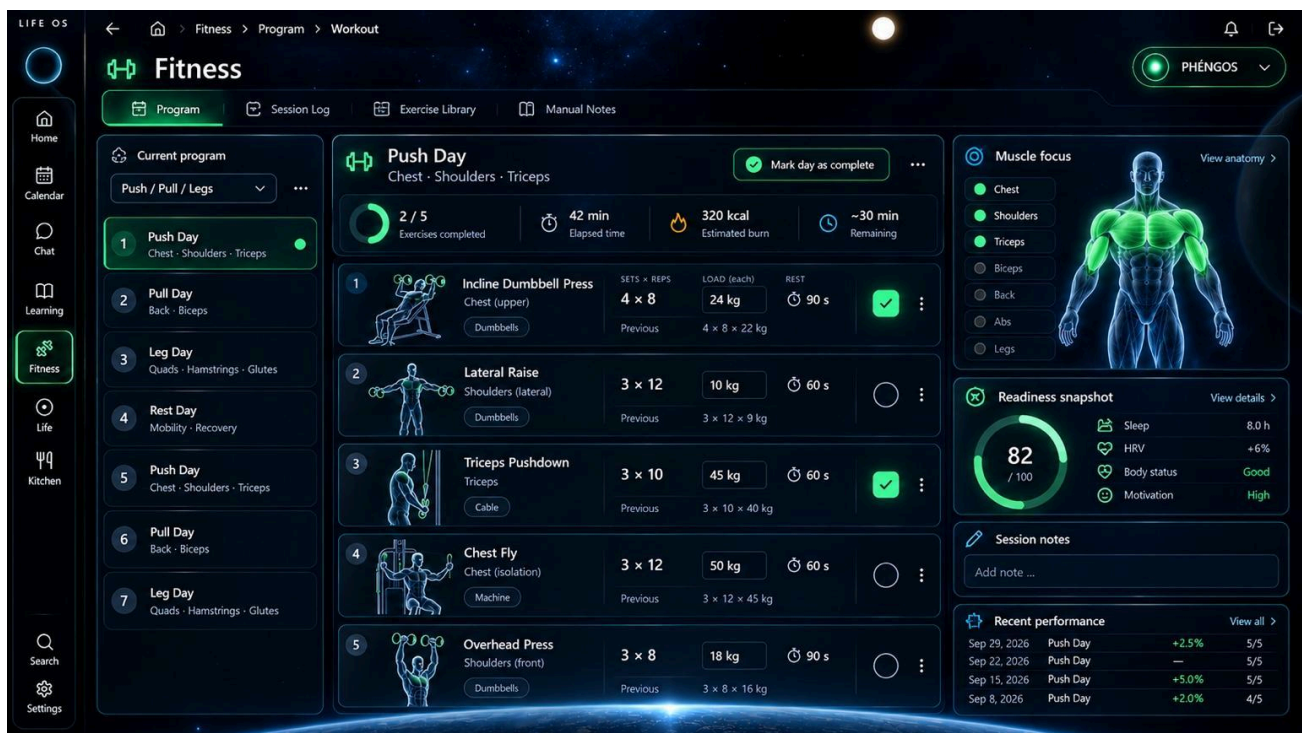


MUST MATCH — Agent prepares selected exercises; deterministic primary/secondary mapping drives specific luminous muscles; user can inspect front/back and contributing exercise list.

IMAGE ERRORS / NON-AUTHORITATIVE — Images are anatomical illustration, not ground-truth intensity or exercise technique. Highlighting is per actual exercise, not predetermined Push/Pull/Legs scene. Any incorrect anatomical colors/labeling must be corrected with mapped regions.

FIGURE FIT03 — Fitness Program / training workspace (approved)

PHOTO: Photos/FIT03_fitness_training_workspace_FROZEN.jpeg



MUST MATCH — Preserve exercise cards, sets/ reps/ loads, status, prior performance, active workout, program selection, context side panels; keep focused editor.

IMAGE ERRORS / NON-AUTHORITATIVE — Do not create a second global navigation or use sample calorie burn as measured fact. Any training image is decorative unless validated as exercise instruction. Show only real session records.

FIGURE FIN01 — Finance Overview (approved)

PHOTO: Photos/FIN01_finance_overview_FROZEN.jpeg



MUST MATCH — Account balance-first header, income/spend/net/safe-to-spend, historical combined chart, compact debt tiles, budgets/category and recent transaction previews.

IMAGE ERRORS / NON-AUTHORITATIVE — Date range selector inside Finance is meaningful and may stay; generic header date chip is removed. Balance line unavailable until trusted balance records exist. Do not show screenshot euros without source. Lower quicklinks must not duplicate subnav.

FIGURE FIN02 — Finance Transactions / manual workspace (approved)

PHOTO: Photos/FIN02_finance_manual_workspace_FROZEN.jpeg

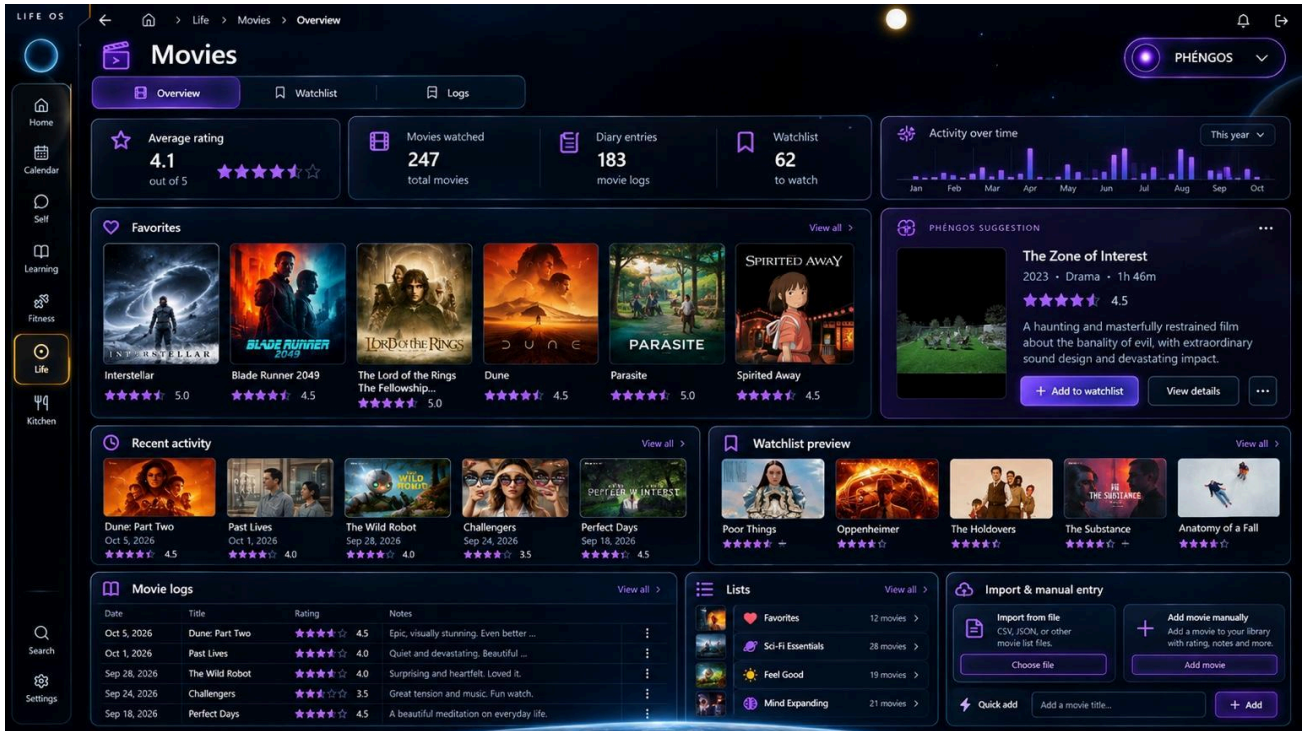


MUST MATCH — Dense filtered transaction grid, manual add, category review, recurring bills, budget and debt contextual panels, maintaining cosmic blue glass and amber accent.

IMAGE ERRORS / NON-AUTHORITATIVE — Do not preserve “Manual Workspace” as distinct duplicate route. One workspace tab layer; forms in Transaction/Detail not below Overview. Financial examples are fabricated demo values.

FIGURE MOV01 — Movies Overview (approved)

PHOTO: Photos/MOV01_movies_overview_FROZEN.jpeg

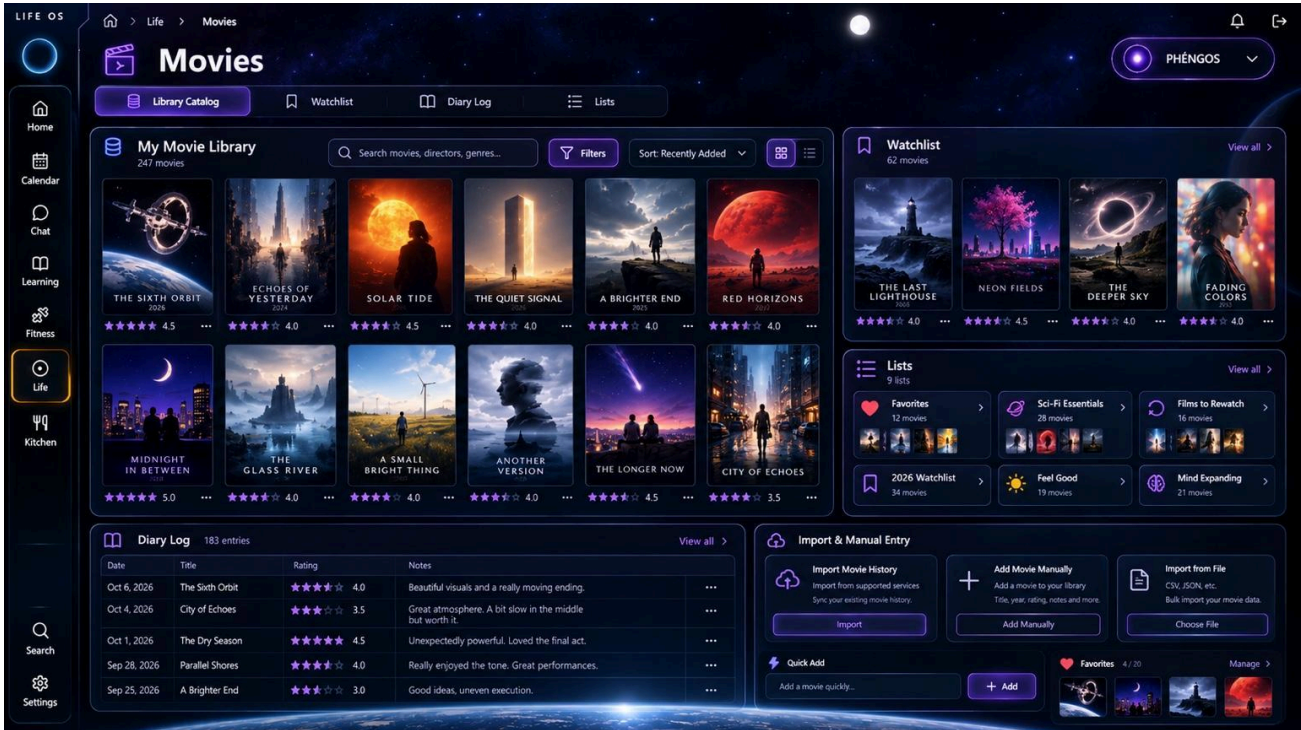


MUST MATCH — User-free profile: star average and ratings, watch stats, time histogram, favorites, optional PHÉNGOS pick, recent activity, watchlist preview, log preview.

IMAGE ERRORS / NON-AUTHORITATIVE — Use only Overview / Watchlist / Logs top tabs. MOV01 is an approved UI image but its generated import wizard must become a link to Settings Letterboxd import; poster content is illustrative.

FIGURE MOV02 — Movies library catalog / workspaces (approved)

PHOTO: Photos/MOV02_movies_catalog_workspace_FROZEN.jpeg



MUST MATCH — 2:3 poster grid, search/filter, watchlist preview, saved lists and diary log, movie detail actions; keep Life highlighted globally.

IMAGE ERRORS / NON-AUTHORITATIVE — The four concept tabs Library Catalog/Watchlist/Diary Log/Lists conflict with later user freeze. Final top navigation is Overview/Watchlist/Logs only. Custom lists still need user-approved backend; imported data does not continuously sync.

27 Supplementary figure register and frozen change log

ID	Asset	Source role / mandatory difference
FIT01	Photos/FIT01_fitness_overview_FROZEN.jpeg	Approved dashboard; fix progress Y-left and time X-bottom.
FIT02	Photos/FIT02_human_neon_behavior_FROZEN.jpeg	Approved conceptual anatomy mapping; actual exercises control glow.
FIT03	Photos/FIT03_fitness_training_workspace_FROZEN.jpeg	Approved functional workout panel; reuse session APIs.

ID	Asset	Source role / mandatory difference
FIN01	Photos/FIN01_finance_overview_FROZEN.jpeg	Approved bank-first dashboard; balances only if verified.
FIN02	Photos/FIN02_finance_manual_workspace_FROZEN.jpeg	Approved transaction workspace; no duplicate navigation/form page.
MOV01	Photos/MOV01_movies_overview_FROZEN.jpeg	Approved movie overview; only 3 tabs; no permanent importer.
MOV02	Photos/MOV02_movies_catalog_workspace_FROZEN.jpeg	Approved poster catalog/workspace; four tabs shown are superseded.

2026-10-08 : Explicit user FREEZE approved target UI for Fitness, Finance, Movies. Existing Chat/Calendar/Kitchen/navigation contracts retained unchanged. Seven full-resolution user-supplied concept images added. New backend ideas are marked PROPOSED / DISCUSS WITH USER, not silently added to the frozen Next Implementation File.

28 Frozen visual precision and cross-agent consistency addendum

28.1 Reuse one LIFE OS neon navigation component

- Cross-agent design requirement: the user explicitly requested that all independently implemented domains inherit the EXISTING dark-window neon visual grammar from Calendar/Kitchen and the global sidebar. Do not interpret individual mockups as permission to invent unrelated tab bars, colors, glowing shapes, sidebars, split-pane shells, or card spacing.
- Top tab strip composition: small icon + concise title, restrained translucent navy/near-black container, thin quiet blue outline, active-domain accent at the selected tab, a narrow brighter leading highlight, and a horizontal gradient/wash that gradually fades into transparency toward the rest of the bar. An understated luminous underline/edge gives direction without a continuous animation.
- The transition is gentle and finite: approximately 220–320 ms for text/underline, 280–420 ms for glass tint, 160–260 ms for panel crossfade, shared easing cubic-bezier(0.22,1,0.36,1). No pulsation, scroll jumps, layout shift or data-value animations; support prefers-reduced-motion. These are implementation starting parameters, not new visual assets.
- Use the EXISTING shared CSS variables, glass card component, main navigation primitives, sidebar-orb styling and existing Kitchen navigation reference (Photos/K08_kitchen_subnav_detail_USER.png). If shared primitives cannot cover these views, extend them centrally rather than creating independent CSS systems in Fitness, Finance and Movies.
- A single canonical workspace navigation surface: Fitness Dashboard / Training / Progress / Nutrition (with scoped Program/Logbook/Exercises inside Training); Finance Overview / Transactions / Budgets / Debts / Analytics; Movies Overview / Watchlist / Logs. Clicking global Fitness enters Fitness; Finance/Movies are reached through Life. No second global menu, domain interstitial, duplicate bottom nav, or giant form stack.
- For all three domains: contextual PHÉNGOS at upper-right with existing sidebar orb geometry tinted by CURRENT workspace; clicking opens a compact dark-glass floating chat window over content without shrinking the dashboard. No bottom chat bar or top-right decorative date widget. For global Chat, the original orb remains WHITE-accented.

- Screenshot priority: the frozen user-uploaded images are layout/art-direction references, not source-of-truth data. Written corrections above win on graph axis, duplicate navigation, importer location, form scope, PHÉNGOS semantics and any fabricated amounts/measurements/posters.

28.2 Exact Fitness observation, goal and projection semantics

- A metric chart has three separately identifiable layers: ACTUAL observations (solid line with observed timestamped points), TODAY marker at the latest available observation, and PROJECTED continuation (dashed line beyond the final observation) with an optional estimated crossing/goal marker. Never join a model estimate into the solid historic record. Label the model/version/assumptions and an explicit evidence sufficiency state.
- For Body Weight and Body Fat: read dated observations from existing BodyMeasurement and GET /body-measurements (units kg and percent). For strength/performance: derive a declared and reproducible metric from completed ExerciseSet history and a selected exercise/consistent test. Lean Mass is available only if actual sourced measurements exist. A general “progress index” is NOT a raw backend fact; its formula and the selected objective must be confirmed before use.
- Goal editor must show the user the current target and baseline and allow entry of genuine dated measurements through existing POST /body-measurements and POST /goals; select which goal metric the trajectory follows. If user adds a correction, preserve source/time/provenance; proposed historical edit/delete endpoints are not currently verified. Do not add fake waist, heart-rate variability, sleep, calories or body measurements as defaults.
- Direction-aware progress proposal (REQUIRES USER APPROVAL for canonical backend): for baseline B, target G, observed X(t), attainment ratio $A(t) = (X(t) - B) / (G - B)$, defined only when G differs from B and units match. A(t) is goal attainment, not health quality. Show raw metric alongside percentage; if moving away from goal, clearly show regression rather than silently clipping to success. For non-linear body composition goals or multi-objective blends, use separate confirmed definitions; do not assume equal weighting.
- Forecast proposal (NOT BACKEND-FROZEN): only estimate arrival if there is adequate time span and sample count, sufficient trend stability, known target, compatible units, credible slope direction, and finite horizon. Otherwise show “Not enough data to estimate.” Forecast should carry method, sample count, last observation timestamp, projection horizon and confidence/uncertainty interval when defensible. Do not show a specific March-2027 promise because a screenshot contains it.
- MUSCLE DISPLAY INPUT: the Fitness agent produces/selects a canonical workout/template and exercise IDs. The deterministic Fitness service resolves each exercise to stable muscle groups and primary/secondary roles; the client maps only those returned region IDs onto one reusable front/back neon anatomy asset. A chest + shoulder selection must not automatically light triceps merely because a demo program is labeled “Push.” It highlights triceps only when actually mapped from selected exercises. No AI vision inference on decorative exercise images.
- HIGHLIGHT STATES: primary solid green glow, secondary softer green glow, not-targeted dim neutral navy; optionally show “unknown mapping” as unhighlighted with a review hint, not guessed anatomically. Hover/click muscles reveal contributing exercises and actual number of scheduled sets. Agent explanation cites approved plan/session and read-only readiness inputs without pretending measurable muscle growth, physical injury or tissue recovery.
- Typed snapshot proposal for editor discussion: {date, approved_workout_id, exercise_ids, regions:[{region_key, role, source_exercise_ids}], source_version, state:proposed|approved|active}. Final schema names/weighted mappings and updating policy are NOT approved backend changes yet (FIT-BE-01).

28.3 Finance money definition and source-aware dashboard gating

- Top KPI cards mean distinct things: verified total current bank balances; posted income in selected month; posted spending in selected month; safe-to-spend from existing current Finance policy; net from posted income minus spending excluding internal transfers. Do not count the same Klarna or card purchase as spending again when the repayment is posted. Missing bank sync means unknown balance, not €0.
- The primary chart owns the central area: 6/12-month actual posted income and spending bars; historical end-of-period verified bank balances as a separate line only when bank snapshots exist. Forecasts are separate dashed segments. Timestamp bucket, account coverage, sync age and balance units are shown; never show simulated bank history as actual.
- Use compact subordinate Klarna/credit-card cards with outstanding amount, as-of/source, next due and status ONLY after a verified liability register or documented manual entry exists. User discussion required for FIN-BE-01; no direct inference from merchants named Klarna or credit-card outgoing transactions.
- Finance current GitHub exposes transaction-derived overview, budgets, recurring and safe_to_spend; bank provider and BankBalanceSnapshot are PLANNED in Next Implementation File §6 and gated by provider qualification. Preserve CSV import as temporary fallback until a production bank connection truly works; never declare Plaid live. Upcoming recurring is estimated, not guaranteed to have posted.
- The Overview displays the bank-first visual hierarchy, compact budget/category summaries and recent transactions; the sole editable transactions review and one manual form belong inside Transactions. Never implement the competing bottom Finance ribbon shown in FIN01 while also implementing the top tab strip.

28.4 Movie statistics, poster source and workspace routing

- Overview shows average USER star rating (with transparent calculation policy), number watched, diary entry count, watchlist count, activity by month, favorite posters, recent watched items and watchlist preview. All values derive from canonical MovieViewing, MovieWatchlistItem and Movie entities, not Letterboxd live API or screenshot text.
- Watchlist is a dedicated poster-first searchable/filterable 2:3 grid with clear add/remove/film details and optional list context. Logs is the chronological film diary with watched_at, user rating, optional notes, rewatches and add log. Full library/pinned favorites/lists can expand from Overview without violating the three-item top navigation.
- Poster attribution and metadata follow Next Implementation File §10 TMDb provider contract. Do not synthesize copyrighted/official-looking fake movie posters, or treat design-mockup sample posters as real catalog data. Import action goes to /settings/integrations/letterboxd and is a one-time/baseline import; do not present a continuously connected Letterboxd badge.
- Explicitly ordered favorites and named ordered lists are proposals (MOV-BE-01) requiring user approval. Summary activity/read projection (MOV-BE-02) should be calculated from existing read models where performant, or proposed as one backend aggregate if a bounded result is needed. Ratings must specify per-viewing vs per-unique-film and whether rewatches count.

28.5 Agent editor discussion gates and evidence

Area / ID	What must be determined with user	What can be reused immediately
FIT-BE-01	Anatomy taxonomy, exercise->muscle mapping source, role labels, corrections, proposed/approved focus changes	WorkoutProgram, WorkoutTemplateExercise, Exercise.primary_muscle_group, today session, readiness
FIT-BE-02	Goal progress index/normalization, goal date projection metric, minimum samples, confidence and limits	FitnessGoal, BodyMeasurement, body_trends, ExerciseSet history
FIT-BE-03	Which manual metrics beyond existing schema, edit/delete policy, measurement source and audit	POST/GET body-measurements and POST/GET goals
FIN-BE-01	Klarna/card outstanding and installment source, creditor/account reconciliation, payment lifecycle	FinanceTransaction, budgets, recurring
FIN-BE-02	Historical balance series source, provider timeline, forecasts, timeframe, backfill and projected labels	Monthly transactions; planned \$6 BankBalanceSnapshot
MOV-BE-01	Ordered/pinned favorites, named lists, create/edit/remove controls and limits	Movie, viewing, watchlist, metadata
MOV-BE-02	Rating average counting rule, histogram bins, monthly activity bucket/filter semantics	MovieViewing.rating/rating_scale, notes, watched_at

STOP RULE: The UI design is frozen, but the seven backend change tickets are discussion proposals, NOT authorized additions to the Next Implementation File. The editor must raise them with the user and record approval before modifying canonical schemas/endpoints. Keep current repo checks repeatable by source file and test outcomes.

Part Three Backend and agent runtime contract

The following backend implementation file is preserved in full. Its workstreams, tests, milestones, out-of-scope rules and definition of done remain authoritative within the source precedence above.

LIFE OS — NEXT IMPLEMENTATION FILE

Status: FROZEN FOR ORCHESTRATION

Date: 2026-10-08

Audience: implementation orchestrator and execution agents

Purpose: implement the approved runtime, integration, finance, food-data, sports-card, media-cache, cost-observability, and developer-control changes without inventing architecture or duplicating existing systems.

1. AUTHORITY AND EXECUTION RULES

1.1 The live repository is implementation truth. Before changing a subsystem, inspect only the relevant current files, migrations, tests, and dependencies. Preserve newer working behavior.

1.2 This document is the product contract. Agents must not reinterpret unspecified behavior, silently expand scope, create parallel frameworks, or replace working modules merely because a different design is preferred.

1.3 If the repository conflicts with this contract, preserve the working implementation, record the exact conflict, and stop dependent work for the orchestrator to resolve. Do not guess.

1.4 Migrations must be additive and reversible. Existing tenant data, CSV finance imports, routed workflows, media assets, calendar entries, memories, skills, and agent configurations must remain usable.

1.5 Each worker must receive disjoint file ownership from the orchestrator. Workers must not revert other workers' changes. Integration, sequencing, architectural decisions, and final verification remain with the primary orchestrator.

1.6 Use targeted tests while developing. Run broader backend/frontend verification only at milestone boundaries.

1.7 No API secrets, OAuth/access tokens, bank credentials, raw financial payloads, prompts containing secrets, or full account identifiers may appear in logs, events, traces, browser storage, API responses, fixtures, or test snapshots.

2. FROZEN SYSTEM ARCHITECTURE

2.1 Domain ownership

Life OS initially continues to use its existing PostgreSQL/Supabase persistence. Do not create a physical database per agent. Domains own canonical tables and service contracts. Agents access domains through typed services/tools; agents do not query another domain's tables directly.

Canonical facts such as transactions, nutrition values, inventory, goals, fixtures, recipes, exercises, schedules, and costs are not agent memory. They remain in their owning domain. Agent memory stores durable preferences, decisions, explanations, and continuity summaries—not copies of canonical operational records.

Cross-domain changes use typed service calls and the existing event/outbox infrastructure. There must be one source of truth for each fact.

2.2 Agent runtime and triggers

JEV is not a scheduler, trigger engine, general orchestrator, or always-running agent.

The runtime must distinguish three wake sources:

TIME: deterministic schedules such as daily or weekly work.

USER: an explicit conversation or “run now” request.

EVENT: a typed domain event that requires follow-up.

Time schedules wake agents deterministically. Users may wake an agent manually. Events may enqueue bounded follow-up work. An LLM/JEV decision must never determine whether a clock fires or whether a due job exists.

Each agent has a work lane containing definitions, schedules, runs, steps, leases/claims, completion state, retries, and next due work. Required job-step kinds are:

CHECK: inspect deterministic state.

ACT: call a typed tool/service.

DECIDE_TYPED: bounded structured decision through JEV or another selected decision provider.

REASON: use a general reasoning model only when deterministic and typed decision paths are insufficient.

WAIT: pause for external/user state.

REPORT: persist a concise result and audit record.

2.3 Duplicate-work prevention

Scheduled work and manual conversations share one completion ledger. Every recurring responsibility has a stable requirement key, recurrence/window identity, and idempotency key.

Before running a scheduled checklist item, the runtime checks whether the requirement was already satisfied inside the current recurrence window by a scheduled run, manual conversation, event-triggered run, or user action. If satisfied, record SKIPPED_ALREADY_SATISFIED and move to the next due item. Do not repeat the work merely because the clock fired.

Use database uniqueness plus transactional claim/lease semantics. Do not rely on prompt instructions to prevent duplicates.

2.4 JEV scope

JEV is allowed only inside an already-triggered run for bounded, typed decisions with a validated schema, such as choosing among approved agents, ranking predefined actions, resolving an ambiguous category, or selecting an approved workflow branch.

JEV receives the minimum necessary structured context. It does not receive whole databases or unrestricted memory. Validate every response. On timeout, invalid schema, or insufficient confidence, use the declared deterministic fallback or route to review. Never silently accept malformed output.

2.5 Existing projects and external runtimes

Do not introduce OpenMausBot or another agent framework as the Life OS runtime. Patterns such as lanes, leases, heartbeats, idempotency, and resumable runs may be borrowed, but Life OS keeps its own runtime and domain contracts.

Symphony/Codex belongs in the developer control plane for repository implementation work. It is not the end-user scheduling engine and must not become a second source of truth for product jobs.

3. COST AND EXECUTION OBSERVABILITY

Add a provider-call ledger that attributes every model, decision, embedding, vision, and image call to:

- tenant/user
- agent/skill
- trigger source
- job definition and run
- step and purpose
- provider and model
- request identifier
- input, cached-input, and output usage when provided
- quantity for image/tool calls
- estimated cost and currency
- latency
- status/error category
- cache hit/miss
- created timestamp

The ledger must support per-agent, per-provider, per-purpose, and per-time-window aggregation. The Agent page must show why money was spent, not only a total.

Do not estimate missing usage as if it were exact. Store estimated values with the pricing-version identifier and an explicit estimate flag. Provider price configuration must be versioned.

4. SETTINGS AND INTEGRATION CREDENTIALS

4.1 Existing architecture to extend

The repository already has write-only provider credential routes and Supabase Vault access in:

```
backend/app/intelligence_settings/provider_credentials.py
backend/app/api/routes_intelligence_settings.py
apps/web/src/features/settings/SettingsPage.tsx
```

Extend/generalize this design. Do not create plaintext credential columns or a parallel encryption system.

4.2 UI and API

Add the user-facing route:

```
/settings/integrations
```

Add a provider-neutral backend surface:

```
GET /api/v1/settings/integrations
```

```
PUT /api/v1/settings/integrations/{provider}/credentials
```

```
DELETE /api/v1/settings/integrations/{provider}/credentials
```

```
POST /api/v1/settings/integrations/{provider}/test
```

The status response may return configured state, scope, environment, capabilities, last success, last attempt, and a sanitized error code. It must never return secret values.

Support tenant-scoped and installation-scoped credentials. API-Football and public catalog keys can normally be installation-scoped so shared data is fetched once. AI-provider credentials can remain tenant/user scoped. Bank connection access tokens are connection-scoped dynamic secrets.

TMDB uses the same provider-neutral routes with provider ID `tmdb`; do not create a second credential framework. The concrete write-only credential operations are:

```
PUT /api/v1/settings/integrations/tmdb/credentials
```

```
DELETE /api/v1/settings/integrations/tmdb/credentials
```

```
POST /api/v1/settings/integrations/tmdb/test
```

Add a TMDB provider detail view under `/settings/integrations/tmdb`. It shows configured/not-configured state, environment, language/region preferences, last test/success, sanitized errors, Test connection, Replace credential, and Remove credential. It never displays the saved read token.

Add `/settings/integrations/letterboxd` as a first-class Settings section. Letterboxd is an import workflow, not an OAuth or live-connected provider. The section accepts an official Letterboxd export ZIP or

supported individual CSV files, previews the import, resolves ambiguous matches, confirms the baseline, and shows import history. Link to this same workflow from Movies; do not implement a duplicate importer.

4.3 Secret-storage contract

Store API-Football keys, USDA keys, image-provider keys, the TMDb read token, Plaid secrets, and bank connection access tokens only in Supabase Vault through backend-only functions. Store non-secret configuration such as provider name, environment, client ID where safe, enabled state, language/region preferences, and Vault reference separately.

Credential submission is write-only. Validation errors must not echo request bodies. Replace and delete operations must be transactional. Removing a provider key disables future calls but does not delete already normalized domain data. SQLite/local environments without Vault fail closed for secret persistence.

5. BEŞİKTAŞ NEXT-MATCH CARD

5.1 Product behavior

The dashboard shows only the single next Beşiktaş match. It does not show a schedule or fixture list.

The card shows:

- Beşiktaş logo
- opponent logo
- home/away arrangement
- opponent name
- localized day and kickoff time
- competition
- “Time not confirmed” when the provider has no confirmed kickoff

Clicking opens the related Calendar entry. If no future fixture exists, hide the card rather than rendering a broken placeholder.

5.2 Existing implementation to preserve

The repository already contains API-Football fixture integration, a standing-calendar rule, 14-day sync logic, calendar bindings, worker execution, and a Phengos fixture card in:

- backend/app/standing_calendar/providers.py
- backend/app/standing_calendar/service.py
- backend/app/core/config.py
- apps/web/src/features/phengos/PhengosContextCards.tsx
- apps/web/src/features/phengos/phengosCardProjection.ts
- apps/web/src/features/calendar/StandingRulesPanel.tsx

Modify these paths. Do not create a second sports subsystem.

5.3 Required change

Use API-Football as the source. Do not scrape Google.

Fetch the provider's next fixture for Beşiktaş using team ID 549 and next=1, rather than limiting visibility to a 14-day date range. Synchronize daily, support manual refresh, and retry transient failures with bounded backoff. A provider update must replace/reschedule the cached fixture and associated Calendar commitment idempotently.

Remove the frontend's existing 48-hour visibility restriction. Always project the nearest scheduled/confirmed future fixture as one compact card; raise its priority inside 48 hours instead of hiding it before then.

No JEV or LLM call is permitted in this workflow.

6. BANK DATA AND FINANCE

6.1 Provider decision

Plaid is the approved first provider only if all qualification gates pass:

- TARGOBANK appears in Plaid Production institution search.
- The Transactions product is supported for that institution.
- The Life OS Plaid account is approved for German/EU Production access.
- Real connection, consent/renewal, balances, and transaction refresh work.
- Pricing/live-call allowance is acceptable.
- Contractual use permits the approved personal-finance/local-storage workflow.

Create a recorded qualification result before production enablement. If a gate cannot be confirmed, the agent may complete Sandbox integration but must mark Production blocked and report the exact missing gate. Do not silently switch providers.

If Plaid is not viable, the next evaluation order is Tink, then Yapily. Implementing either fallback requires an orchestrator/user decision after the Plaid qualification report.

Do not build a direct TARGOBANK PSD2 connector. Its sandbox is mocked test data and live access is intended for regulated/licensed TPP/AISP infrastructure.

6.2 Product behavior

The user connects a bank through Plaid Link and bank-controlled SCA. Life OS synchronizes accounts, balances, and transactions approximately as providers/banks make them available; 1–4 daily transaction refreshes are sufficient. Provide a manual Sync now action without promising real-time streaming.

Life OS stores normalized financial data in its own database and builds the Finance interface and agent tools over that data. Agents never call Plaid directly.

CSV import is not part of the target product once the user has an accepted Production bank connection and the initial account, balance, and transaction synchronization has completed successfully. Preserve all historical CSV-imported transactions and their provenance. During migration, keep the existing importer available only as a temporary fallback until the Production connection is verified; then remove or hide the CSV upload entry point without deleting imported history. If Plaid qualification fails and no approved fallback provider is connected, do not remove the temporary importer prematurely.

Retain manual transactions, receipts, budgets, safe-to-spend, recurring analysis, and current Finance workflows. Connected banking replaces CSV as the target ingestion mechanism; it does not replace the Finance domain or erase historical/manual data.

6.3 Backend contract

Introduce a provider-neutral BankDataProvider interface and a Plaid adapter. Required capabilities:

- create Link token
- exchange temporary/public token server-side
- list accounts and balances
- incremental transaction sync
- refresh request where supported
- revoke/disconnect
- institution/capability lookup
- webhook verification and processing

Use Plaid /transactions/sync with its cursor and added/modified/removed semantics. Restart the whole pagination window from its original cursor on provider mutation errors. Advance the stored cursor only after all pages and local writes commit successfully.

Required persistent concepts:

- BankConnection: tenant/user, provider, provider item ID, institution, environment, status, consent/expiry, last sync, cursor, error state, Vault secret reference.
- BankAccount: connection, provider account ID, display name, mask, type/subtype, currency, active state.
- BankBalanceSnapshot: account, available/current values, currency, provider timestamp, observed timestamp.
- BankSyncRun: reason, cursor range, counts, status, latency, sanitized error.
- Existing FinanceTransaction remains the canonical transaction record.

Map provider transaction IDs into existing external-id/fingerprint/idempotency logic. Correctly reconcile pending-to-posted transitions. Apply modified and removed updates. Preserve source/provenance metadata without retaining unnecessary raw sensitive payloads.

6.4 Banking API

Use provider-neutral domain routes:

POST /api/v1/banking/link-token

POST /api/v1/banking/connections/exchange

GET /api/v1/banking/connections

POST /api/v1/banking/connections/{id}/sync

POST /api/v1/banking/connections/{id}/reconnect

DELETE /api/v1/banking/connections/{id}

GET /api/v1/banking/accounts

GET /api/v1/banking/transactions

POST /api/v1/webhooks/banking/plaid

The webhook route is provider-authenticated/signed and does not use normal browser authentication. It queues idempotent synchronization; it does not perform unbounded work inline.

Disconnect revokes provider access, deletes the Vault token, and stops synchronization. The UI must explicitly offer retain imported history or delete provider-derived history. Do not delete history implicitly.

6.5 Agent access

Expose typed, permission-limited Finance tools over normalized data, including:

- finance.get_spending_summary
- finance.list_transactions
- finance.get_recurring_commitments
- finance.get_budget_state
- finance.get_cashflow_forecast

Tools must omit tokens, raw provider payloads, complete account identifiers, and unnecessary personal data. Sensitive access must be auditable.

7. FOOD AND NUTRITION DATA

7.1 Provider order

Use deterministic provider data before AI:

- existing normalized local item
- user-created/user-corrected item
- Open Food Facts for barcode/packaged products, especially European products
- USDA FoodData Central for generic foods and ingredient nutrients
- model-assisted interpretation only when natural-language input cannot be mapped confidently

Never fabricate calories or nutrients. Calorie, macro, serving, and unit calculations are deterministic.

7.2 Canonical records

Create or extend a provider-neutral food catalog owned by the Kitchen/Nutrition domain. Persist:

- canonical name and aliases
- provider and external ID
- barcode where applicable
- nutrient values with units and per-100g/per-serving basis
- serving quantity/unit
- retrieval/dataset version and last verification
- source confidence

user-correction state
licensing/provenance metadata

User corrections override provider values without deleting original provenance. Cache provider responses and use refresh/version policies rather than repeated calls on every UI search.

7.3 Food API

Add a provider-neutral surface:

GET /api/v1/food/search

GET /api/v1/food/items/{id}

GET /api/v1/food/barcodes/{barcode}

POST /api/v1/food/resolve

PATCH /api/v1/food/items/{id}

Do not put USDA or Open Food Facts response shapes directly into frontend contracts. Normalize in the backend.

8. FOOD AND TRAINING MEDIA

8.1 Existing implementation to preserve

The repository already has media upload/generation, binary SHA-256 deduplication, per-entity generation caching, local content serving, MediaAsset, and POST /api/v1/media/generate in:

```
backend/app/media/service.py
backend/app/media/schemas.py
backend/app/api/routes_media.py
backend/app/database/models.py
```

Extend this media subsystem. Do not create a separate image service or /image duplicate.

8.2 Provider and quality decision

Default new generated card art to OpenAI gpt-image-2.5-flare through the direct Images API, explicit low quality, 1024x1024, stored as compressed WebP when conversion preserves acceptable quality. Do not invoke a separate reasoning model merely to generate a card image.

Keep image generation behind an ImageGenerationProvider adapter. Record actual usage/cost. Do not hard-code the provider across domain services.

Before declaring final production quality, run the frozen benchmark set: ten representative food prompts and ten representative exercise-card prompts using the same templates and dimensions. Record first-attempt acceptance, retries, latency, and actual cost. FLUX.2 klein 9B is the named comparison only if Flare low fails the acceptance target. Do not add multiple production providers without a measured need.

Acceptance target: at least 90 percent usable first attempts for food card art, consistent visual style, no visible text artifacts, and no unsafe/misleading exercise depiction. If the target fails, report results before changing the default.

8.3 Exercise image safety

For exercises, prefer a suitable licensed catalog image from wger when available, retaining per-asset attribution/license metadata. Generated training imagery is decorative card art only. It must not be presented as authoritative exercise-form instruction. Instructional technique requires vetted media/content.

8.4 Versioned templates and semantic cache

Add version-controlled prompt templates in the repository, for example:

- backend/app/media/prompts/food-card/v1.md
- backend/app/media/prompts/exercise-card/v1.md

Each template declares kind, version, allowed variables, visual/style contract, aspect ratio, quality policy, and negative constraints. Domain/agent code supplies validated structured variables such as food_name or exercise_name; it does not compose unrestricted prompts.

Calculate a generation fingerprint from:

- template name and version
- canonical subject key
- normalized allowed variables
- aspect ratio/size
- style version
- provider/model policy where output compatibility requires it

Lookup by this fingerprint before generation. Enforce concurrency safety with a database uniqueness rule or equivalent transactional claim. Repeated Chicken Alfredo or Barbell Back Squat entities must reuse one approved canonical card asset instead of generating again.

Add a generic MediaEntityLink rather than separate image columns on every domain object. Minimum fields: tenant/user, entity type, entity ID, role (card/hero/instruction), asset ID, active state, generation fingerprint, timestamps.

Generation runs in a background job. Meal/workout selection never blocks on an image. The UI shows a designed placeholder until media.asset.ready. Cache hit performs no provider call. Regenerate only on explicit user request or intentional template/style change; do not automatically regenerate the entire library after a version bump.

Validate template variables as data: length limits, control-character removal, allowed fields, and no secret values. Persist template version, rendered-prompt hash, provider/model, source/licensing metadata, and usage-ledger link.

9. EVENTS AND CROSS-DOMAIN CONTRACTS

Use the existing outbox and typed payloads. Minimum events:

1. sports.fixture.updated
2. banking.connection.updated
3. banking.transactions.updated
4. food.catalog.item.resolved
5. media.generation.requested
6. media.asset.ready
7. agent.job.due
8. agent.job.completed
9. movie.watchlist_added
10. movie.watchlist_updated
11. movie.watchlist_removed
12. movie.watched
13. movie.import_previewed
14. movie.import_confirmed

Events identify canonical records; they do not contain provider tokens, complete bank data, large raw payloads, or copied domain state.

Fitness may publish a goal-derived daily nutrition target. Kitchen consumes the target through a typed contract, evaluates meal/inventory needs, and creates shopping needs. Finance supplies budget state through a typed read tool/service. Kitchen does not query Fitness or Finance tables directly. Agents coordinate through contracts and canonical data, not duplicated shared-memory facts.

10. MOVIES, LETTERBOXD BASELINE, AND MOVIE CATALOG

10.1 Existing implementation to preserve

The repository already contains a substantial Movies backend and frontend. Preserve and extend it rather than building a parallel movie service:

`backend/app/database/models.py` contains `Movie`, `MovieExternalId`, `MovieWatchlistItem`, `MovieViewing`, `MovieImportBatch`, and `MovieImportRow`.

`backend/app/movies/service.py` contains catalog, watchlist, history, and recommendation behavior.

`backend/app/movies/providers.py` contains the TMDb adapter/provider boundary.

`backend/app/movies/imports.py` contains Letterboxd preview/confirm handling for watchlist, diary, watched, and ratings data.

`backend/app/api/routes_movies.py` registers the current movie APIs.

`apps/web/src/features/movies/MoviesPage.tsx` contains the current Movies UI and import entry point.

All migrations are additive. Preserve current watchlist, viewing, rating, recommendation, assistant availability, and import behavior unless this contract explicitly changes its presentation or entry point.

10.2 Source of truth and product boundary

Use an official Letterboxd data export once as a migration/baseline. After a confirmed baseline import, Life OS is the canonical source for ongoing watchlist changes, viewing history, ratings, and preferences. Letterboxd and Life OS may then diverge intentionally; repeated Letterboxd synchronization is not required.

A later re-import is an explicit repair/import operation, not a background sync. It must remain idempotent, merge by stable provenance, never duplicate prior rows, and never silently delete local or manually created data. A future Letterboxd-compatible export may be added separately, but is not required here.

Agents interact only with normalized Life OS movie services and tools. They do not access TMDB, Letterboxd files, provider keys, raw provider responses, or image bytes directly.

10.3 Settings UX and routes

Implement the frozen Settings surfaces defined in Section 4:

`/settings/integrations/tmdb` for the write-only TMDB credential, connection test, language/region configuration, status, last success, and provider credits.

`/settings/integrations/letterboxd` for the one-time baseline import, preview, conflict resolution, confirmation, and import history.

A link from Movies to the Letterboxd Settings section; both entry points use the same backend workflow.

The TMDB Settings view contains an approved TMDB logo and the exact notice: “This product uses the TMDB API but is not endorsed or certified by TMDB.” No About page is required. Attribution belongs in this persistent provider/credits panel; it is not required on every poster card. Preserve any additional attribution placement required by the provider terms after legal/license review.

The Letterboxd view must say “Import from Letterboxd,” not Connected or Sync. It accepts the official export ZIP or supported individual files, detects their type, shows validation and match results before mutation, allows explicit resolution of ambiguous rows, and retains a sanitized batch history.

Preserve these existing movie routes and their service behavior:

GET `/api/v1/movies/metadata/search`

POST `/api/v1/movies/metadata/{external_id}/save`

GET `/api/v1/movies/watchlist`

POST `/api/v1/movies/watchlist`

DELETE `/api/v1/movies/watchlist/{movie_id}`

GET `/api/v1/movies/history`

POST `/api/v1/movies/history`

POST `/api/v1/movies/imports/letterboxd/preview`

POST `/api/v1/movies/imports/{batch_id}/confirm`

POST `/api/v1/movies/imports/rows/{row_id}/resolve/{movie_id}`

Extend the current surface only where missing:

POST /api/v1/movies/imports/letterboxd/upload

GET /api/v1/movies/imports/letterboxd

GET /api/v1/movies/imports/{batch_id}

POST /api/v1/movies/watchlist/from-catalog

The upload route accepts bounded multipart ZIP/CSV content and feeds the existing preview service; if the existing preview route can safely accept multipart uploads without breaking callers, extend it instead and do not keep both routes. `from-catalog` atomically resolves a provider external ID, fetches/upserts canonical metadata, and adds the unique watchlist item. Frontend and agent tools call the same application services, not separate implementations.

10.4 Letterboxd baseline import contract

Accept the official Letterboxd export ZIP and individual `watchlist.csv`, `diary.csv`, `watched.csv`, and `ratings.csv` files. Auto-detect supported files inside the archive. Enforce compressed size, uncompressed size, file-count, row-count, content-type/header, and execution-time limits; reject encrypted archives, path traversal, unsupported files, malformed headers, and archive bombs.

Preview is mandatory and read-only. It reports source-file counts and row states including READY, ALREADY_IMPORTED, ALREADY_EXISTS, AMBIGUOUS, INVALID, and RESOLVED, with reasons that do not expose provider credentials or raw unsafe content.

Resolve records in deterministic order: stable Letterboxd URI/identifier and recorded external IDs first, then a unique exact normalized title plus release year. Never silently select among multiple matches. Ambiguous rows remain reviewable until the user chooses a result or skips them.

Confirmation imports watchlist items, viewings/rewatches, tags where supported, and ratings normalized with their original five-point scale/provenance. Preserve source file, source row/reference, batch, content hash, and imported timestamp. Existing viewing behavior that marks a watchlist title watched remains authoritative.

Use an idempotency key derived from tenant, import source, file/content hash, and stable row identity. Re-uploading or reconfirming the same export must not duplicate movies, watchlist entries, viewings, or ratings. Confirmation is transactional at the documented batch/row boundary, records partial/rejected outcomes explicitly, and never overwrites higher-priority local corrections without a visible conflict decision.

After confirmation, enqueue deterministic metadata/poster enrichment for records that lack canonical provider metadata. Import completion does not wait for every poster; the Movies UI uses placeholders until enrichment succeeds.

10.5 TMDb provider and canonical catalog

Extend the existing `MovieMetadataProvider` and TMDb adapter; keep the application provider-neutral so another catalog can replace or supplement TMDb later. Required provider capabilities are search by title/year, fetch detail by provider ID, fetch image configuration, and normalize provider failures.

Automatic matching is allowed only for a unique high-confidence stable-ID match or unique exact normalized title plus release year. Every ambiguous result requires user selection. An agent may present candidates but may not invent or silently choose movie identity.

Persist canonical Movie records and MovieExternalId mappings with provider, external ID, metadata source, language, and fetched/updated timestamps. Store only the minimal raw licensing/provenance metadata needed for audit; do not make raw provider payloads a frontend or agent contract.

Preserve current `poster_url` behavior for compatibility while adding provider-neutral image metadata additively where absent: poster provider, poster path, image-language selection, fetched timestamp, and source/licensing metadata. Retrieve TMDb image base URLs and supported sizes from the provider configuration endpoint and cache that configuration; do not hard-code a CDN base URL as the catalog contract.

Cache normalized catalog results and use a bounded stale-refresh policy. Search, detail lookup, import matching, and poster resolution are deterministic provider operations and must not call a language model.

10.6 Deterministic agent tools

Register typed movie tools over the existing service layer:

```
movies.search_catalog — READ_ONLY
movies.get_watchlist — READ_ONLY
movies.get_recommendations — READ_ONLY and reuses the existing deterministic
    recommendation service
movies.add_to_watchlist — MUTATION
movies.remove_from_watchlist — MUTATION
movies.mark_watched — MUTATION
movies.rate — MUTATION
```

The conversational agent extracts the user's intent; deterministic backend services resolve and mutate records. When search is ambiguous, return typed candidates and require the user to choose. Adding from a provider result is atomic: fetch normalized detail, upsert Movie and MovieExternalId, then insert the unique watchlist item. Repeating the same command returns the existing item without duplication.

Mutations follow the existing tool permission, proposal/confirmation, audit, tenant, and idempotency policy. Preserve the existing “watch when available” behavior. Agents receive normalized result DTOs only and never receive the TMDb token, raw response, Letterboxd upload, or unrestricted catalog transport.

10.7 Poster and media storage

Store provider IDs, image paths, dimensions/language where available, and metadata in PostgreSQL; do not store poster image blobs in PostgreSQL. Construct an allowed CDN URL from cached provider configuration and the requested display size. Use a designed 2:3 placeholder when no poster is available, lazy-load non-visible images, and expose useful alt text.

Optional object-storage caching is allowed only after a provider-license review and must retain source/licensing metadata, original provider/path, fetch time, and a purge/refresh policy no longer than the

applicable provider limit (default maximum six months where TMDB policy permits). The implementation must be able to purge cached provider images without deleting canonical movie records.

Do not use the food/training image-generation pipeline for movie posters and do not generate fake or official-looking posters. The agent never receives or reasons over poster bytes.

10.8 Movies interface

Replace the primary watchlist's admin-style row/list presentation with a responsive 2:3 poster grid. Each card shows the poster and title, with year/status only when useful. Hover/focus actions must also be keyboard accessible. Search, filter, and sort operate on canonical local data.

Selecting a card opens a detail page or drawer with poster/backdrop when available, title/original title, year, runtime, genres, overview, provider/source state, watchlist state, viewing/rating history, deterministic recommendation explanation, and allowed actions. Missing images use the designed placeholder without layout shift.

History may remain a chronological/timeline view. Move the full Letterboxd import workflow to Settings and leave a clear link/status in Movies. The interface must never imply that Letterboxd is live-connected or continuously synchronized.

10.9 Events, observability, and cost

Preserve the existing movie outbox event names listed in Section 9. Movie UI projections subscribe through the existing SSE/world-revision invalidation mechanism and refetch authoritative state after watchlist, viewing, rating, import, or enrichment changes.

Record deterministic catalog/import calls with tenant, integration/provider, purpose, request count, latency, status, retry, and cache-hit metadata. These calls have zero AI/model spend; do not fabricate token cost for them. If TMDB has no metered per-call price, show operational request/cache/error metrics rather than a fictional currency amount.

10.10 Security, attribution, and failure behavior

All routes are authenticated, tenant-scoped, rate/budget limited where appropriate, and use sanitized provider errors. Provider tokens are server-only Vault references and never enter browser responses, agent context, event payloads, workflow JSON, logs, traces, or import batches.

A TMDB outage, missing credential, rate limit, or poster failure does not remove local watchlist/history data. The UI shows cached metadata or placeholders with a retryable integration state. Letterboxd import failures remain resumable/reviewable; they do not roll back unrelated confirmed batches.

11. DEVELOPER CONTROL AREA

11.1 Purpose, route, and source-of-truth rule

Provide a developer-facing control room under `/developer/agents`, with child views for agent overview, runs, workflows, costs, skills/tools, and memory. It makes the system inspectable and configurable without becoming a second runtime. The UI reads and edits canonical backend contracts; it does not execute schedules in the browser, infer status from animations, or keep an independent copy of runtime state.

The overview must show every configured agent as a stable identity card/circle with its name, stable identity color, enabled state, live operational state, current task, next due wake, queued count, latest outcome, and current-period cost. Clicking an agent opens its detail workspace.

11.2 Agent identity and visual status

Each agent receives one stable, accessible identity color stored in its existing profile/configuration. Do not use that color alone to communicate operational state. Preserve the identity hue and express state through ring color, brightness, animation, icon, and a visible text label so the interface remains readable for color-blind users and when animation is disabled.

Required product states are:

- SLEEPING: friendly UI label for backend IDLE; the agent is enabled but has no claimed work and nothing currently eligible to run.
- QUEUED: at least one eligible run is waiting for a worker claim.
- WORKING: a run owns a valid lease and has a fresh worker heartbeat.
- WAITING: the current run is durably waiting for a declared external event, provider response, future time, or user approval.
- RETRYING: a failed attempt has a bounded retry scheduled.
- BLOCKED: progress requires explicit user action, missing configuration, or resolution of a permanent dependency failure.
- FAILED: the latest terminal run failed and no retry is currently eligible.
- PAUSED: the user intentionally suspended the agent or its schedules.
- DISABLED: the agent cannot be triggered.
- STALE: a run still appears claimed but its lease/worker heartbeat has expired.

Status is derived from persisted job, run, step, lease, wait, heartbeat, and configuration records. An agent/model may not self-report that it is WORKING or completed. Precedence must be deterministic: DISABLED, PAUSED, STALE, WORKING, WAITING, RETRYING, BLOCKED, QUEUED, FAILED, then SLEEPING. Document and test any deviation required by the live implementation.

WORKING requires both an unexpired lease and a fresh worker heartbeat. The runtime must persist `heartbeat_at`, lease expiry, current step, attempt, and worker identifier. Heartbeat and stale thresholds are configuration, not prompt text. If a worker disappears, the UI changes to STALE and the normal lease-expiry/recovery path decides whether to retry; the UI never changes execution state by itself.

Suggested visual behavior, subject to the separate approved design:

1. SLEEPING: dim static identity circle.
2. QUEUED: visible queue badge and subtle waiting ring.
3. WORKING: bright identity circle with restrained pulse and rotating progress ring.
4. WAITING: static pause/clock indicator with the wait reason.
5. RETRYING: retry icon and next-attempt time.
6. BLOCKED or FAILED: error outer ring plus explicit label and action.
7. PAUSED, DISABLED, or STALE: reduced saturation with distinct pause, off, or disconnected icon.

Respect `prefers-reduced-motion`. Never make a state understandable only through motion.

11.3 Agent detail workspace

The selected-agent page must contain explicit components, not an unstructured debug dump:

Identity header: agent name, stable color, role, enabled/paused toggle, active skill/profile version, provider/model route.

Now panel: current run, trigger source, current step, start time, elapsed time, lease/heartbeat freshness, wait/block reason, and safe cancel/pause controls where supported.

Queue and schedule panel: next due jobs, recurrence/window, time zone, trigger source, manual Run now, and last satisfaction state.

Run history: terminal status, step trace, retries, sanitized errors, correlation/causation IDs, idempotency outcome, and duration.

Responsibility checklist: recurring requirement keys and whether each current window was completed, skipped as already satisfied, blocked, or still due.

Cost panel: current run, day, month, provider, model, purpose, cached usage, estimated/exact distinction, and drill-down to the provider-call ledger.

Skills and behavior panel: active behavior/profile, skill file identity/version, allowed protocols/tools, permission scopes, and version history.

Memory/Brain panel: entry into the agent-scoped graph defined below.

Workflows panel: published and draft workflows that may wake or call this agent.

Editing skills, behavior, workflow definitions, or agent configuration must be versioned and auditable. Never place credentials in skills, workflow JSON, node parameters, run logs, or memory.

11.4 Real-time delivery

Extend the existing committed event/world-revision and Server-Sent Events infrastructure with invalidations for `agent.job.*`, `agent.run.*`, `agent.step.*`, `agent.wait.*`, and `agent.workflow.*`. Do not introduce a parallel WebSocket transport solely for this page.

The client subscribes to invalidations and refetches authoritative projections. Connection heartbeats only prove that the browser stream is alive; they are not worker heartbeats and must not make an agent appear WORKING. On reconnect or revision gaps, use the existing resynchronization behavior and fetch a full current projection.

Required read surfaces:

```
GET /api/v1/developer/agents
```

```
GET /api/v1/developer/agents/{agent_id}
```

```
GET /api/v1/developer/agents/{agent_id}/runs
```

```
GET /api/v1/developer/agents/{agent_id}/responsibilities
```

```
GET /api/v1/developer/agents/{agent_id}/costs
```

Mutations such as enable, pause, resume, run-now, or cancel must call runtime services and emit audit events. They must use optimistic concurrency/version checks and return the authoritative resulting projection.

11.5 Agent Brain: governed memory graph

Provide an Obsidian-style interactive graph for each agent plus a Global Brain view. An agent brain is a filtered projection over the existing shared memory and canonical references, not a separate memory database or a copy of shared memories. The existing skill/profile memory scopes and domain mapping determine which nodes appear. A memory visible to multiple agents retains one canonical memory ID.

PostgreSQL/Supabase remains the source of truth. Do not add Neo4j or another graph database for the initial implementation. Build graph projections from existing `MemoryItem`, `MemoryEvidence`, `Episode`, `supersession`, `provenance`, `source`, and `domain` data. If general memory-to-memory relations cannot be represented safely, add one tenant-scoped `MemoryRelation` model with source memory ID, target memory ID, typed relation, confidence, provenance, creator, timestamps, version, and soft-deletion fields.

Supported relation types must be finite and validated. Initial types are `SUPPORTS`, `CONTRADICTS`, `SUPERSEDES`, `DERIVED_FROM`, `RELATED_TO`, `APPLIES_TO`, and `EVIDENCED_BY`. Canonical domain entities may appear as reference nodes but are never converted into duplicated memory facts.

Graph APIs must return bounded projections, not the tenant's entire memory collection by default:

```
GET
/api/v1/developer/memory-graph?agent_id=&domain=&status=&query=&depth=&cursor=

GET /api/v1/developer/memory-graph/nodes/{memory_id}

POST /api/v1/developer/memory-graph/nodes/{memory_id}/expand

POST /api/v1/developer/memory-relations

DELETE /api/v1/developer/memory-relations/{relation_id}
```

Reuse the existing memory create, correct, pin/unpin, confirm, and forget services for node mutations. Do not create graph-only mutation logic that bypasses memory policy.

11.6 Agent Brain interaction contract

The graph must support zoom, pan, fit-to-view, minimap or orientation aid, search, multi-filter, node selection, neighborhood expansion, and a detail inspector. Filters include agent, domain, memory type, status, confidence range, date range, source, and pinned/user-confirmed state.

Selecting a node opens a side inspector containing content, domain/type, confidence and effective confidence, importance, status, validity range, created/updated/last-used timestamps where available, provenance/evidence, connected relations, supersession history, canonical references, and which agent scopes can retrieve it.

Available user actions are pin/unpin, confirm, correct, forget, inspect evidence, expand neighbors, hide from the current visual session, and merge duplicates where an existing governed merge service supports it. Correction must preserve the existing replacement/supersession history. Forget performs the existing soft-forget/suppression behavior, removes the embedding, and removes the node from normal retrieval. It is not a hard database delete.

Deleting a relation removes only that explicit relation after confirmation; it does not delete either memory. Canonical reference edges and evidence edges derived from source records are read-only in the graph and can change only through their owning service. Every mutation is tenant-scoped, version-checked, audited, and reflected through real-time invalidation.

Graph visualization does not change model context behavior. Agents continue to receive only retrieval-selected, policy-approved memories and bounded relevant neighborhoods; never send the complete graph to an AI provider.

Use Cytoscape.js or an equivalently maintained permissive library for the memory graph after dependency/license review. Preserve current list-based Personal Model access as an accessible alternative and fallback.

11.7 Agent Workflow Studio

Add `/developer/workflows` as a constrained visual composer inspired by n8n, but do not embed n8n or adopt it as the product runtime. The backend owns scheduling, validation, execution, retries, waits, idempotency, and audit. The browser edits versioned definitions and displays execution traces.

The user flow is:

- Create workflow and choose its name, owning agent, description, and time zone.
- Add exactly one trigger.
- Connect approved deterministic, condition, protocol, wait/approval, and report nodes.
- Configure each node through typed forms generated from its registered schema.
- Validate and simulate with sanitized sample data.
- Publish an immutable version.
- Enable, pause, run now, duplicate as draft, or archive through audited commands.
- Inspect each run as a node-by-node execution trace.

Workflow lifecycle is DRAFT, PUBLISHED, PAUSED, ARCHIVED. Editing a published workflow creates a new draft version; it never mutates the version used by an in-flight or historical run. Publishing records definition hash, creator, timestamp, validation result, and protocol versions.

11.8 Workflow node catalog

Version 1 permits only registered node kinds:

1. TRIGGER_SCHEDULE: deterministic local-time/recurrence schedule with explicit time zone and daylight-saving behavior.
2. TRIGGER_MANUAL: user Run now with an optional typed input form.
3. TRIGGER_EVENT: subscribed typed domain event plus validated filters.
4. READ_PROJECTION: read an approved typed service projection; never raw tables or SQL.
5. CHECK: deterministic requirement/idempotency/precondition check.
6. CONDITION: typed field/operator/value comparison with true/false branches.
7. AGENT_PROTOCOL: invoke an approved versioned agent capability.
8. DECIDE_TYPED: optional bounded JEV/decision-provider choice over predefined alternatives with schema, confidence threshold, and deterministic fallback/review branch.
9. WAIT: durable wait for time, event, provider state, or user approval.
10. REPORT: persist a concise result, notification, or dashboard/audit projection.

11. END: declare terminal success, skipped, blocked, or reviewed outcome.

Version 1 forbids arbitrary code, shell commands, raw SQL, unrestricted HTTP requests, arbitrary prompt nodes, provider credentials in fields, dynamic tool creation, self-modifying workflows, and cycles/loops. Each graph is a directed acyclic graph with one trigger, at least one terminal node, no unreachable/orphan nodes, and bounded fan-out. Later expansion requires a separately frozen contract.

11.9 Agent Protocol Registry

Code defines safe capabilities; the visual editor composes instances of them. Each callable protocol is registered with:

- stable versioned identifier, such as `fitness.create_training_plan@v1`
- owning agent/domain
- purpose and human-readable description
- strict input and output schemas
- allowed read projections and write commands
- permission and tenant scope
- side-effect and approval classification
- idempotency-key strategy and satisfaction evidence
- timeout, retry, and permanent-error policy
- estimated cost class and expected provider/model route
- required integrations/configuration
- deprecation/replacement metadata

The editor lists only protocols permitted for the workflow owner and current tenant. Cross-agent work calls a protocol; it never directly queries another agent's tables, imports another agent's prompt, or hands one agent unrestricted access to another domain.

Example frozen composition:

`calendar.commitment.created` → filter `kind = training` and `status = confirmed` → CHECK current subject/window not already satisfied → call `fitness.create_training_plan@v1` with `calendar_commitment_id` → attach the returned plan through the Calendar/Fitness contract → REPORT and mark the requirement satisfied.

If a manual conversation or prior run already created the plan for the same commitment/window, CHECK records `SKIPPED_ALREADY_SATISFIED` and the action is not repeated.

11.10 Workflow persistence and execution

Add tenant-scoped, additive runtime concepts only where the live repository does not already provide an equivalent:

WorkflowDefinition: stable workflow identity, owner agent, lifecycle, enabled state, time zone, current published version, audit metadata.

WorkflowVersion: immutable graph JSON, schema version, definition hash, protocol versions, validation result, creator, published timestamp.

WorkflowSubscription: compiled schedule or event subscription for the currently enabled published version.

Existing job/run/step/lease/completion-ledger concepts execute workflow nodes; do not create a second execution ledger.

WorkflowRunProjection: read model linking workflow/version to the canonical job run and node-level trace.

The graph JSON is configuration, not executable code. On publish, the backend validates schemas and edges, resolves protocol versions, checks permissions/configuration, rejects cycles, compiles subscriptions and a normalized execution plan, and stores the definition hash transactionally. The scheduler/event dispatcher enqueue canonical jobs referencing the immutable workflow version. The frontend never evaluates conditions or performs agent actions.

Simulation uses read-only/sample projections and stubbed side effects. It must visibly distinguish simulated output from real execution and must not call external providers unless an explicitly labeled cost-bearing test mode is added later.

JEV remains narrow: DECIDE_TYPED may select only a predefined validated branch after the workflow is already running. JEV cannot create/edit/publish workflows, define clocks, fire triggers, add protocols, bypass conditions, or read unrestricted memory/data.

11.11 Workflow validation, safety, and observability

Publish must fail with actionable node-level errors when there is no trigger/terminal, an invalid connection, cycle, schema mismatch, orphan node, unavailable protocol, missing required integration, impossible permission, unbounded fan-out, missing deterministic fallback, or side effect without an idempotency strategy.

Run history must show trigger evidence, immutable workflow/version, node inputs and outputs with sensitive-field redaction, selected branches, retries, waits, approvals, protocol and model versions, idempotency/completion decisions, costs, sanitized failures, and correlation/causation identifiers.

Workflow definitions, versions, layouts, runs, and costs are tenant-scoped. Credentials remain Vault references owned by integration services and never appear in workflow JSON. Archiving a definition stops new triggers but preserves immutable historical versions and runs. Deleting visual nodes from a draft is allowed; deleting published history is not.

Use React Flow ([@xyflow/react](https://github.com/xyflow/react)) or an equivalently maintained permissive library for the visual editor after dependency/license review. n8n may be used only as interaction inspiration; do not add its runtime, database, scheduler, credential store, or licensing dependency. Do not add Temporal in this milestone. Re-evaluate a durable-execution platform only after measured evidence shows the database-backed runtime cannot meet recovery, wait, or scale requirements.

12. WORKSTREAMS AND DEPENDENCIES

The orchestrator may distribute these workstreams only with disjoint file ownership.

A. Integration credential foundation

Own generalized Vault functions/store, integration schemas/routes, /settings/integrations UI, provider status/test contracts, redaction tests. Include the TMDB provider card/detail view and its write-only credential/test/remove behavior. This blocks production keys for B, C, E, and F.

B. Sports next-match projection

Own standing_calendar provider/service changes, worker cadence, fixture idempotency, compact Phengos projection, Calendar link, sports tests. Depends on A for user-entered API-Football credentials but can use injected test credentials.

C. Plaid banking connector

First produce the Plaid/TARGOBANK qualification record. Then own provider adapter, models/migrations, sync worker/webhook, normalization into existing FinanceTransaction, banking routes, connection UI, Finance integration, and banking tests. Depends on A. Do not modify unrelated Finance calculations. Preserve historical CSV-derived records. Keep CSV upload only as a migration fallback until the first accepted Production connection and initial sync; then remove/hide that entry point as the final target behavior.

D. Food catalog

Own provider adapters, catalog models/migrations, normalization/provenance, cache, food routes, and provider-contract tests. Coordinate with Kitchen without rewriting meal planning.

E. Semantic media pipeline

Own image-provider adapter, prompt templates, fingerprint/cache claim, MediaEntityLink, background generation, food/exercise attachment, placeholders, benchmark harness, and media tests. Depends on A for provider key management and coordinates with D for canonical food IDs.

F. Movies, Letterboxd baseline, and catalog

Own additive extension of the existing movie models/services/provider/routes; TMDB configuration and normalization; secure official Letterboxd ZIP/CSV preview, review, confirm, provenance, and idempotency; deterministic movie tools; poster metadata/cache policy; Settings entries; responsive poster-grid/detail UI; events/observability; and movie tests. Coordinate with A for TMDB credentials and K for router/migration/event integration. Do not rebuild the existing Movies backend, create a second importer, scrape Letterboxd, or give agents direct provider access.

G. Agent work runtime and cost ledger

Own job definitions/runs/steps/claims, worker heartbeats, waits, trigger sources, recurrence satisfaction logic, deterministic status projection, JEV bounded decision integration, provider-call attribution, aggregation APIs, real-time invalidation events, and runtime tests. Preserve existing agent and AI usage behavior while migrating callers deliberately.

H. Agent control room UI

Own agent overview/status circles, agent detail components, queues/schedules/responsibilities, run traces, costs, tools/skills/behavior inspection, accessibility, and APIs/projections not owned by G. Depends on G contracts. Reuse existing SSE invalidation transport. Do not implement a second scheduler.

I. Agent Brain graph

Own bounded memory-graph projection APIs, any required additive MemoryRelation model, agent/global graph UI, inspectors, filters, neighborhood expansion, list fallback, and graph tests. All memory mutations must reuse existing memory policy/services. Coordinate with H on navigation and G on invalidations. Do not introduce a graph database or per-agent memory copies.

J. Agent Workflow Studio

Own versioned workflow definitions/versions/subscriptions, protocol registry metadata, validation/compiler, simulation, editor, publish/pause/archive/run-now commands, canonical runtime integration, run trace projection, and workflow tests. Coordinate with G rather than creating parallel runs/leases/schedules. Use only registered node kinds and protocols; do not embed n8n or add arbitrary-code nodes.

K. Integration and acceptance

Primary orchestrator owns migrations ordering, router registration, event contracts, cross-workstream conflicts, end-to-end tests, security review, documentation, and final milestone verification.

13. REQUIRED TESTS

Credential tests:

- values are write-only and absent from every response/log/error
- unsupported providers fail safely
- Vault-unavailable environments fail closed
- replace/delete/test operations have sanitized outcomes
- tenant/installation/connection scopes cannot cross-read
- the TMDB read token is write-only, Vault-backed, absent from browser/agent/log/event payloads, and testable through the generic integration route
- the Letterboxd Settings card is an import workflow and never reports a false live-connected state

Sports tests:

1. only one next future fixture is projected
2. a match beyond 14 days still appears
3. past/cancelled/suppressed fixtures do not appear
4. reschedule updates the existing binding idempotently
5. card always appears before 48 hours and gains priority inside 48 hours
6. no model gateway is called

Banking tests:

- Plaid Link exchange is server-side
- access token is referenced from Vault and never serialized
- cursor pagination commits atomically
- added/modified/removed updates are idempotent
- pending-to-posted reconciliation does not duplicate spending
- repeated webhooks do not duplicate sync runs or transactions
- disconnect revokes/deletes secret and honors explicit history-retention choice

tenant boundaries and webhook verification are enforced
historical CSV-imported transactions remain readable with provenance
CSV upload remains available until an accepted Production connection and initial sync exist
after that gate, CSV upload is absent/disabled while historical and manual transactions remain intact
failure or rollback before the gate does not strand the user without an ingestion path

Food tests:

15. provider precedence is deterministic
16. barcode routes to packaged-food provider
17. generic ingredients route to USDA
18. units/per-100g/per-serving normalize correctly
19. user corrections win while provenance remains
20. no model supplies numeric nutrition values without a sourced record

Media tests:

identical semantic fingerprint returns one asset across multiple entities
concurrent misses generate once
cache hits make no provider call and are recorded as hits
template variables cannot alter the fixed template contract
template version changes create a new optional fingerprint without bulk regeneration
entity selection is not blocked by generation
instructional role cannot use unvetted generated exercise art

Movies tests:

existing movie models, service behavior, routes, and history remain operational through additive migrations
TMDB search/detail/configuration normalization is deterministic, cached, provider-neutral, and makes no model call
automatic catalog matching occurs only for stable IDs or a unique exact title/year; ambiguity requires user resolution
from-catalog atomically upserts Movie/MovieExternalId and one unique watchlist item; retry is idempotent
typed agent tools call the same application services and cannot read provider keys, raw provider responses, imports, or image bytes
official Letterboxd ZIP and individual supported CSV files are auto-detected, bounded, and rejected for traversal, encryption, archive bombs, malformed headers, or unsupported content
preview is read-only; confirm preserves provenance, ratings/viewings/watchlist semantics, and explicit ambiguous/invalid outcomes
repeated upload/confirm of identical or overlapping exports does not duplicate movies, viewings, ratings, or watchlist entries and never deletes local data
metadata enrichment is asynchronous; an unavailable poster/provider leaves canonical local data intact and shows a placeholder
PostgreSQL stores provider paths/metadata rather than poster blobs; provider configuration builds allowed image URLs without a hard-coded catalog base
any object-storage poster cache retains source/licensing metadata and supports expiry/purge without deleting the Movie record
the responsive poster grid, keyboard-accessible cards, detail view, fallback art, and Settings import route meet accessibility/product semantics

the TMDB logo and exact attribution notice are visible in `/settings/integrations/tmdb` without requiring an About page
catalog, poster, and import operations record request/cache/error metrics and report zero AI spend rather than invented token cost
movie mutations emit the preserved outbox events and refresh through existing SSE/world-revision invalidation

Runtime/cost tests:

- TIME, USER, and EVENT triggers create attributable runs
- a completed recurrence requirement is skipped across manual/scheduled paths
- lease expiry and retry do not duplicate committed actions
- invalid JEV output uses declared fallback/review
- every external AI/image call has tenant, agent, job/run, purpose, model, status, and cost metadata
- cached operations do not report provider spend

Agent control room tests:

each derived status matches the persisted configuration/job/run/lease/wait/heartbeat facts and precedence
a model message or browser animation cannot make an agent appear WORKING
expired heartbeat/lease produces STALE and normal recovery without duplicate effects
SLEEPING means enabled with no claimed or eligible work; PAUSED and DISABLED remain distinct
every agent retains its identity color while state also has text/icon/non-color communication
reduced-motion mode removes nonessential animation
SSE invalidations refetch authoritative projections and revision-gap recovery resynchronizes
agent detail exposes current work, schedule, responsibility satisfaction, run trace, errors, skills/tools, and attributable cost
all control mutations are version-checked, tenant-scoped, and audited

Agent Brain tests:

an agent view is a scope-filtered projection and shared memories keep one canonical ID
bounded depth/cursor/filter limits prevent an unbounded full-graph response
provenance, evidence, supersession, confidence, and canonical references render correctly
correction creates a replacement/supersession history rather than silent overwrite
forget uses existing suppression/soft-delete semantics and removes normal retrieval/embedding
deleting an explicit relation does not delete its nodes
derived evidence/canonical edges cannot be mutated through graph-only endpoints
tenant boundaries, expected-version conflicts, audit events, and SSE invalidations are enforced
the accessible list-based Personal Model view remains usable

Workflow Studio tests:

drafts may be edited, while published versions and historical runs are immutable
exactly one supported trigger and at least one terminal are required
cycles, orphan nodes, invalid edges, schema mismatches, unavailable protocols, and unbounded fan-out are rejected
arbitrary code, SQL, HTTP, prompt, credential, and unknown node kinds are rejected
protocol input/output, permission, side-effect, retry, approval, and idempotency contracts are enforced
schedule time zone/daylight-saving and event filters compile deterministically

simulation performs no real side effect or external provider call
publish compiles subscriptions and a normalized execution plan transactionally
manual, schedule, and event triggers execute through the canonical run/step/lease/completion ledger
prior manual or user completion causes SKIPPED_ALREADY_SATISFIED in the same requirement window
DECIDE_TYPED accepts only declared choices and follows fallback/review on invalid or low-confidence output
pause/archive stop new triggers without destroying definitions, versions, or run history
workflow JSON, traces, and errors never expose credentials or unredacted sensitive fields

14. MILESTONE ACCEPTANCE ORDER

Milestone 1: generalized integrations/Vault foundation and /settings/integrations are complete and security tests pass.

Milestone 2: Beşiktaş next-match-only card works with saved API-Football credentials, daily/manual sync, Calendar navigation, and no AI calls.

Milestone 3: Plaid qualification is recorded. Sandbox banking is complete; Production is enabled only if every gate passes. Historical CSV data remains operational. CSV upload remains only until an accepted Production connection and initial sync, then connected banking becomes the sole target ingestion path.

Milestone 4: deterministic food catalog resolves local/user/Open Food Facts/USDA sources with provenance and no fabricated nutrients.

Milestone 5: semantic media cache, templates, background generation, wger-first exercise media, and benchmark report are complete.

Milestone 6: existing Movies backend is extended with write-only/tested TMDb Settings, one-time Letterboxd baseline import and provenance, deterministic catalog/tools, poster metadata/cache policy, responsive poster-grid/detail UI, attribution, events, and security/idempotency tests; no model is used for metadata, matching, import, or posters.

Milestone 7: deterministic agent job runtime, completion ledger, bounded JEV steps, and provider-cost attribution are complete.

Milestone 8: canonical runtime status projection, worker heartbeat/stale recovery, SSE invalidations, and the agent control room expose identity/status, schedules, responsibilities, runs, costs, tools, and skills without duplicating runtime state.

Milestone 9: Agent Brain exposes bounded agent/global memory graphs, provenance, governed edit/forget/relation behavior, real-time refresh, and accessible list fallback without a new graph database or memory copies.

Milestone 10: Workflow Studio composes, validates, simulates, publishes, executes, and traces constrained versioned workflows through the canonical runtime and protocol registry.

Milestone 11: orchestrator runs integrated security, migration, backend, frontend, and end-to-end acceptance; unresolved provider or repository conflicts are reported explicitly.

15. EXPLICITLY OUT OF SCOPE

Google fixture scraping
a visible Beşiktaş schedule or multiple-match dashboard card
second-by-second bank transaction streaming
payment initiation, transfers, or moving money
direct TARGOBANK PSD2 production integration
plaintext secret storage
agents receiving provider access tokens or unrestricted bank payloads
CSV upload as a permanent parallel ingestion path after an accepted connected-bank migration
one physical database per agent
a separate copied memory store per agent
a graph database for the initial Agent Brain implementation
hard deletion of governed memory through the graph UI
JEV controlling schedules, clocks, or trigger existence
JEV creating or publishing workflows, protocols, nodes, or tools
an always-running 24/7 chat agent
OpenMausBot/Symphony as the product runtime
n8n as an embedded/runtime dependency or second scheduler
Temporal or another durable-execution platform in this milestone
arbitrary workflow code, shell, SQL, HTTP, prompt, credential, loop, or self-modifying nodes
duplicating canonical domain data into shared memory
unrestricted agent-authored image prompts
automatic bulk image regeneration
generated exercise art presented as authoritative form instruction
Letterboxd scraping, browser automation, OAuth/live sync, or background polling
treating Letterboxd as canonical after the baseline migration
direct agent access to TMDb, Letterboxd files, provider credentials, raw catalog responses, or poster bytes
poster image blobs in PostgreSQL
generated fake or official-looking movie posters
a duplicate movie backend, duplicate importer, or parallel watchlist source of truth
creating duplicate /image, /sports, /finance, or scheduler frameworks when existing modules can be extended

16. DEFINITION OF DONE

The work is done only when all applicable milestone acceptance criteria and targeted tests pass; existing relevant behavior is preserved; migrations are reviewed; secrets and tenant boundaries are verified; provider limitations are visible in the UI; costs are attributable; and the orchestrator can demonstrate the complete flows:

Save API-Football key in Settings → test → synchronize → see exactly one next Beşiktaş match.
Connect Plaid Sandbox or qualified Production bank → synchronize → see normalized accounts/balances/transactions → Finance agent reads only typed local views.
Resolve a food → source nutrients deterministically → create/select recipe → reuse or background-generate one cached card image.
Select an exercise → reuse licensed catalog media when available → otherwise attach vetted decorative generated art without presenting it as instruction.

Trigger an agent manually and by schedule in the same recurrence window → completed work is not duplicated → usage and reason appear under that agent.

Open the agent control room → distinguish SLEEPING, QUEUED, WORKING, WAITING, RETRYING, BLOCKED, FAILED, PAUSED, DISABLED, and STALE from canonical runtime facts → inspect the current task, next wake, responsibility state, run trace, errors, skills/tools, and costs.

Open an agent Brain → search/filter/zoom/expand a bounded graph → inspect provenance and connections → correct, pin, confirm, or forget through existing governed memory services → see the same canonical memory reflected in every applicable agent scope.

Create a workflow → choose owner and one trigger → connect typed condition/protocol/report nodes → validate and simulate without side effects → publish an immutable version → trigger it manually and through its declared source → inspect the node trace, idempotency decision, and attributable cost.

Connect a qualified Production bank and accept the initial sync → preserve historical CSV-derived data → remove/hide CSV upload → continue automated and manual synchronization through the connected-bank provider.

Save and test the TMDb credential in Settings → import an official Letterboxd export as a previewed/confirmed baseline → resolve ambiguity explicitly → see a responsive poster grid backed by canonical local records → ask an agent to add, watch, or rate a title through typed deterministic tools → observe one idempotent update and zero AI spend for catalog/import/poster work.

END OF FROZEN IMPLEMENTATION CONTRACT

Part Four UI roadmap and open decision register

The following roadmap is preserved in full so unfinished design areas, status vocabulary, change protocol and backend decision gates remain visible to the orchestrator.

LIFE OS — UI ROADMAP

Decisions · remaining areas · existing problems · freeze/update operating protocol

Living master document • Oct 8, 2026 • No per-freeze filenames

STATUS: design specification. No application code has been modified or verified by this package.

01 What this package is

This package is one permanent working handoff. It contains exactly two DOCX masters and a Photos directory. The Roadmap tracks scope, problems, decisions, open design questions and sequencing. The UI Implementation document holds the current cumulative, agent-executable specification and detailed figure mappings. Images live in Photos and are referenced by stable figure IDs.

NOT A NEW VERSION Subsequent design freezes update these same master files and add images and/or pages in place. Internal change entries provide traceability, not new deliverable names.

Master	Single responsibility	Expected change on Update
UI_ROADMAP.docx	Portfolio of design work, freezes, priorities, open questions and cadence	Update area status; retain previous approved decisions and carry next focus.
UI_IMPLEMENTATION.docx	Cumulative agent-level frozen UI contract	Modify/extend affected section, linked figure notes, migration plan and tests. Keep unaffected approved sections.
Photos/	Full-resolution user screenshots and approved concept images	Append referenced files, replace an image only by explicit approval; keep stable IDs.

02 Authority and source precedence

- Latest explicit user FREEZE for UI decisions outranks earlier UI proposals; do not silently replace a frozen choice.
- LIFE_OS_UI_IMPLEMENTATION.docx contains current approved UI behavior. LIFE_OS_UI_ROADMAP.docx tracks decisions and unresolved work.
- The live Fivina/life-os repository is implementation truth: files, contracts and tests determine what has actually shipped.
- Next Implementation File (1).docx from original user ZIP is the next backend/orchestration implementation baseline; it is not superseded by visual mockups.
- Older LIFE_OS_2X implementation handbooks are historical only. Generated images are visual references, never proof of live data or backend features.
- Conflict rule: latest explicit user freeze > written active UI contract > approved image (visual-only) > archived ideas. Code is truth for actual implementation, not necessarily the target design.

03 Working protocol — the word FREEZE

Stage	What we do	Output
01 / Choose	Proactively pick the next domain or utility; inspect existing UI, backend, known bugs, current screenshots and authoritative plan.	Short scope and audit; state what is implemented vs planned.
02 / Discuss	Evaluate keep, improve, merge/remove, build/extend. Offer strong alternatives without restarting proven design.	One coherent design direction; unresolved points called out.
03 / Visualize	Create baseline comparison plus overview, component detail, action, hover/open, responsive/cooking states as needed.	Images with IDs and explicit authority/mismatch notes.
04 / FREEZE	Only the user's explicit FREEZE finalizes design. Do not implement automatically.	Mark decisions FROZEN; exact affected paragraphs approved.
05 / Update	Edit the SAME two master DOCX files; register and embed images; produce ONE updated ZIP package.	Same filenames and same internal folder structure.
06 / Next	Advance to next area; future integration with backend plan once designed.	New OPEN discussion, no accidental cross-domain freeze.

04 Status vocabulary

State	Meaning
OPEN	Unresolved or ready for discussion, not an implementation instruction.
PROPOSED	A considered design or idea, not user-approved.
FROZEN	The user approved a design contract; code may still be unchanged.
IMPLEMENTED	Code has been changed, but it may not have passed acceptance.
VERIFIED	Screenshots, flows, regression checks and tests confirm the frozen requirements.
SUPERSEDED	A later explicit freeze replaced the prior instruction; no parallel conflicting instructions remain.

05 Current frozen decision register

ID	Frozen principle	Where detailed
NAV-001	One persistent sidebar; direct entry to actual domain; no sidebar flyout + second domain launcher; one contextual subcomponent level; preserve deep links.	Implementation §§02–03
DEV-001	Developer independent control-plane module, not Settings or Life; detailed design still open.	Implementation §03
CHAT-001	Self sidebar item is REPLACED by Chat (same existing PHÉNGOS orb; luminous WHITE active/hover glow). Canonical full Chat workspace: threaded	Implementation §04 + Photos CH01–CH04

ID	Frozen principle	Where detailed
	history, General/agent-specific conversations, explicit selector and bounded auto-routing; one shared runtime, no focus mode.	
PHENGOS-002	Contextual top-right PHÉNGOS orb launcher in Calendar and Kitchen, using each page's accent; opens overlapping floating in-page chat; no bottom dock.	Implementation §04
CAL-001	Existing Calendar preserved; Overview, Daily List, Planning; real interactions, no long legacy tool stack.	Implementation §05
KIT-001	Existing Kitchen preserved; Overview, Fridge, Meals, Recipes, Shopping; daily-first, history, manual workspaces, approved visuals.	Implementation §§06–08
COPY-001	No commercial/promotional subtitles beneath titles; keep functional guidance and status copy.	Implementation §02
MOTION-001	Kitchen subnavigation uses restrained, elegant, slowly fading active indication; reduced-motion support.	Implementation §08
FIT-001	Fitness UI update: neon model from deterministic exercise→muscle mapping; training report, metrics, chart with progress VERTICAL and time HORIZONTAL, training/manual workspaces.	Implementation §§17–19; FIT01–FIT03
FIN-001	Finance UI update: bank assets + 6/12-month income/spending/balance chart; compact debt cards; real Finance tools in focused workspaces.	Implementation §§20–21; FIN01–FIN02
MOV-001	Movies UI update: no profile/user header; Overview, Watchlist, Logs; favorites, star summary/distribution, library, diary, lists and poster cards.	Implementation §§22–23; MOV01–MOV02

06 Previously observed problems — preserve findings, fix the presentation

Problem from audits/screenshots	Disposition / relevant freeze
Main-domain sidebar flyout duplicates app navigation	Remove main-domain flyout; one click goes directly to workspace (NAV-001).
Home/PHÉNGOS domain-selection interstitial forces second page before actual workspace	Bypass/remove launcher hop while preserving Home cinematic intro (NAV-001).
Self route overlaps Chat/global PHÉNGOS identity	Expose Chat as main destination; preserve legacy URLs/threads (CHAT-001).

Problem from audits/screenshots	Disposition / relevant freeze
Calendar: repeated Today, Needs Attention, Week Map and raw Calendar Projection under grid	Move detailed work to Daily List/Planning, keep summary in Overview (CAL-001).
Calendar: permanent seeded Add Commitment/Intention forms; Save edit not editing	Move to contextual editors; test actual mutation and confirmation (CAL-001).
Kitchen: long stacked forms after strong dashboard; demo defaults/technical text	Relocate under Fridge/Meals/Recipes/Shopping; no fake data (KIT-001).
Kitchen: seven-day empty meal strip mismatches real daily use	Daily-first food log, agent-prepared meal, optional 2–3-day view (KIT-001).
Kitchen: large blank under Today's meal in approved concept image	Repack aligned dashboard grid; visual figure is directional, whitespace is NOT approved (KIT-001).
Chat dock/route changes when input submitted	Remove bottom dock on Calendar and Kitchen; top-right floating contextual chat (PHENGOS-002).
Unnecessary generic subheadings in workspaces	Remove promotional slogans; retain functional copy (COPY-001).

07 Frozen information architecture

Destination	Contextual workspaces	Policy
Home	Dynamic contextual overview	No Home → menu → Overview loop; no extra launcher page.
Calendar	Overview · Daily List · Planning	Overview day/week/month = view selector, not another global menu.
Chat	Threads · general and specialist agents (implementation detail within Chat)	Independent main destination; sidebar orb white accent.
Learning	Dashboard · Study · Exams · Study Log · Candidates	Hierarchy frozen, visuals OPEN.
Fitness	Dashboard · Training · Progress · Nutrition; Fitness page may expose compact internal Program / Logbook / Metrics views.	Design FROZEN: neon muscle focus, agent report, program and workout, multi-metric trajectory, editable inputs. See FIT01–FIT03. Extra backend ideas require discussion.
Life	Dashboard · Goals · Finance · Social · Movies (Life is parent, not extra sidebar icons)	Finance and Movies visual/workspace designs FROZEN (FIN01–FIN02, MOV01–MOV02); Home/Life parent remains to be designed. Movies internal tabs Overview / Watchlist / Logs.
Kitchen	Overview · Fridge · Meals · Recipes · Shopping	Detailed workspace visual structure FROZEN.
Search / Settings	Settings: Preferences · Integrations · Agent Settings	Utilities; not an excuse for duplicate app navigation.
Developer	Independent control plane	Standalone module; inner UI and access visibility OPEN.
Notebook	Existing route retained	Final placement OPEN; no arbitrary relocation.

08 Completed design areas

Chat — frozen primary workspace and legacy Self replacement

The old Self sidebar destination becomes Chat (label + icon/theme + route), using the same existing PHÉNGOS orb in white. Main Chat supports persisted thread list, new/resumed conversations, explicit General or available specialist agent mode, and automatic service/agent routing for General requests. Calendar/Kitchen use the already-frozen small floating contextual window. Do not design a new focus mode, separate per-domain chatbot backends, or a second Chat navigation hierarchy. Photos CH01–CH04 supply earlier Chat concept references with known superseded artifacts explicitly excluded in the implementation contract.

Calendar — frozen update, not rebuild

Preserve the real calendar grid, insight rail, backend-driven projection and planner service. The header gets the contextual PHÉNGOS trigger; bottom composer disappears. Details and manual forms move into Overview/Daily List/Planning with appropriate interaction state. Date navigation inside the actual calendar remains necessary; the generic always-on page header date tile does not. Figures C01/C02.

Kitchen — frozen update, not rebuild

The approved Kitchen design retains the cosmic glass visuals, narrow sidebar, stable top subnav and content density. Today’s agent-prepared meal, nutrition, food journal, fridge, expiry, recipes and shopping are visible only as useful state. Manual operations remain in focused workspaces. Cooking and pantry logic reuse existing backend. Figures K01–K08.

09 Next areas to explore (not frozen)

Sequence	Area	Audit / candidate work
Deferred by user	Home dashboard / PHÉNGOS overview	Home design intentionally skipped for now; do not mark designed/frozen. Resume only on request.
Then	Global Chat implementation QA (cross-cutting)	NOT A NEW DESIGN CYCLE: test Self → Chat migration, agent picker, thread continuity, full Chat page, contextual handoff and voice-roadmap compatibility after implementation.
FROZEN	Fitness	Approved FIT01–FIT03; translate existing workout/measurement services to visual views; scoped backend gaps require editor/user discussion.
OPEN / possible next	Learning	Learning not frozen; study/exams/logs/candidates require later design review.
Partly FROZEN	Life and Finance	Finance FIN01–FIN02 and Movies MOV01–MOV02 approved; Life parent/Goals/Social are OPEN.

Sequence	Area	Audit / candidate work
		Do not add Finance/Movies main sidebar icons.
Then	Developer	Separate technical control-plane UI; governed runs, workflows, observability; align next implementation backend plan.
Across all	Cross-domain QA / mobile / accessibility	Responsive navigation, keyboard, reduced motion, proper states, screenshot parity, no phantom data.

10 Open technical/functional dependencies

- Kitchen meal history: create/complete paths write MealHistory, but the inspected Kitchen routes lack a dedicated browsable meal-history listing endpoint; history UI needs an auditable read contract before it can be called complete.
- Food catalog: provider-neutral food catalog, Open Food Facts/USDA resolution, normalized nutrients and media caching are already in Next Implementation File (1).docx as planned backend work. UI agents must integrate those plans, not invent a second service.
- Kitchen prepared meal: use agent/runtime outputs or persisted meal plans as evidence; do not present fabricated recommendations as if the app spontaneously authored them.
- Cooking session: persistent backend ActiveWorkspace exists; verify restore/resume path on refresh before promising seamless resumption.
- Ingredient identity, units and substitutions: verify existing normalization and ambiguity boundaries; do not use fuzzy front-end matching for canonical inventory writes.
- App-specific responsive states, exact Developer interface, Notebook placement, advanced voice and advanced agent-switching edge cases remain implementation verification, not a new foundational Chat design cycle.

11 Change-management contract for future UPDATES

- Never generate a new numbered roadmap or implementation filename. Replace the master files inside LIFE_OS_UI_IMPLEMENTATION_PACKAGE.zip.
- Retain all existing frozen decisions unless user explicitly changes one; for changed items, revise the active paragraph and add one concise change-log row (do not append contradictory old instructions).
- Add new screens as new sections/pages within the same UI Implementation DOCX, and update its contents and image register.
- For each photo: put full-resolution asset in Photos/, stable ID and filename in the UI document, embedded readable preview, must-match and illustrative elements, affected code, interactions and QA.
- Update Roadmap status on same pass; do not freeze a new area without the user's word FREEZE.
- Return the one ZIP after each package update; no stand-alone secondary implementation spec.

12 Package change log

Date	Change retained in same master
2026-10-08	Consolidated prior roadmap, navigation, Calendar, Chat and Kitchen freezes into the single canonical two-DOCX-plus-Photos package; removed redundant versioned deliverables.
Future	Add a dated row only when an approved design is incorporated; keep stable filenames.

Date	Change retained in same master
2026-10-08 · Chat reconciliation	Confirmed old Self → Chat full sidebar replacement, reinserted main Chat/agent/thread design preferences and CH01–CH04 visual references; roadmap NEXT corrected to Home. FROZEN contract unchanged elsewhere.
2026-10-08 · Fitness/Finance/Movies freeze	Merged FIT-001 FIN-001 MOV-001 in the SAME master files; seven approved image references. Added backend-editor/user decision queue, not silently approved backend implementation.
2026-10-08 · Photo/package synchronization and precision audit	Rechecked Next Implementation File and live repo; seven uploaded frozen images are byte-identical to Photos FIT/FIN/MOV figures. Added precise graph axes, deterministic anatomy focus, bank-first accounting, three-tab Movies, shared fading neon navigation, user-discussion backend gates. Rebuilt master ZIP from latest DOCXs + all Photos.

13 Package reconciliation and freeze completeness audit

2026-10-08 verification: the downloadable master ZIP was previously stale relative to the latest local documents and omitted four Chat photo assets. This package refresh includes the current two master DOCX files, all cited active images, and two historical full Chat boards. No new design area is opened or silently frozen by this correction.

Decision area	Verified requirement	Status / dependency
NAV-001 / DEV-001	One direct global sidebar; remove sidebar feature flyout and domain-launcher interstitial; Developer independent	Implemented in master contract; migration not yet in app
CHAT-001	Existing “Self” entry becomes “Chat” in same sidebar position; PHÉNGOS orb with luminous WHITE Chat accent; main Chat destination and general/specialist selection	Frozen design present; existing assistant code to be adapted
Chat continuity	Canonical persisted multi-thread history, new/resume, general routing, available specialist roles, approval-gated actions and intentional full-page Chat	Frozen concept; real backend selection/role switching must be verified
PHENGOS-002	Calendar/Kitchen top-right domain-colored orb, floating overlapping chat window, no bottom bar/date chip, no squeezing/no Focus Mode	Frozen design present; no application code changes
CAL-001	Preserve current Calendar grid and insights, Overview/Daily	Frozen update; not rebuild

Decision area	Verified requirement	Status / dependency
	List/Planning, correct Save edit and migrate long forms	
KIT-001	Preserve current Kitchen; Overview/Fridge/Meals/Recipes/S hopping; agent-prepared today meal, food journal/recipe history; fix blank under meal	Frozen update; not rebuild
COPY-001 / MOTION-001	No promotional subtexts; relevant domain accents, slow faded Kitchen tabs, reduced-motion and accessible focus	Frozen universal copy and Kitchen motion
Photos	N01–N02, C01–C02, K01–K08, CH01–CH04; CH05–CH06 are full historical boards with explicit supersession notes	18 total PNG files; all referenced in implementation visual register
Next design area	Home / PHÉNGOS Overview, not a new Chat design cycle	Open; no freeze yet
FIT-001	Exercise-mapped neon body, training/report, progress/goal trajectories, metric input; correct axis orientation despite photo.	Frozen UI; new projection + secondary-muscle mapping discuss with user
FIN-001	Account/balance led overview, income-vs-spend by month, safe-to-spend, due bills, compact liabilities, Transactions/Budgets/Debts/Analytics.	Frozen UI; banking is previously planned; liability data model discuss with user
MOV-001	Life→Movies; Overview/Watchlist/Logs; poster library and diary; star stats, favorites/lists and canonical movie import.	Frozen UI; lists/favorites + stats read projection discuss with user
New reference photos	FIT01–03, FIN01–02, MOV01–02; screenshot corrections are mandatory.	Seven new JPEG files; total 25 reference images

Image precedence: explicit latest FREEZE → active implementation sections → approved target images → historical concept references. Current repository remains truth for implemented behavior, not a source of permission to override the frozen target.

14 New freezes: Fitness, Finance and Movies

The user explicitly approved the seven accompanying screenshots. This freezes their target UI and interaction direction, subject to documented corrections. It does not declare the current code implemented, authorize new banking providers or silently approve proposed data models.

ID / placement	Frozen design scope	Images
FIT-001 / main Fitness	Daily training, deterministic neon muscle focus, agent report, program/workout, readiness, calories, multi-metric progress; metric-entry workplace	FIT01–FIT03
FIN-001 / Life → Finance	Account/balance-first financial overview, monthly financial trajectories, budgets/categories, compact Klarna/card obligations, transaction and manual workspaces	FIN01–FIN02
MOV-001 / Life → Movies	Movie-first dashboard without personal profile, Overview/Watchlist/Logs, average stars and activity chart, favorites, poster-based library, diary and lists	MOV01–MOV02

15 Backend editor decision queue — OPEN, discussion required

The following features are NOT granted automatic backend scope by the UI freeze. The backend/Next Implementation File editor must discuss definitions and data contracts with the user before adding them. Proven existing capabilities must be reused.

Ticket	Requested / proposed extension	Decision gate
BE-FIT-01	Versioned muscle taxonomy and weighted primary/secondary muscle mapping for exercise-specific neon rendering; read projection for today	Discuss muscle detail, mapping maintenance and intensity meaning
BE-FIT-02	Historical goal-trajectory projection + multi-metric strength/weight/body-fat/lean-mass progress; confidence and sample thresholds	Discuss goal-specific metrics, date estimates and progress-index formulas
BE-FIT-03	Metric correction/edit with provenance and session-history query scaling if existing APIs insufficient	Discuss source selection, edit policy and data fields

Ticket	Requested / proposed extension	Decision gate
BE-FIN-01	Liability/installment register for Klarna and credit cards; deduplicated bank transactions, due dates and payments	Discuss source and exact accounting semantics
BE-FIN-02	Time-bucket income/spend and timestamped actual balance series; historical 6/12-month trajectory and forecast assumptions	Reuse planned BankBalanceSnapshot; discuss backfill and forecasting
BE-MOV-01	Explicit pinned/ordered favorites and custom named list records; library rating summary endpoint if client data insufficient	Discuss favorite limit, ordering, list names and statistics definition

16 Sequencing after this update

- Frozen design handoff ready for Fitness, Finance and Movies; implementation agents first verify current HEAD, API contracts and pending Next Implementation milestones before editing.
- Home remains explicitly deferred; Chat, Calendar and Kitchen stay frozen and are unaffected by this change.
- Learning, Life parent/Goals/Social, Developer detail and responsive verification remain open for future sessions.
- When backend additions are selected, ask user to approve the backend changes before the editor revises Next Implementation File. Do not hide new models behind “UI-only” work.

17 Current FREEZE reconciliation and execution gate

- Approved seven photo references (FIT01–FIT03, FIN01–FIN02, MOV01–MOV02) are bundled in Photos/. Their sample accounts, medical/fitness metrics, goal dates, posters and amounts are not user data. These are approved visual direction images, subject to explicit written corrections.
- Correct graph axes across all Fitness screens: LEFT VERTICAL Y = selected progress/measurement; BOTTOM HORIZONTAL X = time and dated observations. The selected metric may be generalized goal attainment, strength, body fat, weight or lean mass where actual data exists. Do not assume a weight-loss goal.
- No extra navigation: preserve single existing dark-neon fading horizontal tab design (K08 reference) and the global sidebar. Fitness is main Fitness; Finance and Movies remain inside Life. Movies is precisely Overview / Watchlist / Logs. Finance has one top strip; manual entry in Transactions, not duplicate strips.
- Backend evidence was checked against GitHub master: current Fitness endpoints have programs/templates/sessions/sets/body measurements/trends/goals/readiness; Finance has transaction-derived overview, budgets, recurring/safe-to-spend; Movies has canonical viewing/ratings/notes/watchlist and import preview-confirm. The Next Implementation File further specifies conditional bank connection, media and Letterboxd/TMDB improvements which must not be claimed shipped.
- Backend discussion queue only (not implementation authorization): FIT-BE-01/02/03, FIN-BE-01/02, MOV-BE-01/02. Ask user before amending Next Implementation File. Existing models/services are the first option; no duplicate subsystems.
- Package discipline: exactly ONE master ZIP containing LIFE_OS_UI_ROADMAP.docx, LIFE_OS_UI_IMPLEMENTATION.docx and Photos/. Future UPDATE mutates this same package, preserving prior freezes and replacing superseded concepts carefully.